Deggendorf Institute of Technology

Master's Thesis

Automatic Program Generation for a Sheet-Metal Handling Robot in Front of a Press Brake

Sooraj Sudhakaran

2026

TECHNISCHE HOCHSCHULE DEGGENDORF
Campus Cham

Master's Thesis

Deggendorf Institute of Technology, Campus Cham

Faculty of Applied Natural Sciences and Industrial Engineering

Master Mechatronics & Cyber-Physical Systems

Automatische Programmerzeugung für einen Blechhandhabungsroboter vor einer Abkantpresse

Automatic Program Generation for a Sheet-Metal Handling Robot in Front of a Press Brake

master's thesis to obtain the academic degree:

Master of Engineering (M.Eng.)

submitted by: Sooraj Sudhakaran, Cham, 12305322

first examiner: Prof. Dr. Matthias Górka

supervisor: Martin Naumann

Cham, *May 5, 2026*

Declaration of Original Authorship

Name of student: Sooraj Sudhakaran

Name of first examiner: Prof. Dr. Matthias Górka

If external: Name of supervisor: Martin Naumann

Title of the master's thesis: Automatische Programmerzeugung für einen Blechhandhabungsroboter vor einer Abkantpresse

1. I hereby declare that this final thesis is my own work, has not been submitted for any other degree at any other university or institution, does not contain or use any sources or resources other than those referenced, and that all direct and paraphrased quotes have been duly cited as such.

Cham, (05.04.2026) ....

Signature of student

2. I hereby consent to my completed final thesis being made available to a broader public through the library of Deggendorf Institute of Technology.

☐ yes ☐ no

If Yes:
I certify and warrant that I am the sole owner and holder of all rights to the final thesis, including the right to use the tables, figures, illustrations, plans, or similar, submitted therein and that no third-party rights or claims, nor any statutory provisions, will be infringed as a result of its publication.

Cham, (05.04.2026) ....

Signature of student

To be filled in by the first examiner if the author agrees with public accessibility of the master's thesis

Collection of a copy of the final thesis by the library, including the lending of the copy, is hereby endorsed / not endorsed.

☐ endorsed ☐ not endorsed

Cham, (05.04.2026) ....

Signature of first examiner

Declaration on the Use of Generative AI Tools

Affidavit

I hereby declare that I have prepared this thesis independently and without unauthorised assistance. All sources and references used have been properly acknowledged. This thesis has not been submitted in the same or similar form for any other examination.

Use of Generative Artificial Intelligence (AI)

During the preparation of this thesis, generative AI tools (ChatGPT, OpenAI, GPT-5 model family) were used in a limited and supportive capacity. This included language refinement, grammar correction, text structuring, formatting support, LaTeX assistance, and consistency checking.

AI tools were not used to generate experimental data, perform technical analysis, design the system, implement algorithms, or derive scientific conclusions. All technical content, including system design, implementation, evaluation, and interpretation of results, was developed independently by the author.

All AI-assisted content was reviewed and validated by the author, who assumes full responsibility for the submitted work. A record of AI usage is documented in the appendix prompt log.

Final Statement

I confirm that the use of AI tools complies with the applicable thesis guidelines. I am aware that any omission or misrepresentation may be considered academic misconduct.

Student Name: Sooraj Sudhakaran

Matriculation Number: 12305322

Degree Programme: Master Mechatronics & Cyber-Physical Systems

Thesis Title: Automatic Program Generation for a
Sheet-Metal Handling Robot in Front of a Press Brake

Place and Date: Cham, May 5, 2026

Signature: _____

Acknowledgements

I would like to express my sincere gratitude to my university supervisor, Prof. Dr. Matthias Górka, for his valuable guidance, continuous support, and constructive feedback throughout the development of this thesis.

I am equally grateful to my company supervisor, Martin Naumann, for his practical insights, encouragement, and support during my work at KEBA Group.

I would like to extend my sincere appreciation to Saif Abdullah (KEBA Group, drag&bot team) for his dedicated support in learning the drag&bot software environment and for his assistance in organising and facilitating the successful execution of this thesis.

I gratefully acknowledge KEBA Group for providing the necessary infrastructure and the opportunity to carry out this thesis within an industrial environment. I also appreciate the cooperation and support of all KEBA team members.

My sincere thanks go to the drag&bot team for their valuable support in software and robotic simulation, which significantly contributed to the successful completion of this work.

I also acknowledge Deggendorf Institute of Technology for providing the academic environment and opportunity to pursue this thesis.

I would like to thank my friends for their support in accessing relevant research resources through the University of Stuttgart.

Finally, I am deeply grateful to my family, friends, and colleagues for their continuous encouragement and moral support throughout this project.

Abstract

Press brake sheet metal bending is a key manufacturing process; however, despite the availability of robotic programming tools, implementations often rely on manual programming. Operators must define motion sequences, teach waypoints, and adapt programs for each new workpiece, resulting in increased effort and limited flexibility.

This thesis presents an automated planning system for robotic press brake bending. The proposed approach integrates workpiece orientation determination, constraint-based grip planning, automatic regrip handling, and trajectory generation within a modular pipeline driven by geometric input and configuration data. The system generates execution-ready robot programs without manual waypoint teaching and incorporates validation steps to ensure safe and feasible operation, requiring only bend-side selection from the operator.

The system is evaluated in a virtual simulation environment using repeated workflow and feasibility trials. The evaluation covers nominal direct bending on Side A, opposite-side bending on Side E, geometry variation, workpiece position variation, regrip-required scenarios, and failure-prone conditions. The results show that automated generation substantially reduces preparation effort while maintaining feasible simulated execution within the evaluated scenario set.

Keywords: robotic press brake bending; automatic robot program generation; grip planning; regrip planning; trajectory generation; flexible manufacturing.

Contents

Acknowledgements **v**

Abstract **vii**

List of Figures **xi**

List of Tables **xiii**

List of Code Snippets **xv**

List of Abbreviations **xvii**

List of Symbols **xix**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Statement 2

1.3 Research Objectives 2

1.4 Research Hypothesis 3

1.5 Scope of Work 3

1.6 Methodology Overview 4

1.7 Thesis Structure 5

2 State of the Art and Literature Review **7**

2.1 Fundamentals of Industrial Robotics 7

2.2 Sheet Metal Bending and Press Brake Systems 7

2.2.1 Process Characteristics and Automation Challenges 8

2.3 Manual and Teach Pendant Programming 9

2.3.1 Limitations of Manual Teach Pendant Programming 10

2.3.2 Transition Towards Automated Programming 10
2.4 Offline Programming Approaches for Industrial Robots 10
2.4.1 Digital Twin and Simulation-Based Programming 11
2.5 Automatic Robot Program Generation and Rule-Based Approaches . . 12
2.5.1 Task-Level Programming and Abstraction 12
2.5.2 Rule-Based and Constraint-Based Planning 12
2.6 Robotic Sheet Metal Bending Systems 13
2.6.1 Regrip Planning in Sheet Metal Bending 14
2.6.2 Grasp Planning for Sheet Metal Handling 15
2.6.3 Trajectory Planning in Robotic Bending Systems 15
2.6.4 Limitations of Existing Robotic Bending Systems 15
2.7 Industrial Software Platforms and System Implementations 16
2.7.1 Simulation and Offline Programming Environments 16
2.7.2 Graphical Programming and Integrated Execution Systems . . 17
2.7.3 Summary and Limitations of Industrial Platforms 17
2.7.4 Critical Comparison of Existing Approaches 18
2.8 Research Gap and Motivation 20

3 System Concept and Methodology 21

3.1 Problem Definition and Objectives 21
3.2 Requirement Definition and Management 22
3.2.1 Functional Requirements 22
3.2.2 Non-Functional Requirements 23
3.2.3 Software Platform and Tool Requirements 23
3.2.4 System Constraints and Assumptions 26
3.2.5 System Traceability 27

4 Development of Concepts 29

4.1 Purpose and Scope of Concept Development 29

4.2 Concept A – Structured Operator Workflow with Guided Teach-In Programming . . . 29
4.3 Concept B – Automatic Program Generation with Integrated Constraint Validation . . . 32
4.4 Concept C – CAD-Based Offline Programming with Simulation . . . 34
4.5 Concept Decision Using the Weighted Sum Model . . . 37

5 Implementation 43

5.1 Introduction and Scope . . . 43
5.2 System Architecture, Interfaces, and Configuration . . . 46
5.2.1 Layered Software Architecture and Module Responsibilities . . . 46
5.2.2 Runtime Communication and Configuration . . . 50
5.2.3 Operator Interface and Three-Dimensional Sheet Viewer . . . 54
5.3 Simulation Environment and Pose Acquisition . . . 58
5.4 Safety Validation and Runtime Monitoring . . . 62
5.5 Orientation and Grip Planning . . . 65
5.5.1 Zone-Based Orientation Admissibility and Pickup Validation . . . 65
5.5.2 Three-Level Orientation Selection Strategy . . . 66
5.5.3 Grip Position Planning and Feasibility Validation . . . 69
5.6 Gripper State, Regrip, and Opposite-Side Handling . . . 75
5.6.1 Regrip Planning and Execution . . . 76
5.6.2 Opposite-Side Bending Workflow . . . 79
5.7 Trajectory Generation and Execution . . . 83
5.7.1 Task Sequence Assembly for Execution . . . 83
5.7.2 Trajectory Structure and Interpolation . . . 84
5.7.3 Execution Validation and Path Reduction . . . 85
5.8 Implementation Summary . . . 87

6 Evaluation and Validation 91

6.1 Introduction to Evaluation . . . 91

6.2 Evaluation Methodology 91
6.2.1 Measurement Principles 91
6.3 Experimental Setup and Scenarios 94
6.4 Data Analysis and Graph Interpretation 95
6.4.1 Overall Time Performance 95
6.4.2 Effect of Workpiece Position 96
6.4.3 Regrip and Opposite-Side Workflow Complexity 97
6.4.4 Planning-Time Interpretation 99
6.5 Requirement Verification 99
6.6 Success and Failure Analysis 101
6.7 Gap Analysis 103
6.8 Improvement Plan 104
6.9 Summary of Findings 105

7 Conclusion and Future Work 107

7.1 Conclusion 107
7.2 Contributions 108
7.3 Limitations 109
7.4 Future Work 109
7.5 Final Remarks 111

References 113

Appendix 117

A.1 AI Usage Documentation 117
A.2 Prompt Log and Supporting Files 117
A.3 Supplementary Evaluation Documentation 117
A.4 Requirement Traceability and Compliance Tables 118
A.5 Hardware Implementation Report 123

List of Figures

2.1 Robotic press brake bending system with integrated robot handling . . 8

2.2 Teach pendant interface for manual robot programming. 9

2.3 Robotic sheet metal bending system with integrated robot and press brake 14

4.1 Detailed function-block configuration for manual motion and gripper sequencing in Concept A. 30

5.1 Complete end-to-end system workflow 45

5.2 Communication architecture of the implemented system 52

5.3 Operator-facing sheet selection dashboard embedded in the drag&bot Operator Cockpit, showing bend-side selection, workflow status, mode selection, and the three-dimensional workpiece preview. 55

5.4 Operator workflow 57

5.5 Kinematic structure and coordinate-frame assignment of the UR5e manipulator 59

5.6 Configured TCP of the UR5e with the modelled two-cup vacuum gripper used for sheet handling. 60

5.7 Virtual test environment in the drag&bot operator panel 60

5.8 Spatial zone classification of the robot workspace based on the sheet centroid position. 62

5.9 Grip planning workflow 73

5.10 Opposite-side flip workflow 81

6.1 Comparison of mean total process time including preparation for manual and automated workflows. The figure separates preparation time and simulated execution time, showing that the automated workflow reduces the total process time mainly by reducing preparation effort. . 96

6.2 Effect of initial workpiece position on mean total process time for manual and automated workflows. Plane A and Plane B denote two placement conditions with different reachability constraints. 97

6.3 Mean regrip count for manual and automated workflows under different workpiece placement conditions. The count excludes the initial pickup and represents intermediate regrips required to maintain reachability and orientation feasibility. 98

6.4 Automated success and failure rate across six feasibility scenarios. Each scenario contains ten repeated runs and is classified according to completion, failure, or controlled-stop behaviour. 102

List of Tables

2.1 Critical comparison of existing robotic programming and planning approaches with the proposed thesis approach. 19

3.1 Functional Requirements for Automatic Robot Program Generation. . . 24

3.2 Non-Functional Requirements – NFR-01 to NFR-05 with acceptance criteria and verification methods. 25

4.1 Weighted Sum Model (WSM) evaluation of Concept A, Concept B, and Concept C. 39

5.1 End-to-end data flow from workpiece and process input to executable robot commands. 46

5.2 Grouped module contract overview showing the main inputs, outputs, and stop conditions across the implemented system layers. 49

5.3 Principal AnyRobot runtime interfaces used by the implemented communication layer. 51

5.4 Mapping between implemented functions and the Chapter 6 evaluation evidence. 87

5.5 Failure Mode and Effects Analysis (FMEA) of the robotic bending system. 89

6.1 Functional requirement verification based on evaluated scenarios and observed trial outcomes. 100

6.2 Non-functional and software requirement verification based on evaluated scenarios and observed trial outcomes. 101

1 Evaluation trial structure used for the final simulation-based validation. 118

2 Requirement Traceability Matrix – functional requirements 119

3 Requirement Traceability Matrix – non-functional and system requirements 120

4 Requirement compliance mapping – functional requirements 121

5 Requirement compliance mapping – non-functional and system requirements 122

6 Hardware and platform configuration used for the simulation-based implementation. 123

List of Code Snippets

5.1 Pseudocode for runtime command dispatch and result handling. 52
5.2 Pseudocode for three-level orientation selection. 68
5.3 Pseudocode for grip candidate search and selection. 71
5.4 Pseudocode for regrip decision and insertion logic. 77
5.5 Pseudocode for waypoint generation and path reduction. 84

List of Abbreviations

API	:	Application Programming Interface
CAD	:	Computer-Aided Design
CAM	:	Computer-Aided Manufacturing
Digital IO	:	Digital Input/Output
DNB	:	drag&bot
DOF	:	Degree of Freedom
FMEA	:	Failure Mode and Effects Analysis
FR	:	Functional Requirements
HI	:	Human Interaction
HI Dialog	:	Human Interaction (drag&bot-specific dialog service)
HMI	:	Human–Machine Interface
HTTP	:	Hypertext Transfer Protocol
HTTPS	:	Hypertext Transfer Protocol Secure
IO	:	Input/Output
IP	:	Internet Protocol
JSON	:	JavaScript Object Notation
LiDAR	:	Light Detection and Ranging
LIN	:	Linear motion command
MAC	:	Message Authentication Code
NFR	:	Non-Functional Requirements
OLP	:	Offline Programming
PTP	:	Point-to-point motion command
ROS	:	Robot Operating System
SHA-512	:	Secure Hash Algorithm 512-bit
SLAM	:	Simultaneous Localisation and Mapping
SR	:	Software Requirements
STL	:	Standard Tessellation Language
TC	:	Test Case
TCP	:	Tool Centre Point
UI	:	User Interface
UR5e	:	Universal Robots 5e (robot model)
URDF	:	Unified Robot Description Format
WebSocket	:	Web-based communication protocol for real-time data exchange defined in RFC 6455
WSM	:	Weighted Sum Model
YAML	:	YAML Ain’t Markup Language

List of Symbols

Symbol	Description	Unit
A_{cup}	Contact area of one suction cup	m^2
a_{xy}	Resultant dynamic acceleration in the horizontal plane	$\text{m}\,\text{s}^{-2}$
a_z	Dynamic acceleration in the vertical direction	$\text{m}\,\text{s}^{-2}$
α	Euler angle corresponding to rotation about the Z-axis in the ZYX convention	rad
β	Euler angle corresponding to rotation about the Y-axis in the ZYX convention	rad
d_{cup}	Diameter of one suction cup	m
d_{face}	Reference-face angular offset used for orientation correction	rad
D	Detectability rating in the FMEA	n.u.
Δ_{ang}	Maximum angular deviation between current and reference fixture poses	rad
Δ_{pos}	Euclidean positional deviation between current and reference fixture poses	m
ΔP	Pressure differential between ambient pressure and vacuum pressure	Pa
ΔRX	Rotational deviation about the X-axis	rad
ΔRY	Rotational deviation about the Y-axis	rad
ΔRZ	Rotational deviation about the Z-axis	rad
Δx	Positional deviation in the X direction	m
Δy	Positional deviation in the Y direction	m
Δz	Positional deviation in the Z direction	m
client	Client identifier used during authentication	n.u.
dest	Destination field used during authentication	n.u.
end	Validity end field used during authentication	n.u.
η	Efficiency factor accounting for leakage and surface conditions	n.u.
F_{required}	Required holding force under gravitational and dynamic loading	N
F_{vacuum}	Available vacuum holding force	N
g	Gravitational acceleration	$\text{m}\,\text{s}^{-2}$
γ	Euler angle corresponding to rotation about the X-axis in the ZYX convention	rad
λ	Interpolation parameter ranging from 0 to 1	n.u.

Symbol	Description	Unit
level	Access level string used during authentication	n.u.
m	Mass of the handled workpiece	kg
MAC	Message authentication code generated for the AnyRobot authentication handshake	n.u.
n	Number of active suction cups	n.u.
N	Number of trials used for an average calculation	n.u.
N_{failure}	Number of failed trials in a scenario	n.u.
N_{success}	Number of successful trials in a scenario	n.u.
N_{total}	Total number of evaluated trials in a scenario	n.u.
O	Occurrence rating in the FMEA	n.u.
$\mathbf{p}(\lambda)$	Interpolated translational position along a trajectory segment	m
$\mathbf{p}_s$	Translational component of the start pose	m
$\mathbf{p}_t$	Translational component of the target pose	m
R	Rotation matrix	n.u.
R_{new}	Rotation matrix after applying the reference-face offset correction	n.u.
R_{raw}	Raw rotation matrix from the tracked pose	n.u.
R_x, R_y, R_z	Elementary rotation matrices about the X-, Y-, and Z-axes	n.u.
R_{ij}	Element of a rotation matrix in row i and column j	n.u.
rand	Random value used during authentication	n.u.
RPN	Risk Priority Number in the FMEA	n.u.
R_{failure}	Failure rate for an evaluated scenario	%
R_{success}	Success rate for an evaluated scenario	%
RX	Rotation about the X-axis	rad
RX'	Corrected or fallback rotation about the X-axis	rad
RY	Rotation about the Y-axis	rad
RY'	Corrected or fallback rotation about the Y-axis	rad
RZ	Rotation about the Z-axis	rad
RZ'	Corrected or fallback rotation about the Z-axis	rad
RZ_{max}	Upper admissible limit for rotation about the Z-axis	rad
RZ_{min}	Lower admissible limit for rotation about the Z-axis	rad
S	Severity rating in the FMEA	n.u.
SF	Safety factor for the required holding force	n.u.
SHA-512	Hashing function used for generating the authentication token	n.u.
secret	Shared runtime secret used during authentication	n.u.
$s(u)$	Minimum-jerk progression function	n.u.

Symbol	Description	Unit
c_{regrip}	Number of intermediate regrip operations after the initial pickup	n.u.
t_{exec}	Simulated execution time of a prepared workflow	s
t_{plan}	Automated planning time from planning trigger to accepted or rejected sequence	s
t_{prep}	Preparation time before simulated execution starts	s
t_{total}	Total process time calculated from preparation and execution time	s
$\mathbf{T}_s$	Start pose of a trajectory segment	n.u.
$\mathbf{T}_t$	Target pose of a trajectory segment	n.u.
t	Session timestamp used during authentication	s
u	Normalised progression variable for minimum-jerk interpolation	n.u.
x	Translation along the X-axis	m
x_i	Measured value of a quantity in trial i	context-dependent
x_{th}	Threshold value for workspace classification in the X direction	m
$\bar{x}$	Arithmetic mean of a measured trial quantity	context-dependent
y	Translation along the Y-axis	m
y_{th}	Threshold value for workspace classification in the Y direction	m
z	Translation along the Z-axis	m

1 Introduction

1.1 Background and Motivation

Sheet metal bending is a common manufacturing process for forming sheet metal parts [1], [2]. In such processes, the bend sequence, tooling conditions, workpiece handling, and the position of the workpiece relative to the machine are closely interconnected.

To reduce human effort, industrial robots are replacing human operators in press brake cells. Consequently, the precision of task planning becomes more demanding, as gripping positions, repositioning, and robot motion planning must be considered [1], [2].

Although robotic operation can reduce direct manual handling, programming these systems still requires considerable effort. In particular, after each operation, the process steps must be adapted by the robot. Therefore, robot programming must be performed in a precise and consistent manner to enable accurate adaptation to new parts [3], [4]. In conventional robotic bending programs, engineers design the robot path manually and generate motion sequences and process logic for each workpiece. This approach can be efficient for determining the shortest path under fixed conditions; however, it becomes inefficient when the sheet geometry, initial placement, or required bending sequence changes. In such cases, the program must be reprogrammed for each individual part and verified before execution in accordance with safety guidelines. This procedure is time-consuming and requires expert knowledge in robot systems, motion planning, and process control. Furthermore, it increases overall processing time [1], [2], [3]. The increasing demand for flexible manufacturing necessitates a reduction in manual programming. In robotic press brake bending programs, tasks such as grip position calculation, motion sequence generation, pose generation, regrip sequencing, and validation must be performed in a more precise and systematic manner while reducing manual programming effort. Motivated by this challenge, this thesis investigates how automated program generation and planning can support faster and more repeatable task execution [1], [3].

1.2 Problem Statement

Compared to manual sheet metal bending, the use of robots reduces physical effort; however, despite their adoption in press brake bending, programming still requires significant time, manual effort, and skilled expertise to implement the required process logic. This results in high programming effort, limited accessibility for non-expert users, increased process time, and stringent verification requirements [3], [4].

A further limitation is the adaptation to changes in the shape, orientation, or geometry of the workpiece. In robotic bending, such changes affect grasp feasibility, reachability, and the required bending sequence. In manual programming, these adjustments require reprogramming and reteaching of the robot, which reduces flexibility [1], [2].

Manual programming becomes particularly challenging and time-consuming in regripping and multistage bending operations. These tasks require detailed consideration of robot kinematics, including reachability analysis and determination of appropriate coordinates for workpiece positioning. Any change in the workpiece position requires reteaching the robot with the updated placement. Furthermore, it must be verified that all repositioning steps and subsequent bending operations remain feasible and reachable. When performed manually, this process becomes increasingly complex and difficult to adapt to new workpieces, placements, or process sequences, thereby reducing consistency and repeatability [1], [2].

These limitations highlight the need for a systematic and automated approach to robot program generation, which can evaluate feasibility, safety, and consistency prior to execution and enable repeatable and adaptable operation under varying conditions without extensive manual programming effort.

1.3 Research Objectives

The main objective of this thesis is to develop an automated program generation approach for robotic press brake bending based on geometric input parameters, thereby reducing the need for manual programming and reprogramming effort. In addition, the system enables adaptive handling of workpieces, including changes in position, orientation, and geometry.

A key objective is to integrate multiple planning components into a unified framework. These components include orientation determination, grip position planning, regrip sequence handling, and trajectory generation for robotic operations, all of which are required to maintain feasible and continuous execution of the bending process. Furthermore, the system incorporates structured validation mechanisms

during program execution to ensure feasibility, safety, and consistency.

Finally, the overall objective is to reduce operator dependency, minimise pre-programming time and manual programming effort while maintaining consistent and repeatable system behaviour. By automating these critical planning tasks, the proposed solution aims to support flexibility in robotic bending workflows.

1.4 Research Hypothesis

The research hypothesis of this thesis is:

> If robot-program generation is automated from workpiece geometry, then preparation time decreases while simulated execution remains feasible.

This hypothesis connects the degree of automation with two measurable outcomes: preparation effort and execution feasibility. Preparation effort is evaluated by comparing the manual programming workflow with the automated workflow. Execution feasibility is evaluated through simulation-based validation of the generated robot programs in the drag&bot virtual environment.

The hypothesis is tested within the defined thesis scope, namely rectangular sheet-metal workpieces, the Universal Robots 5e (UR5e) simulation model, and the implemented drag&bot-based virtual press-brake cell. It does not claim validation for all industrial press-brake systems or physical robot deployments.

1.5 Scope of Work

This thesis focuses on the development and evaluation of an automated planning system for robotic press brake bending within a virtual simulation environment. The primary scope of this work is limited to the generation of robot programs and the validation of their execution feasibility using simulated data prior to implementation on a physical robotic system. The physical implementation on a real robotic press brake is beyond the scope of this thesis.

The system is developed and evaluated in simulation using the UR5e manipulator model. The UR5e provides six degrees of freedom, a reach of approximately 850 mm, a payload capacity of 5 kg, and high repeatability, making it suitable for developing and testing the structured robotic handling workflow considered in this thesis [5]. Although other robot models are available within the virtual simulation environment, the UR5e is used as the reference manipulator for consistent program generation and feasibility evaluation. Future extension toward physical robot testing remains outside the scope of this work.

The implementation is tailored to rectangular sheet metal bending tasks with defined geometric properties and specified wall heights. However, this work does not aim to address all possible industrial variations or complex multi-process workflows, such as highly customised production scenarios or integration with additional manufacturing processes.

The evaluation focuses on reducing manual effort and programming time through automation using repeated simulation-based workflow trials and automated feasibility trials. The evaluated scenario set covers nominal direct bending on Side A, opposite-side bending on Side E, workpiece geometry variation, workpiece position variation, regrip-required workflows, and failure-prone conditions. Physical robot validation, material forming accuracy, and certified collision-free industrial commissioning remain outside the scope of this thesis and are considered directions for future work.

1.6 Methodology Overview

The proposed solution follows a structured methodology that combines concept evaluation, modular system design, and systematic validation. The overall approach is designed to ensure that the developed system meets both functional and non-functional requirements while supporting reliable and repeatable automated robot program generation.

Initially, multiple solution concepts are analysed and compared based on a set of defined evaluation criteria. These criteria include the capability to support automated program generation and the extent to which each concept satisfies functional, non-functional, and software requirements. Based on this analysis, concept-based case studies are conducted and reviewed, and the most suitable concept that satisfies the defined requirements is selected for implementation.

The selected concept is then implemented as a modular planning pipeline consisting of distinct components responsible for orientation determination, grip planning, regrip handling, and trajectory generation. To support robust and consistent system behaviour, the architecture is designed using a layered approach, ensuring that different levels of functionality are logically separated. Validation mechanisms are integrated at multiple stages within the pipeline to support safe, consistent, feasible, and repeatable execution of the generated robot programs.

Finally, the system is evaluated through repeated manual-versus-automated workflow trials and automated scenario trials. The workflow comparison quantifies preparation time, execution time, and total process time, while the automated feasibility trials assess success rate, controlled-stop behaviour, planning time, and robustness across nominal, opposite-side, geometry-change, position-change, regrip-required,

and failure-prone scenarios.

1.7 Thesis Structure

The remainder of this thesis is structured as follows:

Because the approved scope of this thesis is simulation-based validation, physical prototype realisation and real robot commissioning are outside the scope of the present work and are addressed as future work. The development, simulation, realisation, and verification activities are therefore organised across Chapters 4 to 6 rather than as separate standalone chapters.

Chapter 2 reviews the state of the art in robotic press brake bending and identifies key limitations of existing approaches.

Chapter 3 defines the system requirements and outlines the methodology used for concept development.

Chapter 4 presents the evaluation of alternative solution concepts and justifies the selection of the proposed approach.

Chapter 5 describes the implementation of the system, including architecture, planning modules, and validation mechanisms.

Chapter 6 evaluates the developed system using repeated workflow and feasibility trials, comparing manual and automated process time and verifying behaviour across nominal, opposite-side, geometry-change, position-change, regrip-required, and failure-prone scenarios.

Finally, Chapter 7 summarises the contributions of the thesis and discusses limitations and directions for future work.

2 State of the Art and Literature Review

2.1 Fundamentals of Industrial Robotics

Industrial robots are central components of modern manufacturing systems, as they provide precise execution, reduce effort and processing time, and increase productivity and quality. According to international standards, industrial robots are reprogrammable and capable of performing multiple tasks with three or more programmable axes [6].

Such systems are widely used in manufacturing operations where stable and repeatable motion is required to ensure high productivity and precision. A typical industrial robot consists of several actuated joints that provide the Degree of Freedom (DOF) capability required to position and orient the end-effector in three-dimensional space. This task is usually defined by specifying the pose of the Tool Centre Point (TCP), which represents the reference point for robot motion execution, while the robot flange serves as the physical interface to which the tool is attached [7].

Robot programming can be understood across different levels of abstraction. At a lower level, the operator manually defines the program by specifying poses, trajectories, and process sequences. At a higher level, the user specifies only the desired output, while the system automatically derives and executes the required motion commands. In industrial practice, manual programming and offline programming are still widely used [3]. Research on higher-level and skill-based programming aims to reduce the need for low-level programming effort [8].

This shift is particularly important for applications that require high repeatability, frequent adaptation to different workpieces, and operation in high-production environments. In applications such as robotic sheet metal bending, the robot must handle varying workpiece geometries, limited workspace, tool access constraints, and required reorientation steps. Consequently, the task becomes more complex and demanding than conventional pick-and-place operations, thereby motivating the need for advanced robot programming methods, press brake automation, and integrated planning approaches.

2.2 Sheet Metal Bending and Press Brake Systems

Sheet metal bending is a simple manufacturing process where a sheet of metal is bent into the required shape through plastic deformation. Sheet metal forming

processes are widely used in industrial manufacturing due to their ability to produce a variety of products with different geometrical shapes [9]. In the case of press brake operations, sheet metal is positioned between the forming tools such as the punch and die.

Industrial robots have been used in modern manufacturing to perform activities such as handling materials, processing, and assembling parts without relying heavily on humans.[10] Figure 2.1 illustrates a typical robotic press brake setup, where an industrial robot is integrated with the bending machine to automate handling operations.



Figure 2.1: Robotic press brake bending system with integrated robot handling [11].

In automated press brake cells, the robot typically picks the flat sheet, positions it at the tooling, supports it during bending, reorients it between process steps, and unloads the final component. This automation can improve repeatability and reduce direct operator involvement, particularly in repetitive production settings [2]. However, the process is difficult to fully automate because each bend alters the physical shape of the part, requiring regripping and repositioning, which in turn necessitate updated grip planning for subsequent process steps.

2.2.1 Process Characteristics and Automation Challenges

The changing geometry throughout the bending process is one of the defining characteristics of sheet metal bending. A motion that is feasible for one bend may become infeasible after the completion of a previous bending step, as the part envelope, tool accessibility, and boundary conditions change. Therefore, process planning must consider the bend sequence, tooling capability, and handling strategy jointly rather than as isolated decisions. These parameters must be continuously adapted in response to geometric changes during the process [1].

For robotic handling, these process characteristics introduce several practical challenges. The robot must maintain a stable grasp of the partially formed workpiece, operate within its reachable workspace, and preserve an appropriate orientation for subsequent operations. In many cases, a single grasp is not sufficient to complete

the full bending sequence due to varying bending sides and reachability limitations. To address this, the workpiece must be temporarily placed down and regrasped in a different configuration. This requires the generation of additional trajectories and coordinated motion plans to ensure that the process can be completed without violating kinematic and reachability constraints.

These issues are introduced here as process-level constraints; the detailed planning problems of regripping, grasp selection, and trajectory generation are discussed later in Section 2.6.

2.3 Manual and Teach Pendant Programming

There are several methods for programming industrial robots in manufacturing environments. One common approach is manual programming using a teach pendant. In this approach, the operator defines the robot's desired positions by entering Cartesian or joint coordinates. These positions are stored as waypoints and executed in a predefined sequence, which must be arranged manually using a robot controller or through a virtual simulation environment [12].

For this purpose, a robotic controller and Human–Machine Interface (HMI) devices, as shown in Figure 2.2, or a virtual simulation environment are required. Teach pendant programming remains useful because it provides the operator with direct control over the robot and allows real-time verification through Cartesian movements or joint-based inputs via HMI devices. For stable, repetitive, and well-defined tasks, this method is practical and reliable.

However, when the process involves a large number of motion coordinates or requires frequent adaptation to new workpiece geometries and changing workpiece placements, significant limitations become apparent.

Figure 2.2 illustrates a typical teach pendant interface used for manual robot programming.

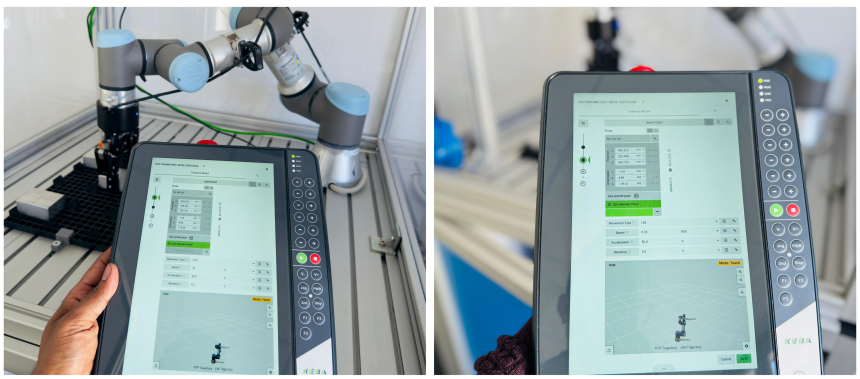


Figure 2.2: Teach pendant interface for manual robot programming.

2.3.1 Limitations of Manual Teach Pendant Programming

The main limitation of manual teach pendant programming is the high level of expert effort required. The operator must have a thorough understanding of robot kinematics, safety guidelines, coordinate systems, robot specifications, tool frames, reachability, and motion constraints.

As task complexity increases, the number of required waypoints also increases. To achieve smooth motion, many intermediate waypoints must be defined, which significantly increases programming effort. Consequently, the programming process becomes slower, highly dependent on individual expertise, and more time-consuming for program generation.

Manual programming also requires frequent reprogramming when the geometry of the workpiece changes. Even if the overall process logic remains similar, the robot program often needs to be adapted, which further increases effort. This makes manual programming inefficient for flexible, high-productivity environments with high part variation [3]. In addition, constraints related to reachability, safety, and path feasibility must be continuously verified, further increasing the complexity of the programming process.

2.3.2 Transition Towards Automated Programming

Robotic sheet metal bending clearly highlights these limitations. The constraints of manual programming have motivated the development of approaches that increase the level of abstraction. Instead of manually defining every pose, engineers increasingly employ virtual commissioning, offline programming, task-level programming, and rule-based planning to reduce direct interaction with the robot.

These approaches do not eliminate engineering effort entirely; however, they shift a significant portion of the workload from manual waypoint teaching to model-based planning and validation, thereby improving consistency, repeatability, and overall efficiency. These challenges have further led to the adoption of advanced programming strategies, particularly virtual commissioning and offline programming, which shift programming effort from the shop floor to a simulation-driven environment. The following section discusses these approaches in detail.

2.4 Offline Programming Approaches for Industrial Robots

Offline Programming (OLP) approaches enable robot programs to be prepared and tested in a virtual environment before execution on a physical robot. Engineers use Computer-Aided Design (CAD) models and simulation tools to define robot motions

and generate executable code. These models allow engineers to define paths, evaluate reachability, identify errors, and ensure compliance with safety guidelines. By shifting programming activities to a virtual simulation environment, setup time is reduced and trial-and-error during production is minimised [3].

Recent developments in industrial robotics emphasize flexibility and rapid adaptation to new tasks. Offline programming supports these requirements by enabling efficient reprogramming and validation without interrupting ongoing production processes. In applications such as robotic sheet metal bending, offline programming plays a critical role due to the complexity of the process. Each bending operation changes the geometry of the workpiece, requiring careful re-planning of robot motion. Simulation-based validation in offline programming environments allows engineers to evaluate these constraints prior to deployment, thereby improving reliability and process efficiency.

Despite these advantages, offline programming does not fully resolve the programming challenges. The quality of the generated program depends on the accuracy of the virtual model and its calibration with the physical system. Deviations between the virtual and real environments may require additional adjustments during commissioning. Furthermore, many offline programming tools still require the engineer to manually define task sequences, waypoints, and process logic. Although the virtual environment can validate programs without a physical robot, it does not inherently determine optimal grasp positions or automatically adapt to frequent changes, often requiring reprogramming.

These limitations reduce the effectiveness of conventional offline programming for processes such as robotic sheet metal bending, where task geometry and constraints evolve continuously during execution.

2.4.1 Digital Twin and Simulation-Based Programming

The digital twin concept extends offline programming by linking a virtual representation with the physical robot system. A digital twin supports monitoring, validation, and adjustment by combining model-based simulation with data from the real process. In manufacturing, digital twin approaches are considered part of smart production systems, as they support analysis, synchronization, and cyber-physical integration [13].

In industrial robotics, tools such as RoboDK, ABB RobotStudio, KUKA.Sim, and drag&bot provide virtual environments for programming and validation. These software platforms include robot models that represent kinematic behaviour and enable the detection of errors within the simulation environment. Such capabilities make simulation-based programming particularly valuable for complex robotic cells, where direct trial-and-error programming would be inefficient or pose operational

risks.

However, manual intervention is still required, particularly in defining programming and sequencing operations. These approaches do not fully eliminate programming challenges, as waypoints and programs must still be manually defined and implemented. In addition, tasks such as regripping and reorientation often require manual reprogramming, which highlights the limitations of these methods in achieving fully automated and adaptable robot programming.

2.5 Automatic Robot Program Generation and Rule-Based Approaches

Automatic robot program generation aims to reduce the need for manual programming by deriving robot actions from task descriptions, process parameters, geometric inputs, or predefined rules. Instead of explicitly programming every motion, as in manual teach pendant or conventional offline programming methods, the system computes feasible actions based on available information and executes them according to defined rules and constraints.

2.5.1 Task-Level Programming and Abstraction

Task-level programming is a key approach used in rule-based systems to automate robot programming. In this approach, the user specifies the desired task outcome, while the system determines how the task should be executed. Early work introduced abstraction layers and symbolic task descriptions to separate task intent from low-level robot motion [14].

Recent approaches utilise skills, motion primitives, and reusable task modules. Operations such as grasping, placing, or constrained motion are represented as modular skills, which can be combined to execute complex robotic programs [8]. While this improves flexibility and reduces low-level programming effort, the system still requires mechanisms to select valid actions based on workpiece constraints and to verify whether the resulting motions are feasible and executable.

2.5.2 Rule-Based and Constraint-Based Planning

Rule-based planning is commonly applied in industrial robotics by executing robot programs based on predefined rules. These rules define how a workpiece should be handled during the process, including allowable orientations, grasp configurations, and motion types. Constraint-based planning extends this concept by explicitly

evaluating geometric, kinematic, and process-related conditions before selecting a solution [3].

Such methods are widely used because they provide deterministic behaviour in well-defined applications. However, as product variability increases in terms of geometry, positioning, and process constraints, the engineering effort required to define, adapt, and validate these rules also increases. In robotic sheet metal bending, this issue is particularly significant because bend sequencing, tooling, grasp selection, and robot motion planning are closely coupled [1], [3].

For complex tasks, the main challenge is not only decision-making but also the reliable coordination of multiple planning components within the process. Grasp planning, regrip planning, trajectory generation, and reachability checking are interdependent and influence each other. If these components are addressed independently, the resulting overall process may become infeasible. Therefore, research in robotic manipulation emphasizes the importance of integrated planning frameworks, particularly in scenarios where grasping and motion constraints are tightly coupled [15].

2.6 Robotic Sheet Metal Bending Systems

Robotic sheet metal bending systems combine an industrial manipulator with a press brake to automate the handling and positioning of sheet metal parts. The robot picks the sheet, aligns it with the tooling, supports it during the bending process, and prepares subsequent process steps. This integration reduces direct human handling and effort while enabling precise product output; however, it also introduces planning challenges that are more complex than conventional material transfer tasks.

The robot typically picks the sheet placed in front of the press brake, as shown in Figure 2.1, and supports the workpiece during bending, as illustrated in Figure 2.3. This setup represents a typical robotic bending system, where the robot operates in coordination with the press brake to execute a sequence of manipulation and forming operations.

The process typically starts with a flat sheet, and after each bending operation, the geometry of the workpiece changes. This affects grasp accessibility, tool clearance, and reachable robot configurations, and some grasp positions may become unreachable due to kinematic constraints. Research in robotic bending has shown that successful automation depends on coordinating process planning with robot motion planning [1]. The following subsections discuss the main planning aspects relevant to this thesis.

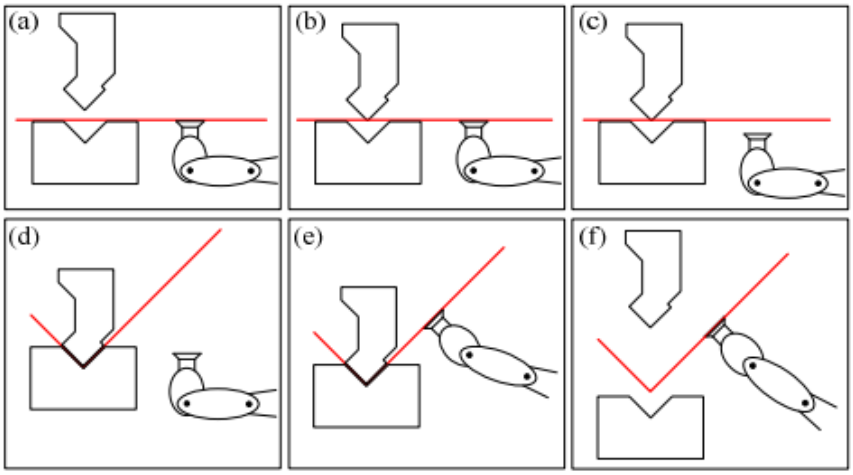


Figure 2.3: Robotic sheet metal bending system with integrated robot and press brake [4].

2.6.1 Regrip Planning in Sheet Metal Bending

Regripping is often necessary when the robot cannot execute all bending steps using a single grasp. The required orientation for subsequent bends may conflict with the initial grasp, or the formed geometry may make the original grasp unsuitable. In such cases, the robot places the workpiece at an intermediate location and performs a regrip in a new configuration.

From a robotic perspective, a valid regrip sequence must consider grasp stability, reachability, and the availability of intermediate placements [16]. However, in rule-based approaches, regrip planning is often executed through predefined sequences or manually designed rules, which are reliable for known parts and geometries but reduce flexibility when the workpiece geometry or bend sequence changes.

The challenge is that regrip planning is not limited to selecting a new grasp. A regrip decision directly affects subsequent trajectory generation and the accessibility of future bending steps. Therefore, regrip planning cannot be treated independently from motion planning. In robotic press brake bending, tooling, grasping, and motion planning are tightly coupled, and a valid bending strategy must be evaluated across the entire process sequence rather than at isolated steps [1].

In existing workflows, grasp selection, regrip decisions, and trajectory validation are often performed as separate steps. This separation can lead to inconsistencies, such as a grasp that is stable but unreachable, a regrip pose that is geometrically valid but difficult to approach, or a trajectory that becomes infeasible after subsequent bending operations. These dependencies highlight the importance of integrated planning for scalable and consistent robotic bending.

2.6.2 Grasp Planning for Sheet Metal Handling

Grasp planning for sheet metal handling is particularly challenging because sheet metal parts are thin, often large relative to their thickness, and may deform or slip if the grasp is not properly selected. Vacuum grippers are commonly used to handle flat sheet surfaces without requiring mechanical clamping around the part. Their performance depends on suction cup placement, contact quality, available holding force, and load distribution during motion [17].

General grasp planning methods consider object geometry, contact conditions, and task constraints when selecting stable grasp configurations [15]. In sheet metal bending, the selected grasp must also ensure that the bend line and tool access regions remain unobstructed. A grasp that is suitable for handling the initial flat sheet may interfere with subsequent bending operations, particularly after the workpiece geometry changes. Therefore, grasp planning must be integrated with the process sequence rather than being based solely on the initial geometry.

2.6.3 Trajectory Planning in Robotic Bending Systems

Trajectory planning in robotic bending systems must generate motions that are collision-free, reachable, and compatible with process constraints. The robot operates in close proximity to the punch, die, workpiece, and other cell components, leaving limited space for avoidance maneuvers. At the same time, it must maintain appropriate orientation to ensure accurate positioning of the sheet during bending.

Classical trajectory generation methods provide a foundation for smooth and continuous robot motion [18]. However, robotic bending introduces additional complexity due to continuous changes in workpiece geometry. The planner must account for evolving collision conditions, workspace limitations, potential singularities, and coordination with grasp and regrip decisions [1].

In many industrial systems, these challenges are addressed using conservative motion templates or manually defined waypoints. While this improves reliability, it limits adaptability and flexibility when process conditions change.

2.6.4 Limitations of Existing Robotic Bending Systems

Existing robotic bending systems demonstrate that press brake automation is technically feasible; however, several limitations remain. Many systems rely on predefined handling strategies and carefully engineered programs. While effective for stable production scenarios, they are less suitable for flexible environments where part geometry and process requirements frequently change.

Although rule-based approaches reduce effort compared to manual and offline programming, a key limitation is the separation of planning tasks. Grasp planning, regrip planning, trajectory generation, and process validation are often treated independently. This separation can result in fragmented workflows, where decisions that are valid in one stage lead to infeasibility in another. For example, a selected grasp may be stable but unsuitable for subsequent motion, or a predefined trajectory may fail when the workpiece geometry changes.

Another limitation is the continued reliance on expert intervention. Even with simulation and offline programming tools, engineers are often required to define motion sequences, verify reachability, adjust waypoints, and validate process logic manually. This increases setup time and reduces adaptability to new parts.

These limitations highlight the need for a unified approach that integrates constraint-aware planning, automated trajectory generation, and structured execution to ensure consistent, repeatable, and flexible robotic bending operations.

2.7 Industrial Software Platforms and System Implementations

Industrial robot applications are commonly developed using software platforms that support programming, simulation, validation, and execution. These platforms play a central role in bridging theoretical methods and practical deployment by providing structured tools for developing, testing, and deploying robotic workflows in industrial environments.

2.7.1 Simulation and Offline Programming Environments

Simulation and offline programming environments enable robot programs to be developed, tested, and validated in a virtual environment before execution on a physical system. These environments typically use CAD-based models of the robot, tooling, and workcell, allowing engineers to evaluate reachability, analyse motion, and visualise robot behaviour prior to deployment. Such capabilities have been widely reported in the literature as key advantages of offline programming, particularly in reducing commissioning time and minimising trial-and-error on the shop floor [3].

In addition, modern industrial tools such as RoboDK provide functionalities for motion simulation and integration with multiple robot controllers, further supporting practical deployment in manufacturing environments [19]. However, despite these advantages, offline programming environments are not fully automated planning systems. These systems rely on manual configuration within the software, where the user is responsible for defining task sequences, selecting key poses, and con-

figuring the logical structure of the process. For complex manipulation tasks, this manual configuration remains a significant part of the engineering effort, limiting scalability in flexible production scenarios.

2.7.2 Graphical Programming and Integrated Execution Systems

Graphical programming platforms, such as drag&bot, aim to reduce the complexity of robot programming by allowing users to construct workflows using modular function blocks [20]. The AnyRobot interface enables external software to communicate with the drag&bot runtime through a WebSocket-based interface [21]. Together, the graphical workflow environment and the runtime interface support the development of robotic applications without requiring extensive low-level programming knowledge.

In the context of this thesis, drag&bot is relevant because it provides an integrated drag-and-drop programming and execution environment [20]. This environment can be combined with the AnyRobot interface for runtime-connected robotic workflow deployment [21]. The KEBA/drag&bot relationship also provides the industrial context for selecting the platform in this work [22].

These platforms are effective for structured tasks where predefined modules can be reused and combined efficiently. However, similar to offline programming environments, graphical systems generally rely on user-defined workflows and logic. While they facilitate execution and monitoring, they do not inherently solve the planning problem.

In particular, they do not automatically determine key decisions such as grasp selection, regrip strategies, or trajectory generation in the presence of changing workpiece geometries. These decisions remain dependent on user input or external planning methods. This limitation is especially critical in robotic sheet metal bending, where grasping, regripping, and motion planning are strongly interdependent and must be coordinated across the process sequence.

2.7.3 Summary and Limitations of Industrial Platforms

Existing industrial software platforms provide powerful tools for simulation, validation, and execution of robotic applications. They significantly improve programming efficiency compared to manual approaches and support practical deployment in industrial environments. However, their primary limitation lies in the lack of integrated automatic planning capabilities. While these platforms assist in implementing and validating robot programs, they still require substantial manual input for task definition, decision-making, and coordination of planning components.

This limitation becomes increasingly significant in complex applications such as robotic sheet metal bending, where multiple interdependent planning tasks must be considered simultaneously. These observations highlight the need for approaches that go beyond tool-based programming and provide integrated, constraint-aware, and automated planning capabilities, which motivates the work presented in this thesis.

2.7.4 Critical Comparison of Existing Approaches

The reviewed approaches differ not only in their level of automation, but also in the degree to which they integrate planning decisions into an executable industrial workflow. Manual and teach-pendant programming provide direct control but rely heavily on operator expertise. Offline programming and digital-twin approaches improve simulation and validation, but they often still require manual sequence definition, accurate model preparation, and repeated engineering adjustment when workpiece conditions change. Academic planning methods provide strong solutions for individual subproblems such as grasp selection, regripping, or trajectory generation, but they are frequently not connected to the complete runtime workflow of an industrial press-brake cell.

Table 2.1 summarises these differences and clarifies the positioning of the proposed thesis approach. The originality of this work therefore lies not in introducing a new general theory of grasping or trajectory planning, but in integrating orientation determination, constraint-based grip selection, automatic regrip handling, opposite-side bending logic, trajectory generation, and drag&bot runtime execution into one deterministic workflow within the defined simulation-based press-brake cell.

Approach	Automation Level	Regrip Support	Adaptability	Runtime Integration	Main Limitation
Manual / teach pendant	Low; poses and sequence are operator-defined.	Manual only.	Low; reteaching is required for changed geometry or placement.	Direct or platform-specific execution.	High programming effort and no integrated planning.
Offline programming tools	Partial; simulation-assisted sequence preparation.	Mostly manual or template-based.	Moderate; model and sequence updates are still required.	Tool-dependent.	Expert sequence definition and repeated adjustment remain necessary.
CAD/digital twin	Medium to high; model-based planning is possible.	Possible when modelled explicitly.	Moderate; depends on calibrated and updated models.	External interfaces.	Model preparation and calibration dominate deployment effort.
Academic planners	High for isolated subproblems.	Often strong in isolated regrasp planners.	Narrow; usually scenario-specific.	Usually not industrial runtime.	Often not connected to an end-to-end industrial workflow.
Proposed approach	High within the defined simulation scope.	Automatic regrip decision and insertion.	High for evaluated geometry and placement changes.	drag&bot + AnyRobot.	Simulation validation and rectangular-geometry scope.

Table 2.1: Critical comparison of existing robotic programming and planning approaches with the proposed thesis approach.

2.8 Research Gap and Motivation

The literature shows a clear progression from manual robot programming toward simulation-supported and partially automated methods. Manual programming remains reliable for stable tasks; however, it is not well suited to scenarios where workpiece geometry, position, and regrip orientation change frequently. Offline programming improves safety and commissioning efficiency, yet it still depends on user-defined task sequences and accurate virtual models. Automatic programming and rule-based methods further reduce manual effort, but many approaches focus on individual decisions rather than the complete bending workflow.

In robotic sheet metal bending, this separation represents a major limitation. The workpiece changes shape after each bending step, which affects grasp feasibility, regrip requirements, and reachable trajectories. A grasp that is valid at one stage may obstruct tool access in a later step; a regrip pose may be geometrically feasible but difficult to reach; and a trajectory that is valid in isolation may become infeasible once the formed part is attached to the robot.

The identified research gap is the lack of a practical workflow that integrates grasp planning, regrip planning, and constraint-aware trajectory generation for robotic press brake bending. Existing methods reduce manual effort and improve precision compared to manual press brake operation; however, they do not sufficiently address these interdependent planning tasks as a coordinated process under workspace, kinematic, and process constraints.

Addressing this gap is essential for flexible automation. A system that can generate valid trajectories, intermediate configurations, and decision processes such as regrip calculations, and transform these into executable robot motions, can significantly reduce engineering effort while improving adaptability, consistency, and repeatability.

This thesis therefore proposes an integrated and constraint-aware approach in which automated trajectory generation, workspace-aware planning, and deterministic execution are combined within a structured pipeline. The role of the user is limited to specifying the desired output, while the system processes this input and decomposes it into executable tasks.

The identified gap and the proposed direction form the basis for the concept development presented in the following chapter.

3 System Concept and Methodology

3.1 Problem Definition and Objectives

The state-of-the-art analysis indicates that existing approaches do not fully provide an integrated solution for robotic sheet metal bending. In particular, planning tasks such as grasp selection, regripping, and trajectory generation are often handled separately, which limits the level of automation.

A key challenge arises from the changing geometry of the workpiece during the bending process. Each bending step modifies the part shape, affecting reachability and feasible handling strategies. The problem addressed in this work is the lack of a systematic method for automatically generating robot programs for press brake sheet metal bending that considers changing workpiece geometry, handling feasibility, and regripping requirements within a unified framework.

The objective of this thesis is to develop a method for the automatic generation of robot programs that improves adaptability to varying workpiece geometries while reducing manual programming effort and programming time. In addition, the approach aims to enable usability for non-expert operators by requiring only the specification of the desired output. To achieve this, a unified framework is proposed that integrates handling, regripping, and trajectory generation into a single workflow. The approach focuses on generating robot motions directly from process inputs rather than relying on manually defined waypoint sequences, particularly for multi-stage sheet metal bending operations.

The specific objectives are:

1. Automatic generation of robot programs based on geometry and process parameters.
2. Constraint-aware trajectory generation considering workspace and kinematic limitations.
3. Support for multi-stage bending, including regripping and reorientation.
4. Determination of feasible grip points based on accessibility and process stage.
5. Integration with an industrial robot programming environment.

The proposed method is evaluated in a simulation environment, with a focus on conceptual development and virtual validation.

3.2 Requirement Definition and Management

The system concept developed in this thesis defines a structured set of requirements based on the problem described in Section 3.1. These requirements specify the expected functionality and describe the intended system behaviour in a clear and transparent manner.

The requirements are organised into functional requirements (FR), non-functional requirements (NFR), and software requirements (SR), based on industrial and company-specific criteria. This structured classification supports consistent system design, implementation, and validation, while also enabling traceability throughout the development process.

Furthermore, the defined requirements provide a basis for evaluating whether the developed system satisfies the intended objectives during the validation phase.

For consistent interpretation, several terms are used with fixed meanings throughout the thesis. An execution-ready workflow is a generated or prepared sequence that can be started in the virtual cell without additional manual waypoint teaching or function-block sequencing. A configuration change denotes an update of workpiece, process, or safety parameters without source-code modification. Deterministic behaviour means that repeated runs under the same scenario definition lead to the same completion, failure, or controlled-stop classification. A controlled stop denotes safe interruption before infeasible or unsafe continuation; it is therefore treated as safe system behaviour, but not as a completed execution trial.

3.2.1 Functional Requirements

The functional requirements define the capabilities that the system must provide to enable automatic generation of robot programs for sheet metal bending operations. A primary requirement is that the program must be generated from operator-defined inputs. This directly addresses the limitations of manual programming identified in the previous chapters and forms the core of the proposed solution.

The system must generate feasible robot trajectories for handling and bending operations based on target poses and process parameters. These trajectories must satisfy geometric and kinematic constraints within the defined workspace. Furthermore, the system must support bending on multiple faces of the sheet, requiring multi-stage operations that include reorientation and regripping.

Grip positions must be determined automatically according to the process stage, as variations in workpiece geometry require adaptive grasp selection to ensure stable handling throughout the process sequence. The generated programs and trajectories must be verified prior to execution. If infeasible conditions are detected, such

as unreachable poses or invalid configurations, the system must initiate replanning or terminate execution safely when recovery is not possible.

The complete program must be validated in a simulation environment before execution to ensure that the generated sequence is feasible, consistent, and compliant with system constraints. The detailed functional requirements corresponding to these capabilities are summarised in Table 3.1.

3.2.2 Non-Functional Requirements

In addition to functional requirements, non-functional requirements define the quality attributes that the system must exhibit. These requirements describe how the system behaves during operation and cover aspects such as usability, maintainability, robustness, consistency, and repeatability.

A key design objective is to ensure usability, allowing operators to define process inputs without requiring detailed technical knowledge. This ensures that usability is considered as a fundamental requirement for practical adoption. The system must produce deterministic and repeatable results for identical input conditions, thereby ensuring consistent behaviour.

The system architecture separates planning, validation, and execution into distinct components, enabling modular development, maintainability, and extensibility without introducing unnecessary dependencies across the system. Maintainability is further supported by a structured design that facilitates modification and future enhancements.

Within the defined scope, the system should tolerate minor variations in input parameters and process conditions while maintaining stable and reliable output. Robustness is addressed by handling variations without compromising system performance or validity of results.

The detailed non-functional requirements corresponding to these quality attributes are summarised in Table 3.2.

3.2.3 Software Platform and Tool Requirements

The software platform and tool requirements define the technical environment in which the proposed system is implemented and evaluated. They are grouped as SR-01 to SR-03 because the virtual runtime, communication interface, and tool reference frame must work together for reliable program generation and execution.

SR-01 Virtual execution environment. The system shall operate within a virtual robot programming environment that supports simulation, execution, and valida-

ID	Functional Requirement (SMART)	Observable Acceptance Criteria	Verification Method
FR-00	The system shall generate an execution-ready drag&bot robot program from work-piece geometry, configuration data, and operator bend-side input for the evaluated press-brake scenarios.	For valid evaluated inputs, the generated work-flow shall be ready for simulated execution with-out manual waypoint teaching or function-block sequencing; unresolved infeasible cases shall stop in a controlled way rather than continue blindly.	Chapter 6 workflow comparison and scenario validation
FR-01	The system shall detect unreachable or infeasible target poses before robot motion commands are dispatched.	Unreachable or unsafe target poses shall be rejected before unsafe motion execution and shall generate warning or error feedback for operator decision-making.	Workspace-constrained and failure-prone valida-tion
FR-02	The system shall allow the operator to select the required bend side before automated program generation or execution.	The selected bend side shall be reflected in the generated workflow without manual resequenc-ing, including Side A direct bending and Side E opposite-side bending.	Chapter 6 Side A/-Side E validation
FR-03	The system shall automatically insert a regrip operation when the current workpiece pose cannot directly reach the next target pose.	The regrip logic shall be inserted when direct execution is infeasible and shall execute or stop safely in regrip-required and opposite-side scenarios without manual waypoint construction.	Regrip-required and Side E scenario validation
FR-04	The system shall handle changes in initial workpiece orientation through automatic orientation validation and correction.	The automated workflow shall complete the evaluated orientation-variation scenario without manual correction of the generated sequence.	Automated orientation-variation trials
FR-05	The system shall support opposite-side bending by generating the required flip and reorientation workflow automatically.	The opposite-side workflow shall generate flip and regrip behaviour automatically and shall complete the evaluated Side E scenario when no unresolved workspace or fixture constraint remains; otherwise it shall stop in a controlled way.	Side E opposite-side workflow trials
FR-06	The system shall determine grip positions that satisfy edge-clearance, hole-avoidance, and holding-force checks before execution.	For evaluated valid workpiece states, selected grips shall pass geometric and holding-force checks before trajectory execution; invalid grips shall be rejected.	TC-06 grip-planning test
FR-07	The system shall detect unsafe conditions and either replan or stop execution in a controlled way before unsafe continuation.	Unsafe cases shall be handled through warn-ing feedback, replanning where possible, or controlled stop before unsafe continuation; a controlled stop is safe system behaviour but an incomplete execution trial.	Failure-prone scenario validation

Table 3.1: Functional Requirements for Automatic Robot Program Generation.

ID	Non-Functional Requirement	Observable Acceptance Criteria	Verification Method
NFR-01	Operator interaction during normal automated generation shall be limited to configuration or input selection and execution confirmation.	The automated workflow comparison shall require no manual waypoint teaching, function-block sequencing, or manual reprogramming before simulated execution.	Manual-versus-automated workflow validation
NFR-02	Workpiece geometry, process settings, and safety parameters shall be configurable without source-code modification.	Geometry-change and position-change scenarios shall be handled through configuration or operator input updates without rebuilding or modifying source code.	Geometry and position variation validation
NFR-03	Unsafe or infeasible conditions shall produce clear warning or error feedback before continuation.	Detected unsafe or infeasible conditions shall generate warning or error feedback and shall require operator acknowledgement, replanning, or controlled termination before continuation.	Failure-prone and warning-behaviour validation
NFR-04	The system shall use a deterministic execution sequence for a given configuration.	Repeated automated runs under the same scenario definition shall produce consistent completion, failure, or controlled-stop classifications.	Repeated automated scenario validation
NFR-05	Fixture pose deviations shall be checked against defined positional and angular tolerance limits before execution.	Position deviation above 5 mm or angular deviation above 0,10 rad shall trigger a fixture warning and controlled operator decision before execution.	Fixture perturbation and constrained-placement validation

Table 3.2: Non-Functional Requirements – NFR-01 to NFR-05 with acceptance criteria and verification methods.

tion of generated robot programs. In this work, drag&bot is used as the runtime environment for the virtual UR5e press-brake cell. The acceptance criterion is that a stable connection to the configured runtime is established before execution and that generated motion, service, object-handling, and input-output operations can be validated in the virtual cell.

SR-02 Communication interface. The planning system shall communicate with the execution environment through the AnyRobot interface using WebSocket (web-based communication protocol). This interface provides a controlled abstraction for robot motion commands, input-output signals, object handling, and runtime feedback without requiring direct access to robot-specific controller interfaces. The acceptance criterion is that every relevant runtime request receives acknowledgement before the workflow continues.

SR-03 Tool centre point definition. All generated motion targets shall be defined with respect to the configured TCP. In this thesis, the required TCP identifier is `project_3_tcp`, including the documented offsets and orientation parameters. The acceptance criterion is that any future TCP modification is handled through configuration and reflected in the generated poses without structural changes to the planning logic.

3.2.4 System Constraints and Assumptions

The proposed system is developed under a defined set of constraints and assumptions. At the geometric level, the workpiece model is restricted to rectangular sheet metal parts with exactly four edges aligned with the global coordinate system. A single global wall height parameter is assumed for all bending operations; therefore, edge-specific wall height variation is not supported.

Internal features such as holes are considered only for avoidance during gripping. Circular holes are defined by their centre coordinates and radius, and rectangular cut-outs can be specified using predefined coordinate inputs. However, more complex geometries, such as polygonal cut-outs or irregular contours, are outside the scope of this work.

The bending process is limited to right-angle ($90°$) bends, and user-defined bend angles are not supported. Each sheet edge can be assigned to only one bending operation. Bending can be performed either on the primary side or the opposite side, but simultaneous selection of both sides is not supported. The total number of bending operations is limited to four edges of the workpiece, including both primary and opposite-side bending stages.

From a system perspective, the robot configuration, tooling setup, and workspace layout are assumed to remain fixed during operation. The system assumes the use

of a vacuum-based gripping mechanism with a predefined TCP configuration. No external perception systems, such as cameras, Light Detection and Ranging (LiDAR), or other vision-based sensors, are used. All robot programs, poses, grip points, regripping decisions, and motion sequences are generated computationally based on predefined geometric input parameters, operator-defined inputs, and configuration data. Consequently, the system does not rely on real-time sensing but derives handling strategies through deterministic calculations.

Grip feasibility is evaluated using simplified physical considerations, including pressure distribution, friction, and load capacity. Alternative gripping methods, such as mechanical, magnetic, or multi-fingered grippers, are excluded.

Advanced capabilities, including dynamic collision-free trajectory generation, real-time motion adaptation, and environment-aware path optimisation, are not considered. These constraints ensure that the system remains aligned with the objective of deterministic, consistent, and repeatable automatic program generation within a controlled industrial environment.

3.2.5 System Traceability

Traceability is used to keep the transition from requirements to implementation and validation transparent. Each functional, non-functional, and software requirement is assigned an identifier such as FR-01, NFR-01, or SR-01. These identifiers are reused in the implementation chapter and in the validation tables so that every implemented function can be linked back to its requirement and verification activity.

Test Case (TC) denotes a validation case in the traceability matrix. The complete mapping between requirements, implementation modules, and validation scenarios is provided in Appendix A.4, Table 2.

4 Development of Concepts

4.1 Purpose and Scope of Concept Development

Concept development is a fundamental stage in systematic engineering design, in which different solution principles are generated and evaluated before implementation. By comparing alternative solutions against defined requirements, the risk of selecting an unsuitable design solution is reduced [23].

The research gap identified in Section 2.8 highlights the absence of a systematic approach for automatic robot program generation in press brake sheet metal bending, where existing solutions do not fully address the defined requirements. These challenges are translated into measurable engineering objectives and requirements in Chapter 3, including functional requirements (FR-00 to FR-07), non-functional requirements (NFR-01 to NFR-05), and software platform requirements (SR-01 to SR-03), with the full traceability mapping provided in Appendix A.4, Table 2.

To address this gap, three conceptual solution approaches are developed and evaluated in this chapter. Concept A represents a structured manual programming workflow, Concept B proposes a rule-based automatic program generation approach, and Concept C explores a CAD-integrated offline programming methodology [3], [12].

These concepts are first compared qualitatively and subsequently evaluated using a weighted scoring method based on defined evaluation criteria. The final concept is selected using the Weighted Sum Model (WSM), a quantitative decision-making method in which evaluation criteria are assigned relative weights and each concept is scored accordingly. This ensures that the concept selection process is systematic, transparent, consistent, and aligned with the defined requirements.

4.2 Concept A – Structured Operator Workflow with Guided Teach-In Programming

Method and workflow.

Concept A follows a structured manual programming approach in which the operator defines robot motions and execution strategies, typically using a teach pendant as described in Section 2.3. In this approach, robot trajectories, gripper actions, and execution sequences are created manually by the user, while the system pro-

vides a structured interface to organise the programming process [12]. The created programs are stored and reused for similar future executions.

This represents a conventional programming method, which improves usability by structuring the sequence of operations and reducing unstructured workflows. The operator constructs the robot program manually within the drag&bot environment by assembling function blocks for motion execution, gripper control, and sequencing logic.

In this context, waypoint generation is performed manually by moving the robot TCP to the required positions and recording these points as configured function blocks. All operations, including gripping, regripping, handling, and opposite-face bending, are defined manually by the operator. This highlights the high implementation feasibility of Concept A. The arrangement and configuration of function blocks are illustrated in Figure 4.1, where motion commands and gripper operations are explicitly defined.

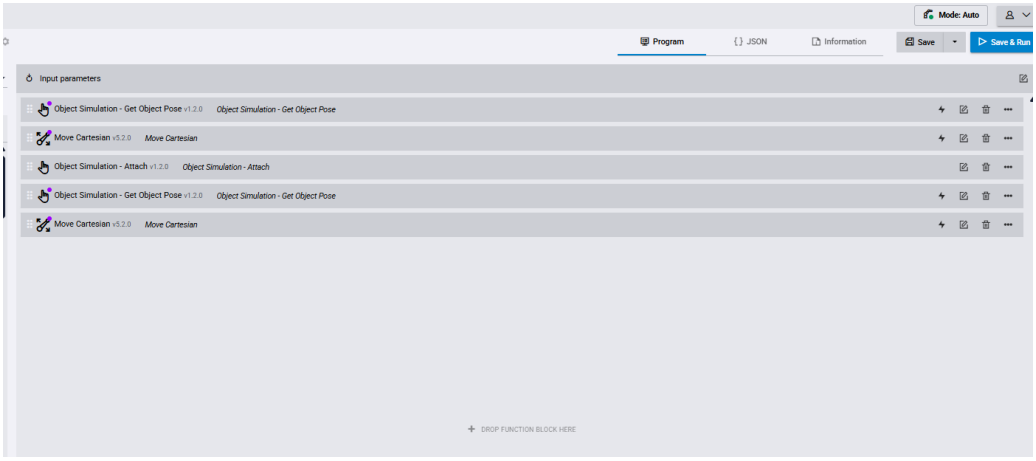


Figure 4.1: Detailed function-block configuration for manual motion and gripper sequencing in Concept A.

As a result, all handling strategies, execution logic, and process decisions depend entirely on operator expertise. No automated planning or validation is performed prior to execution. Feasibility issues, such as reachability constraints, are identified only at runtime, which may lead to unreachable targets, process interruptions, or system failures. This lack of pre-execution validation increases setup effort and reduces reliability, particularly in complex bending scenarios.

Furthermore, changes in workpiece geometry during the bending process are not automatically considered. The operator must manually adapt the program to account for geometric changes, which requires additional effort and expertise. For operations such as opposite-side handling or sheet flipping, the operator must explicitly teach the robot the required motions at specific stages. Consequently, the programming effort increases significantly, especially for multi-stage bending processes.

From a system integration perspective, Concept A can be implemented within the drag&bot environment using function blocks for robot motion, gripper control, and execution sequencing. However, it does not provide native integration with higher-level automation frameworks such as AnyRobot, and TCP definitions must be configured manually.

The concept is relatively easy to implement and does not require complex infrastructure or advanced planning algorithms. However, this simplicity comes at the cost of functionality, as it does not support automatic waypoint generation, regrip planning, geometry-aware adaptation, or integrated validation mechanisms. As a result, Concept A does not address the automation requirements identified in the state-of-the-art analysis.

System structure.

The system structure of Concept A consists of two main layers: a manual workflow construction layer and an execution layer. In the first layer, the operator defines the complete process sequence by specifying robot motions, gripper actions, and task order using function blocks within the programming environment.

In the second layer, the execution environment interprets and executes the manually defined sequence. No intermediate planning layer exists for automatic trajectory generation, geometric reasoning, or constraint evaluation. As a result, the system relies entirely on operator-defined logic and does not incorporate adaptive behaviour or automated validation prior to execution.

Assessment.

Concept A is straightforward to implement and provides full transparency and control to the operator. It is suitable for stable and repetitive tasks with minimal variation. However, the limitations of this approach are significant with respect to the objectives of this thesis.

Although programming effort may be reduced through structured tools such as drag&bot, the approach still requires substantial expertise in robot kinematics, path generation, and process planning. All robot motions, handling steps, and process logic must be defined manually, resulting in high development effort and increased setup time [3], [12].

Furthermore, the approach lacks scalability and adaptability. Changes in workpiece geometry or process conditions require manual reprogramming, which limits its applicability in flexible manufacturing environments. The concept does not support automatic grip point calculation, regrip planning, or trajectory generation. The operator must manually define all intermediate positions, gripper actions, and motion sequences, which limits the level of automation.

In addition, integrated validation and decision-making are not supported. Feasibility

checks are not performed prior to execution, and infeasible configurations are often detected only at runtime, leading to inefficiencies and potential operational risks.

Although Concept A can be implemented using industrial platforms such as drag&bot, it remains a purely manual programming approach. It does not support automatic program generation, adaptive planning, or constraint-aware decision-making. Consequently, it does not fulfil the functional and non-functional requirements related to automation, adaptability, and validation defined in this work. The evaluation of this concept is quantified in the weighted sum model presented in Section 4.5.

For these reasons, Concept A is not suitable as a final solution for addressing the identified research gap and achieving the objectives of this thesis.

4.3 Concept B – Automatic Program Generation with Integrated Constraint Validation

Method and workflow.

Unlike Concept A, Concept B introduces automation into the programming workflow. In this approach, robot motion paths and process logic are directly derived from geometric inputs and configuration parameters. Instead of explicitly defining waypoints, the system automatically generates robot trajectories, grip positions, and execution sequences based on defined constraints.

Waypoint generation is performed algorithmically based on the workpiece position, where intermediate points are determined automatically using planning logic. This reduces direct manual teach-in and is expected to reduce programming effort. The user is required to specify only the target output pose. Regripping operations and opposite-side bending are handled through structured logic that combines sequence control with input-output coordination, enabling consistent handling strategies across different process stages. The number of required regrips and the necessity of sheet flipping are determined automatically based on the desired output configuration.

Validation is integrated into the planning process and evaluates all trajectories dynamically through constraint checking, including orientation, reachability, and geometric feasibility. This ensures that only executable configurations are considered prior to motion generation, thereby improving reliability and reducing runtime errors. The program adapts to different geometric dimensions, workpiece placements, and process parameters without requiring modifications to the underlying program logic. This enables efficient handling of varying workpieces without full reprogramming or repeated simulation setup.

In accordance with the defined requirements, Concept B is implemented within the drag&bot virtual environment, satisfying the software requirement SR-01. Communication with the execution system is established through the AnyRobot interface using a WebSocket-based architecture [21]. In the implemented architecture, this interface separates planning logic from runtime command execution.

Grip planning is performed using a parameterised approach based on workpiece geometry, suction cup properties, and physical constraints. Candidate grip positions are evaluated against edge clearance, spacing constraints, and vacuum holding force requirements [17]. Only physically feasible and stable configurations are selected for further processing, ensuring safe handling conditions prior to trajectory generation. If infeasible configurations are detected during planning, the system automatically initiates regripping sequences or adjusts the workpiece orientation to maintain feasibility.

Opposite-side bending operations are handled through a structured flipping and repositioning workflow. When opposite-side processing is required, the system automatically determines the need for flipping and executes the corresponding handling sequence. All validation steps, including orientation verification, grip feasibility, and reachability assessment, are consistently applied throughout the process [1].

Robot trajectories are generated using linear Cartesian motion commands with smooth velocity profiles, ensuring stable handling and predictable execution behaviour [12]. All motion targets are defined relative to a predefined TCP, ensuring consistency across the system and alignment with execution requirements.

Overall, Concept B represents a configuration-driven approach that enables automatic program generation, integrated validation, and adaptive handling of changing workpiece conditions. The system operates deterministically based on defined input parameters, ensuring consistency, repeatability, and reproducibility of generated robot programs.

System structure.

The system architecture of Concept B consists of five functional layers. The input and configuration layer collects process parameters and geometric data. The evaluation layer applies constraint checks, including grip feasibility, edge clearance, and workspace limitations.

The planning layer determines the required process sequence, including regripping and orientation changes, based on reachability and feasibility conditions. The trajectory generation layer produces executable robot motions, while the validation layer ensures that all generated poses are feasible prior to execution.

These layers are connected to the drag&bot runtime environment through the Any-

Robot interface, enabling direct execution of generated programs. The separation between configuration and logic allows the system to adapt to new geometries without modifying core algorithms, thereby supporting scalability and maintainability.

Assessment.

Concept B directly addresses the research gap identified in Section 2.8 by integrating automatic program generation, constraint-aware validation, and adaptive handling within a unified framework. This is intended to reduce programming and reprogramming effort while improving efficiency.

Compared to manual programming approaches, the concept reduces engineering effort by eliminating explicit waypoint definition and manual sequence design. This supports functional requirements such as FR-00, FR-03, FR-05, and FR-07, while the configuration-driven architecture satisfies NFR-02 by enabling adaptation to new part geometries without modifying source code. The concept also addresses NFR-03, NFR-04, and NFR-05 through structured validation and deterministic execution behaviour.

The system operates within the drag&bot runtime environment via the AnyRobot interface, satisfying SR-01 and SR-02. All motion targets are defined relative to the predefined TCP, ensuring consistency with execution requirements.

However, the concept operates under defined assumptions, including simplified workpiece geometry and constrained process conditions. Validation has been performed in a virtual simulation environment. Integration with physical hardware and advanced capabilities such as collision detection are identified as areas for future work.

4.4 Concept C – CAD-Based Offline Programming with Simulation

Method and workflow.

Concept C follows a CAD-based offline programming (OLP) approach in which robot programs are generated and validated within a simulation environment prior to execution. This concept relies on a complete geometric model of the robot cell, including the robot, tooling, press brake, and workpiece.

In this approach, robot trajectories, grip positions, and process sequences are defined and evaluated within a virtual environment based on CAD models of the workpiece and production setup. Standard formats such as Unified Robot Description Format (URDF) are typically used to represent robot and environment models. Motion planning and reachability analysis are performed in advance, without direct interaction with the physical robot, and the generated programs are subsequently transferred to the robot controller for execution.

Regripping and opposite-side bending operations are handled within the simulation workflow by defining motion sequences based on the digital model. These operations can be adapted dynamically by updating the CAD model when changes in geometry or process conditions occur. However, such modifications require updates to the model and repeated simulation cycles.

A key advantage of this approach is the ability to perform comprehensive pre-execution validation. Offline programming and simulation allow robot programs to be tested in a virtual environment before deployment [3]. Virtual commissioning supports earlier detection of system inconsistencies and reduces commissioning effort [24]. Industrial OLP tools such as ABB RobotStudio, KUKA.Sim, and FANUC ROBOGUIDE implement similar methodologies and are widely used in structured production environments [3]. These systems rely on accurate geometric and kinematic models, including correct TCP definitions, to ensure consistency between simulation and execution.

Higher levels of automation can also be achieved in research-oriented Robot Operating System (ROS)-based planning frameworks. ROS provides a software infrastructure for robotic system integration [25]. MoveIt supports motion planning and related manipulation-planning functionality within such frameworks [26]. Unity-based interfaces can be used for simulation or visualisation connectivity with ROS 2 systems [27]. Perception and localisation stacks such as Simultaneous Localisation and Mapping (SLAM) can further extend robot-system awareness where mapping is required [28]. However, these architectures are outside the implementation scope of this thesis and are considered here only as alternative concept directions.

Most importantly, CAD-based OLP systems and ROS–Unity architectures do not provide direct compatibility with the required execution environment defined in this thesis. They do not integrate with the drag&bot platform specified in SR-01, nor do they support the WebSocket-based AnyRobot communication interface defined in SR-02. Therefore, although Concept C is capable of achieving a high level of automation within its own framework, it does not satisfy the required software platform and integration constraints.

System structure.

The system structure of Concept C follows a multi-layer architecture. The CAD modelling layer provides digital representations of the robot, press brake, tooling, and workpiece. These models form the basis of the digital twin and must accurately represent the physical system, including geometric dimensions and TCP configurations.

The offline programming and simulation layer uses these models to perform trajectory planning and reachability analysis. Validation prior to execution is achieved using kinematic solvers and simulation tools. In ROS-based implementations, this

layer can also support automated trajectory generation, inverse kinematics, and constraint-aware motion planning.

The communication and integration layer is responsible for transferring generated robot programs from the simulation environment to the execution system. The execution layer consists of the robot controller, which executes the generated program on the physical system.

The overall architecture depends strongly on the accuracy of the digital model. Deviations between the virtual and physical environments may lead to execution errors that require manual correction during commissioning [3], [12]. Although ROS–Unity-based extensions provide additional capabilities, they operate as separate environments and require custom interfaces for communication with industrial robot controllers. As a result, the system remains loosely coupled and lacks direct integration with the target execution platform defined in this thesis.

Assessment.

Concept C provides strong advantages in structured industrial environments, particularly where accurate digital models of the robot cell are available. Simulation-based validation enables early detection of collisions, unreachable poses, and process inconsistencies, thereby reducing commissioning effort and improving operational reliability. These characteristics make CAD-based offline programming a widely adopted approach in industrial robotics.

With the integration of ROS-based planning frameworks and Unity-based simulation, Concept C can achieve a high level of automation. Automated trajectory generation, inverse kinematics, and constraint-aware motion planning can be performed within the ROS/MoveIt planning framework [25], [26], while Unity-based environments can be used as visualisation and simulation interfaces connected to ROS 2 systems [27].

However, several limitations exist within the context of this thesis. First, the approach depends on accurate and continuously updated CAD models of the robot cell, including tooling, fixtures, and workpieces. Second, both industrial OLP platforms and ROS-based frameworks require specialised expertise and additional development effort for implementation, including CAD integration and kinematic modelling.

Most importantly, Concept C does not satisfy the defined software platform and tool requirements. The approach depends on external simulation and programming environments that operate independently from the drag&bot runtime environment specified in SR-01. Furthermore, it does not support the WebSocket-based AnyRobot communication interface required in SR-02.

As a result, although Concept C enables a high level of automation within its own framework, it cannot be integrated into the required system architecture. This

incompatibility represents a fundamental constraint. Therefore, the rejection of Concept C is not based on its technical capability, but on its inability to satisfy the mandatory software requirements. Since SR-01 and SR-02 are non-negotiable constraints defined by the target industrial environment, Concept C cannot be considered a feasible solution within the scope of this thesis.

4.5 Concept Decision Using the Weighted Sum Model

The concept selection in this work is performed using the WSM, a structured multi-criteria decision-making method widely used in engineering design. The WSM enables quantitative comparison of alternative concepts by evaluating them against a defined set of criteria derived from the system requirements.

Each evaluation criterion is assigned a weighting factor based on its relative importance in addressing the research objectives. The concepts are then evaluated against each criterion using a consistent scoring scale from 1 to 5, where 1 indicates minimal or no satisfaction and 5 indicates a high level of satisfaction. The overall score for each concept is obtained by multiplying the assigned weight with the corresponding score and summing the results across all criteria.

This method ensures that the concept selection process remains transparent, systematic, and aligned with the functional, non-functional, and system-level requirements defined in Chapter 3. It also reduces subjectivity by providing a structured framework for comparing alternative solution approaches.

For the scoring interpretation, a score of 1 indicates that the concept does not satisfy the criterion or satisfies it only minimally. A score of 2 indicates weak fulfilment with major limitations. A score of 3 indicates partial fulfilment under additional effort or constraints. A score of 4 indicates good fulfilment within the scope of this thesis. A score of 5 indicates complete fulfilment without major remaining limitations. The scores are therefore not experimental measurements, but engineering decision scores based on requirement fulfilment, implementation constraints, platform compatibility, expected programming effort, and the limitations identified in the concept descriptions.

The evaluation criteria are derived directly from the functional requirements (FR-00 to FR-07), non-functional requirements (NFR-01 to NFR-05), and software platform requirements (SR-01 to SR-03) defined in Chapter 3. The assigned weight for each criterion reflects its relative importance with respect to the identified research gap and overall system objectives.

1. **Programming effort reduction (20%):** prioritises concepts that minimise manual waypoint definition and reduce reprogramming effort.

2. **Automated regrip and opposite-side handling (15%):** evaluates whether complex handling operations can be generated automatically rather than manually sequenced. Robotic sheet-metal bending requires coordinated bending-sequence, grasp, and repositioning planning [1]. Repositioning and regrip timing are important planning aspects in robot-assisted bending [2].
3. **Pre-execution validation and safety (15%):** assesses reachability, orientation feasibility, handling constraints, replanning, and safe termination.
4. **Adaptability (10%):** favours configuration-driven adaptation without changes to the underlying program logic.
5. **Platform compatibility (15%):** checks compliance with drag&bot and AnyRobot constraints, especially SR-01 and SR-02.
6. **Operator usability (10%):** rewards approaches that reduce the need for specialised programming knowledge during operation.
7. **Level of automation (15%):** evaluates automatic generation of trajectories, handling decisions, and process sequencing.

Table 4.1 presents the complete WSM evaluation with score justifications and weighted totals for all three concepts.

Evaluation Criterion	Wt.	Concept A	Sc.	Concept B	Sc.	Concept C	Sc.
Programming Effort Reduction	20%	Manual programming throughout.	1	Automatic generation from configuration; simulation-scoped.	4	High automation but requires CAD setup and model maintenance.	4
Automated Regrip and Opposite-Side Handling	15%	No automation.	1	Rule-based regrip and opposite-side workflow; recovery still limited.	4	Predefined in simulation, limited runtime adaptation.	3
Pre-Execution Validation and Safety	15%	Detected during execution only.	1	Pre-checks with controlled stop and limited replanning.	4	Strong simulation-based validation.	4
Adaptability to Changing Workpiece Geometry	10%	Manual reprogramming required.	1	Parameter-based adaptation within rectangular-sheet scope.	4	Requires CAD update and resimulation; partial automation possible.	3
Platform Compatibility	15%	Partial compatibility.	2	Compatible with required drag&bot + AnyRobot environment.	4	Not compatible with required platform; fails SR-01 and SR-02.	1
Operator Usability	10%	High expertise required.	2	Minimal operator interaction; engineering setup remains.	4	Moderate usability; CAD and simulation knowledge required.	3
Level of Automation	15%	No automation; manual teach programming.	1	Automatic rule-based program generation in the evaluated scope.	4	High automation achievable through OLP and ROS-based planning.	4
Weighted Total Score (100%)		**Concept A:** 1,25		**Concept B (selected):** 4,00		**Concept C:** 3,20	

Table 4.1: Weighted Sum Model (WSM) evaluation of Concept A, Concept B, and Concept C.

The scores in Table 4.1 are intentionally conservative. Concept B receives high scores because it fulfils the required automation, regrip, validation, and platform-integration objectives, but it is not scored as perfect because the implementation remains simulation-based and constrained by the defined geometry and workspace assumptions. Concept C is technically strong in external CAD, OLP, or ROS-based environments, but its ranking is limited by the mandatory drag&bot and AnyRobot platform constraints. Concept A remains limited because it depends on manual waypoint and sequence definition even though it is simple to realise.

The WSM results confirm the qualitative comparison of the three concepts. Concept B achieves the highest overall score (4,00), followed by Concept C (3,20) and Concept A (1,25). Concept B performs consistently well across all evaluation criteria, particularly in programming effort reduction, automated handling, validation capability, adaptability, and platform compatibility. This balanced performance makes it the most suitable candidate within the defined system constraints.

The low score of Concept A results from its dependence on manual programming. Although the drag&bot function-block environment can structure the workflow and improve transparency, the operator must still define waypoints, gripper actions, regrip motions, and execution sequences manually. Therefore, Concept A receives low scores for programming effort reduction, level of automation, adaptability, and pre-execution validation. It is not scored higher because it does not generate robot programs automatically and does not provide integrated geometry-aware planning.

Concept B receives the highest score because it directly addresses the central requirements of this thesis. Robot motions, grip points, regrip decisions, and execution sequences are generated from configuration and geometric input rather than manually taught waypoints. It also satisfies the required drag&bot and AnyRobot integration constraints. However, Concept B is generally scored as 4 rather than 5 because the developed system is evaluated only in simulation and remains limited by predefined configuration data, simplified geometry assumptions, and the absence of physical robot validation.

Concept C receives intermediate-to-high scores for automation and validation because CAD-based offline programming and simulation environments can support trajectory planning, reachability analysis, and collision checking. However, its score is reduced because it requires CAD model preparation, simulation setup, specialised engineering knowledge, and repeated model updates when the workpiece or cell configuration changes. Most importantly, Concept C receives the lowest score for platform compatibility because it does not satisfy the mandatory drag&bot and AnyRobot requirements defined in SR-01 and SR-02. Therefore, it is technically capable but not suitable for the selected implementation environment.

This distinction is important for the final decision. Concept C is not rejected because it lacks technical capability, but because its main strengths lie in external CAD, OLP,

or ROS-based environments that do not satisfy the mandatory platform constraints of this thesis.

Concept A achieves the lowest score, as it relies on manual programming and does not significantly reduce engineering effort. While it is simple to implement and compatible with existing tools, it does not address the primary objective of this thesis, which is the automation of robot program generation.

Based on the weighted evaluation and the defined requirement constraints, Concept B is selected for implementation. The decision is not based solely on achieving the highest score, but on providing the most effective balance between automation, adaptability, validation capability, and full compatibility with the defined software platform.

The implementation of Concept B is presented in Chapter 5.

5 Implementation

5.1 Introduction and Scope

This chapter presents the implementation of the selected concept as a fully developed system. Based on the evaluation results using the WSM, Concept B is selected for implementation, as it satisfies the majority of the defined functional, non-functional, and software platform requirements outlined in Chapter 3. In particular, Concept B enables automatic robot program generation, integrated constraint validation, and seamless operation within the drag&bot runtime environment.

The system generates robot programs automatically using input configurations provided by the user and determines feasible solutions to execute complete sheet metal bending operations. This eliminates manual waypoint generation and function block arrangement, thereby reducing operator intervention and improving consistency and repeatability.

The implementation was developed from the requirements and concept decision described in the previous chapters. The defined requirements were translated into a modular software structure that separates configuration handling, communication, planning, validation, and execution. In practical terms, the system starts from workpiece and process parameters, the selected bend side, robot and tool settings, and the tracked sheet pose in the virtual cell. It then checks whether the initial state is valid, determines a feasible pickup orientation and grip, applies regrip or flip logic when required, and converts the resulting task sequence into robot motion and gripper commands. The output of the pipeline is therefore an executable sequence of handling and bending-related actions sent to the drag&bot runtime through the AnyRobot interface.

The implementation is structured around four key domains. The chapter first presents the system architecture, including communication, configuration, and operator interface structures. It then describes the development and simulation environment, followed by the functional subsystems in execution order, including safety validation, orientation determination, grip planning, gripper control, regrip handling, opposite-side bending, and trajectory generation. The chapter concludes with a Failure Mode and Effects Analysis and a requirement-to-implementation traceability mapping.

The system operates entirely within the drag&bot virtual simulation environment. Communication with the robot runtime is established through the AnyRobot inter-

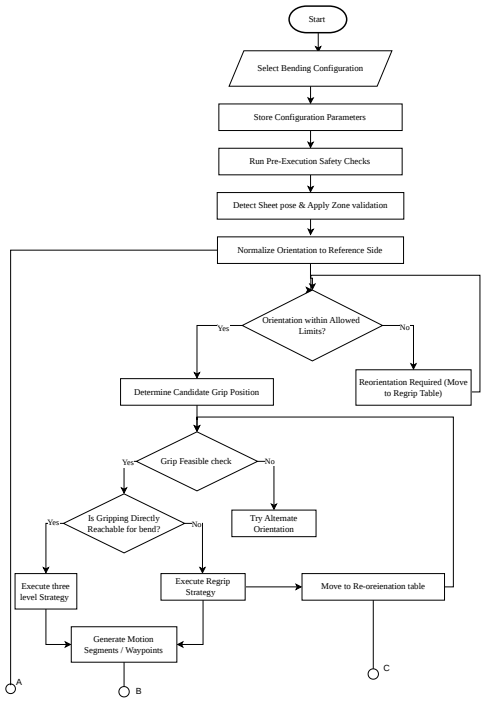
face using an authenticated WebSocket connection, ensuring compliance with SR-01 and SR-02. All process parameters are loaded from a central YAML Ain't Markup Language (YAML) configuration file at the beginning of each execution cycle. This enables adaptation to different workpiece geometries without modifying the source code, thereby satisfying NFR-02.

Operator input is provided through an interactive 3D sheet bending viewer integrated into the drag&bot operator cockpit. This interface improves usability and reduces the likelihood of incorrect input selection. Ubuntu Linux is used as the development platform due to its compatibility with ROS-based systems and stable integration with the drag&bot environment [25].

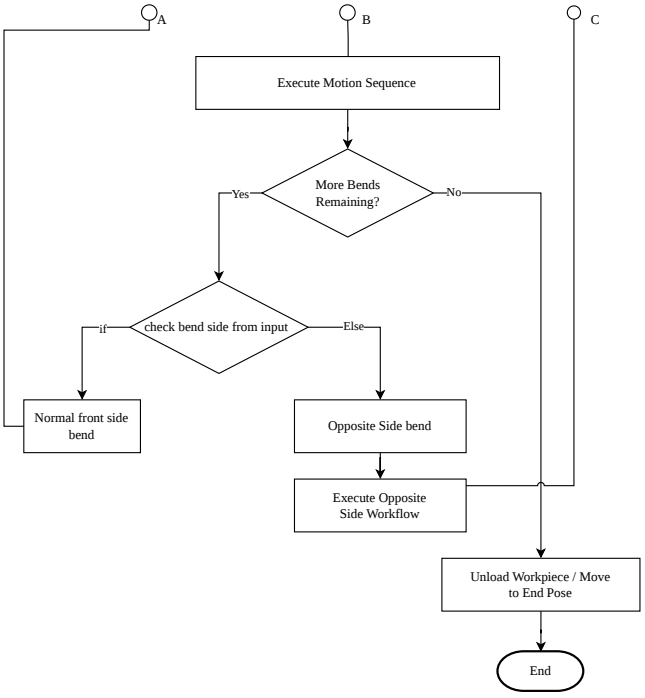
Figure 5.1 summarises the end-to-end execution sequence from session initialisation to bending execution. The workflow begins with configuration selection and parameter initialisation, followed by pre-execution safety validation to ensure that all operating conditions are within acceptable limits. Based on the detected sheet pose, a valid pickup orientation is determined, after which feasible grip positions are computed using constraint-based evaluation.

If direct execution is not feasible, the system automatically applies regrip strategies to achieve a reachable configuration. The validated task sequence is then converted into motion commands and executed by the robot. The process iterates over all required bending steps and includes additional handling for opposite-side operations before concluding with the final unloading stage. This structured workflow ensures deterministic execution, consistency, and repeatability across all functional subsystems within the automated bending process.

The resulting data flow is summarised in Table 5.1. The table separates the information entering each stage from the processing result produced for the next module, so that the implementation can be read as a continuous transformation from user and geometry input into executable runtime commands.



(a) Upstream stages from configuration loading to orientation and grip planning.



(b) Downstream execution stages from regrip or flip handling to trajectory execution and completion.

Figure 5.1: Complete end-to-end system workflow. Part a shows the upstream stages, while Part b shows the downstream execution stages from planning continuation to completion.

Step	Input	Processing Stage	Output	Next Module
1	Workpiece geometry, bend-side selection, TCP, gripper, and safety parameters	Configuration and operator-input initialisation	Structured task and configuration data	Safety validation
2	Configuration data, fixture states, and operator start decision	Pre-execution validation and runtime readiness checks	Permission to continue or controlled stop	Pose acquisition and zone classification
3	Tracked sheet pose and workspace zone	Orientation admissibility and pickup-side validation	Selected pickup orientation or conservative fallback	Grip planning
4	Selected orientation, sheet geometry, and gripper parameters	Grip candidate generation, filtering, and force validation	Feasible grip configuration and pickup poses	Regrip and sequence planning
5	Current workpiece pose, selected grip, and target bending pose	Direct reachability, regrip, and opposite-side handling logic	Validated task sequence with required handling actions	Trajectory generation
6	Ordered task poses and handling actions	Interpolation, kinematic validation, path reduction, and command formatting	Executable robot motion and gripper commands	AnyRobot runtime communication

Table 5.1: End-to-end data flow from workpiece and process input to executable robot commands.

5.2 System Architecture, Interfaces, and Configuration

The following section describes the software structure used to implement this pipeline before the individual planning functions are discussed.

5.2.1 Layered Software Architecture and Module Responsibilities

The implemented system follows a systematic layered architecture in which each module depends on the functionality provided by the underlying layer. This architectural design supports press brake automation by enabling independent testability, structured constraint evaluation, maintainability, and safe replaceability of components.

Testability is achieved by allowing modules to be evaluated using pose data and YAML-based configuration inputs without requiring an active WebSocket connection. This enables verification of orientation classification and grip feasibility computations prior to establishing communication with the runtime environment.

Replaceability is ensured through modular separation. Individual modules can be

modified or replaced without affecting the entire system, as each layer operates independently within its defined responsibility. This supports maintainability and satisfies NFR-02.

Safety ordering is enforced by implementing validation prior to motion generation. This ensures that safety checks and kinematic feasibility evaluations are performed before execution, thereby satisfying FR-07 and NFR-03.

Five layers are defined within the system architecture.

Layer 1 (Input and Configuration) is responsible for acquiring all required parameters from the input file, including the operator-defined bend-side selection, before establishing the WebSocket connection.

Layer 2 (Geometry and Constraint Evaluation) performs the computation of geometric and physical constraints based on input pose data.

Layer 3 (Planning and Decision Logic) determines the overall execution strategy and coordinates the interaction between Layers 2 and 4 without directly generating motion commands. This layer prepares the execution sequence based on evaluated data.

Layer 4 (Motion Generation and Communication) serves as the interface for robot communication. It generates Linear motion command (LIN) and Point-to-point motion command (PTP) waypoints, converts them into JavaScript Object Notation (JSON) format, and transmits them via an authenticated WebSocket connection.

Layer 5 (Safety and Validation) is implemented as an independent validation layer prior to execution. This layer identifies invalid poses and generates warnings before transmitting commands to the robot runtime.

The layer numbering describes responsibility rather than chronological execution. During runtime, validation is invoked before motion dispatch and monitoring continues during execution, while communication remains centralised in Layer 4. This distinction is important because the safety layer is architecturally separated but operationally active before and during command execution.

The layered architecture is implemented as a fixed dependency structure in which input and configuration handling, geometric evaluation, planning logic, motion generation, and safety validation remain clearly separated. This ordering ensures that validation is enforced before motion dispatch and that communication is centralised through the runtime layer.

The progression of data throughout a bending cycle follows a structured flow across the system layers. Initially, sheet geometry, bend-side selection, and tool parameters are provided as input. These inputs are processed within the constraint evaluation layer, where raw pose data is transformed into a validated pickup orientation

and a feasible grip configuration.

Based on this information, an execution strategy is determined and translated into motion commands, which are transmitted to the runtime environment through the AnyRobot interface. Errors introduced at any stage may propagate through subsequent computations and affect execution outcomes.

Accurate orientation determination is critical for successful workpiece acquisition, while grip feasibility evaluation ensures stable handling during transport. During trajectory generation, a balance must be maintained between path accuracy and controller limitations, requiring controlled reduction of trajectory points while preserving motion characteristics.

Module responsibilities and dependency discipline.

The implementation consists of software modules organised across the five system layers. Table 5.2 summarises the grouped module contracts by showing what each functional group receives, what it produces, and how failure or stop conditions are handled before or during execution.

Layer / Module Group	Input	Output	Failure or Stop Condition
L1 – Input and configuration setup	Operator request, workpiece data, YAML parameters	Validated task configuration; operator interface	Stop if mandatory parameters or runtime settings are missing.
L5 – Safety validation and monitoring	Operator decision, fixture pose, safety limits, runtime feedback	Continue permission, warning, or status feedback	Controlled stop if tolerance or motion feedback fails.
L2 – Zone and pose classification	Tracked sheet pose	Workspace zone and valid pose status	Stop if the sheet is missing or lies in the dead-band region.
L2 – Orientation planning	Sheet pose, bend side, zone, reference offsets	Pickup orientation or fallback candidate	Fallback or stop if no orientation is admissible.
L2 – Grip planning	Sheet geometry, holes, gripper data, orientation	Feasible suction-cup layout and pickup poses	Reject grip if clearance, spacing, or holding-force checks fail.
L3 – Primary workflow orchestration	Configuration, bend selection, grip result, target poses	Ordered primary-side task sequence	Controlled stop if required planning output is infeasible.
L3 – Regrip and opposite-side handling	Current pose, target bend, Side E need, feasibility result	Direct path, regrip, or flip-and-continuation sequence	Controlled stop if no reachable regrip or post-flip solution is found.
L4 – Trajectory generation and waypoint handling	Task poses, handling actions, motion limits	Reduced waypoint sequence and motion command list	Reject motion if workspace, singularity, or inverse-kinematics checks fail.
L4 – Runtime communication and actuation	Commands, gripper actions, object requests, runtime endpoint	AnyRobot exchange, IO state, execution feedback	Stop or retry if authentication, command acknowledgement, or IO action fails.

Table 5.2: Grouped module contract overview showing the main inputs, outputs, and stop conditions across the implemented system layers.

The modules exchange data as part of one execution pipeline, but their dependencies are restricted to preserve a clear separation of responsibilities. This means that modules cooperate during program generation while specific tasks, such as communication, safety validation, and motion dispatch, remain assigned to defined layers.

First, runtime communication is centralised in Layer 4. The WebSocket client and the command type definitions are the only modules that construct WebSocket messages and perform direct network communication. Modules in the other layers therefore access the runtime environment through this communication layer instead of opening their own connections. As a result, authentication handling, message formatting, and connection recovery are implemented in one place and remain consistent across all runtime interactions.

Second, safety validation is separated into Layer 5. The safety checker is executed before the runtime workflow continues and is not called directly by lower-layer planning modules during motion execution. This fixed ordering ensures that safety checks cannot be skipped, delayed, or reordered by the planning and execution logic.

Third, robot motion dispatch is centralised through the trajectory generator. All waypoint sequences are passed through this module before being transmitted to the runtime environment. This ensures that workspace constraints and singularity-related checks are applied consistently before motion execution.

After defining these dependency rules, the Layer 4 communication mechanism is described in more detail.

5.2.2 Runtime Communication and Configuration

The AnyRobot interface is the drag&bot communication interface that enables external planning software to interact with the robot runtime. In the implemented system, all runtime communication is performed through this interface, which acts as the supported external Application Programming Interface (API) for drag&bot. Through ROSBridge v2, the interface translates JSON messages exchanged over a persistent WebSocket connection into native ROS operations. This allows the planning software to run in standard Python 3 without requiring a local ROS installation [21].

The communication layer is implemented as a Python-based runtime client with structured command objects. The planning modules do not communicate with ROS directly. Instead, they pass validated motion waypoints, input-output actions, object-management requests, and operator-interaction requests to the communication layer. The runtime client serialises these requests into JSON messages, trans-

mits them through the authenticated AnyRobot WebSocket connection, and waits for the corresponding runtime response before the workflow continues.

A persistent WebSocket connection is preferred over repeated Hypertext Transfer Protocol (HTTP) requests because the system must exchange commands and state information continuously during execution. WebSocket communication reduces overhead compared with repeated HTTP polling for real-time data exchange [29]. Rosbridge enables non-ROS applications to communicate with ROS through socket-based interfaces [30]. As illustrated in Figure 5.2, the planning software communicates with the runtime through a configured endpoint of the form `ws://<runtime-ip>:<port>`. Dedicated service and topic interfaces separate motion execution, object management, gripper control, operator interaction, and execution feedback.

Table 5.3 summarises the principal runtime interfaces used in the implemented workflow. The YAML configuration is not exchanged during runtime; it is loaded once during initialisation so that missing or inconsistent parameters can be detected before command transmission begins.

Runtime Function	**Interface**	**Use in the Implemented Workflow**
Motion command dispatch	`/command_list`	Transmits generated LIN and PTP waypoint sequences to the runtime environment.
Motion result feedback	`/command_result`	Returns execution results and motion status information after command dispatch.
Digital input-output control	`/robot/set_digital_ios`, `/robot/get_digital_ios`	Controls and queries vacuum gripper signals and related robot input-output states.
Object management	`/dnb_object_manager/...`	Queries and updates workpiece and fixture states used during attachment, detachment, and validation.
Operator interaction	`/human_interaction/request_hi`	Requests operator confirmation or warning acknowledgement through the drag&bot interface.

Table 5.3: Principal AnyRobot runtime interfaces used by the implemented communication layer.

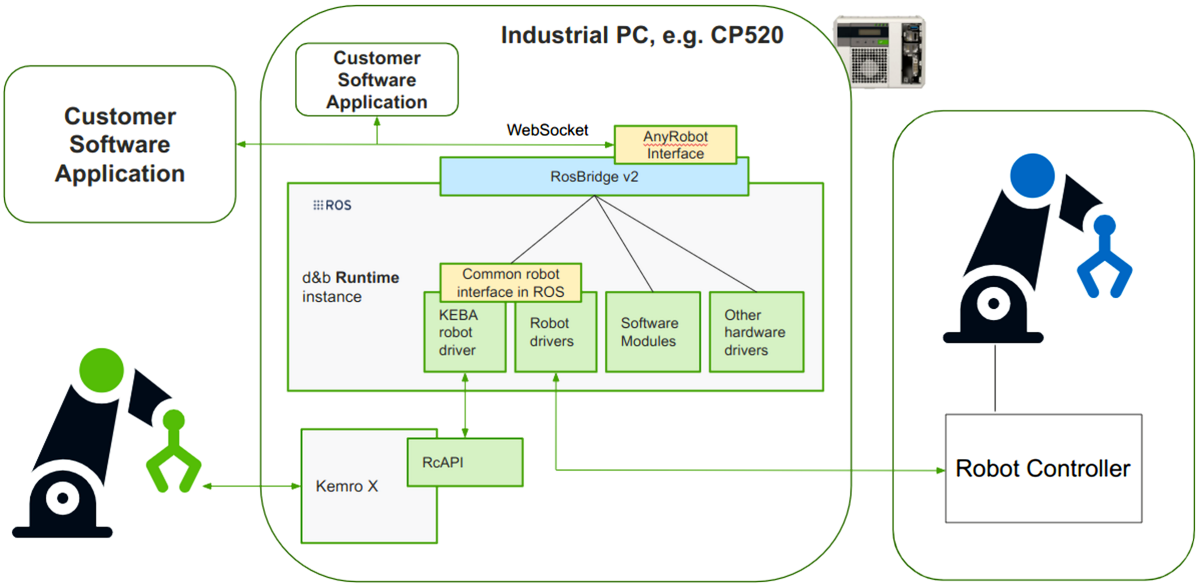


Figure 5.2: Communication architecture of the implemented system, illustrating the interaction between the planning software, the drag&bot runtime environment, and the robot controller [21].

When the connection is established, the runtime client performs the ROSAuth handshake defined by the drag&bot AnyRobot interface specification [21]. This step prevents unauthorised command transmission through the local WebSocket endpoint. The shared secret required for the Secure Hash Algorithm 512-bit (SHA-512)-based authentication code is stored outside the source code as an environment variable. After authentication, planning modules exchange only validated command objects with the communication layer; the low-level message formatting remains encapsulated inside the runtime client.

Motion execution is treated as a blocking runtime interaction in the communication layer. Before dispatching a movement, the client subscribes to `/command_result`; after the runtime returns the result code, the client releases the motion sequence and the planning workflow continues. Additional runtime supervision through an independent monitoring subprocess is described in Section 5.4.

The implemented communication sequence is summarised in Listing 5.1. The listing abstracts the runtime client behaviour to the essential steps required for authenticated command transmission, result monitoring, and controlled termination after a failed command.

```
Load runtime endpoint and authentication parameters
Open persistent WebSocket connection to AnyRobot runtime
Perform ROSAuth handshake using external secret
Subscribe to the motion result feedback topic

for each validated command group in execution_sequence do
    serialise motion and handling actions as JSON command list
```

```
    transmit command list through the runtime client
    wait for blocking result feedback

    if result code indicates failure then
        stop remaining command dispatch
        notify operator through human-interaction service
        record stop reason for evaluation
        terminate execution safely
    end if
end for

Close runtime interaction after all command groups complete
```

Listing 5.1: Pseudocode for runtime command dispatch and result handling.

Configuration. The YAML configuration file serves as the engineering-level entry point for process-related parameters and provides a flexible mechanism for adapting the system to different workpieces without modifying the underlying source code, thereby satisfying NFR-02. At the beginning of each session, the file is parsed once, and the resulting configuration data is stored as a dictionary that is shared across all modules in Layers 2 to 4. The structure of the file is organised into six clearly defined top-level sections, each corresponding to a specific functional aspect of the system architecture, which supports clarity and maintainability. YAML is selected over JSON primarily due to its support for inline comments. This allows engineers to document the meaning and physical relevance of individual parameters directly within the configuration file, eliminating the need to consult external documentation. To ensure robustness and security, the configuration is loaded using a safe parsing method, which prevents the execution of unintended code in the event that the file is modified or corrupted. A key design decision concerns the handling of spatial consistency during the regrip process. Rather than defining approach and placement poses independently, all such poses are derived relative to a single reference pose defined for the orientation table. For example, the approach position is calculated by applying a fixed vertical offset to the placement pose. This design ensures that any change in the position of the orientation table requires only a single modification within the configuration file. In contrast, storing these poses independently could introduce inconsistencies: if the table position is updated but the corresponding approach position is not, the system may execute motions based on outdated spatial data. Such inconsistencies are particularly problematic because they may not be detected during configuration validation, as each parameter remains individually valid despite being collectively incorrect.

The configuration is organised into runtime communication settings, robot and TCP parameters, workpiece dimensions and bend-side selection, gripper parameters, motion limits, and safety thresholds. This grouping is sufficient for adapting the virtual cell to different rectangular workpieces while keeping the planning logic un-

changed. A generalised YAML example is retained as an electronic supporting file rather than reproduced in the main chapter.

While the YAML file provides a structured and flexible way to define engineering configuration parameters, it is less suitable for normal operator interaction because the workpiece geometry and selected bend sides are not visually represented. Therefore, an operator interface with an interactive three-dimensional viewer was implemented to provide clearer visual feedback and a more convenient input method during execution.

5.2.3 Operator Interface and Three-Dimensional Sheet Viewer

The operator interface denotes the user-facing dashboard used during execution to provide process input without editing configuration files. It combines bend-side selection, workpiece visualisation, mode selection, workflow status, and confirmation controls in a single view inside the drag&bot Operator Cockpit. This allows the operator to inspect the intended bending input visually before automated planning begins.

Operator input for each execution cycle is provided through an interactive three-dimensional sheet bending viewer developed in this work. The viewer is implemented as a local Plotly Dash application and renders the workpiece as an interactive mesh derived from the corresponding Standard Tessellation Language (STL) file. Presenting the sheet directly as a 3D model makes bend-side selection more intuitive than relying only on abstract side labels, while the planning system handles the underlying geometric calculations.

The viewer is embedded in the drag&bot Operator Cockpit through a Custom URL panel, so configuration, monitoring, and execution remain within one interface. The operator-facing dashboard is shown in Figure 5.3. The figure illustrates the embedded control panel, including bend-side selection, workflow status, mode selection, and the three-dimensional preview used for visual checking before program generation.

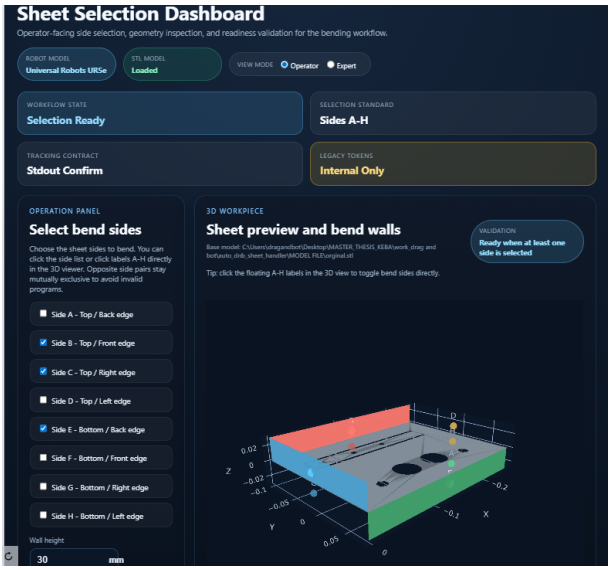


Figure 5.3: Operator-facing sheet selection dashboard embedded in the drag&bot Operator Cockpit, showing bend-side selection, workflow status, mode selection, and the three-dimensional workpiece preview.

To accommodate different levels of operator expertise, the interface is divided into basic and expert modes. Basic mode limits interaction to predefined bend-side selections and wall-height input, while expert mode exposes additional geometric, tool, and pose parameters for controlled engineering adjustment.

In parallel with user input, the system analyses the imported mesh to identify internal features such as holes. The resulting feature information is used during grip planning to exclude unsuitable regions, and the workpiece mass is estimated from material density when required for holding-force validation. Once the operator confirms the selection, the parameters are transmitted to the planning module in a structured format and further interaction is disabled to prevent configuration changes during active robot motion.

The complete sequence of operator interaction and system processing is summarised in Figure 5.4, which illustrates the progression from initial configuration through planning, strategy selection, and final execution.

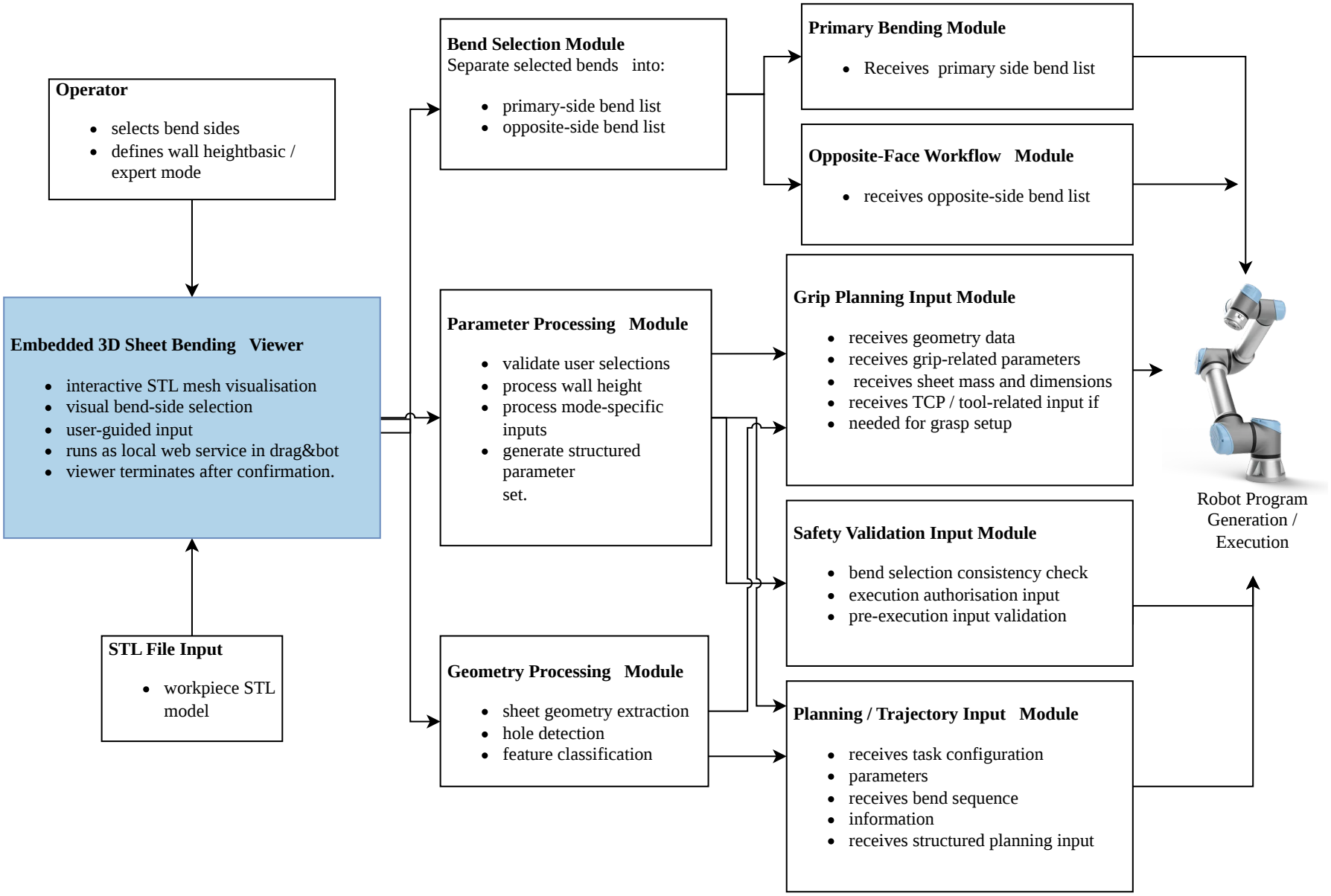


Figure 5.4: Operator workflow. Every gate must be passed before the next begins. All operator interactions occur within the drag&bot environment.

5.3 Simulation Environment and Pose Acquisition

With the architectural framework established in Section 5.2, a controlled environment is required to execute, test, and validate the developed system prior to deployment on physical hardware. This section presents the development and simulation environment used in this work, including the drag&bot platform, the robot model and tool configuration, and the spatial zone classification strategy used to structure the workspace.

Simulation platform, robot model, and TCP.

The drag&bot platform is used as the primary environment for system development, integration, and validation. It provides a virtual robot controller together with the AnyRobot WebSocket interface, enabling the planning software to interact with the robot runtime without requiring direct access to physical equipment [21]. This supports safe testing, repeatability, and efficient iteration during development. The use of virtualised environments is consistent with current industrial trends in digital manufacturing and digital twin-based validation [13].

All modules are implemented in Python and executed within an Ubuntu-based environment alongside the drag&bot runtime. Communication is established through a single authenticated WebSocket connection (e.g., `ws://<local-ip>:9091`), which is used for motion execution, Input/Output (IO) control, object management, and operator interaction. This unified communication structure ensures consistent behaviour across all subsystems.

All session-specific parameters are initialised at the beginning of execution and explicitly passed between modules. This avoids hidden dependencies and ensures that all components operate on consistent data, supporting modularity and maintainability.

Within this environment, the UR5e manipulator is used as the robot model. The robot provides six rotating joints, a reach of approximately 850 mm, a payload capacity of 5 kg, and a maximum joint speed of 180°/s [5]. These properties are suitable for the simulated press-brake handling task because the sheet must be picked, reoriented, flipped, and positioned at different approach angles during the bending sequence. The official joint range specification further supports this selection, as all joints provide a range of $\pm 360^\circ$ except the elbow joint, which provides $\pm 160^\circ$ [5]. In the implemented simulation setup, this manipulator model is therefore used to evaluate generated motion targets for reachability and feasibility before execution.

The kinematic structure of the UR5e, including the six joint frames used to describe the manipulator chain, is illustrated in Figure 5.5. This representation is included to clarify the joint-based motion capability that is relevant for large orientation changes

during sheet handling.

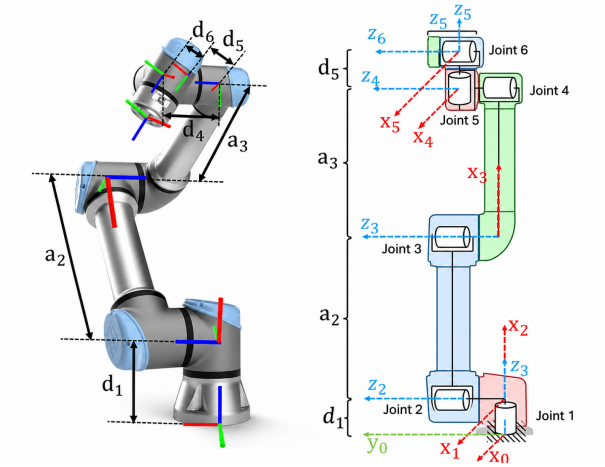


Figure 5.5: Kinematic structure and coordinate-frame assignment of the UR5e manipulator used to illustrate the six-joint robot layout [31].

The end-effector used in the simulated cell is a two-cup vacuum gripper designed and modelled according to the company-provided tooling requirements for the project. The gripper geometry determines the position and orientation of the tool reference frame and therefore directly defines the TCP used for all generated motion targets. In particular, the measured offset between the robot flange and the vacuum-cup contact region is represented in the configured TCP, ensuring that planned robot poses correspond to the actual gripping position on the sheet.

The TCP used for all motion targets is defined in the drag&bot tool management system as `project_3_tcp`, in accordance with requirement SR-03. The TCP is specified using the ZYX Euler convention with offsets relative to the robot flange: $x = 0{,}000$ m, $y = 0{,}034$ m, $z = 0{,}194$ m, $RZ = -90{,}0°$, $RY = -75{,}0°$, and $RX = 0{,}0°$. These parameters ensure that all generated poses remain consistent with the physical gripper configuration. The TCP identifier is provided through the operator input, allowing future adjustments without modification of the underlying source code. The configured UR5e TCP used for the generated motion targets is shown in Figure 5.6.

Virtual cell layout, object pose retrieval, and spatial zone classification.

The simulated robotic cell within the drag&bot environment includes the robot, press brake, orientation table, flip station, vacuum gripper, and a tracked sheet object, which serves as the reference for all geometric computations. The complete virtual test environment is shown in Figure 5.7. This operator-panel view documents the implemented cell layout used during development and validation, including the relative arrangement of the robot, press brake, handling stations, and tracked workpiece. The spatial configuration of these elements reflects the intended physical setup, ensuring that simulation results are representative of real-world operation.

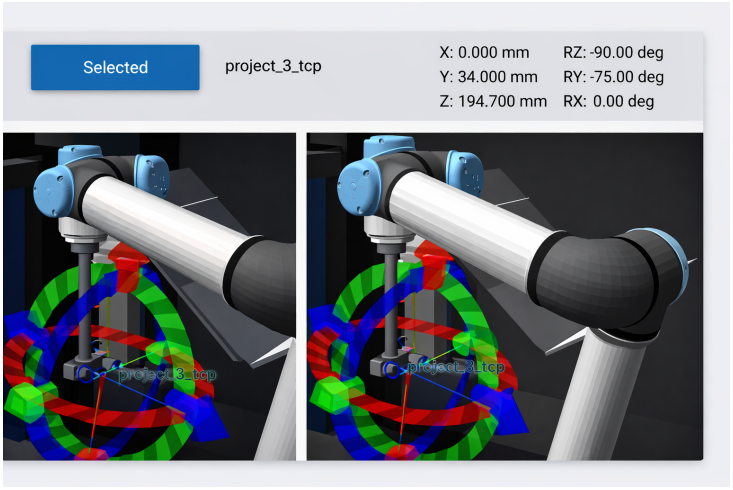


Figure 5.6: Configured TCP of the UR5e with the modelled two-cup vacuum gripper used for sheet handling.

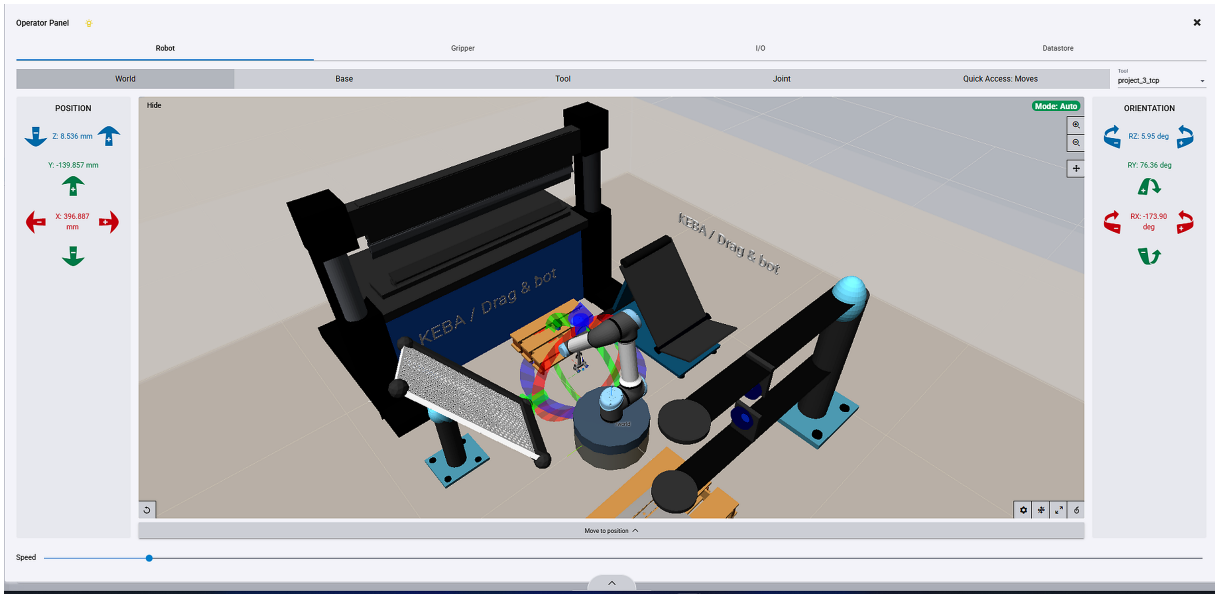


Figure 5.7: Virtual test environment of the robotic press brake cell in the drag&bot operator panel, showing the UR5e robot, press brake, handling stations, vacuum gripper, and tracked sheet object used for simulation-based development and validation.

In this work, object tracking refers to the retrieval of the current sheet state from the drag&bot object manager rather than to an independent perception algorithm. In accordance with the system assumptions defined in Section 3.2.4, no camera, LiDAR, or external vision-based sensing is used. Instead, the simulated sheet object is maintained within the drag&bot runtime and identified through a persistent object identifier. The planning software queries this object state through the AnyRobot interface and reads the latest available position and orientation in the robot world frame. Where available, the stored object dimensions are also read to check consistency with the configured workpiece geometry.

At the beginning of each execution cycle, the system retrieves the six-degree-of-freedom pose of the sheet from the object manager. The pose is expressed in the

robot world coordinate frame, where the origin is located at the robot base. Accurate pose acquisition is a fundamental requirement in robotic manipulation systems and is widely addressed in modern robotics literature [12]. Based on this information, the position of the sheet centroid in the XY plane is used to classify the workpiece into one of four spatial zones.

The workspace is partitioned into four regions relative to the robot base. A centroid with negative X and positive Y corresponds to Zone 1 (rear-left), while a centroid with negative X and negative Y corresponds to Zone 2 (front-left). Positive X and positive Y correspond to Zone 3 (rear-right), and positive X with negative Y defines Zone 4 (front-right). The classification uses dead-band thresholds of ± 10 mm in the X direction and ± 50 mm in the Y direction around the coordinate axes. In this context, the dead band denotes a neutral region near a zone boundary in which no zone is assigned. These values are implementation parameters selected for the simulated cell rather than general robotics limits. They prevent unstable zone assignment when the sheet centroid lies close to a boundary, where small pose variations could otherwise change the sign of the coordinate and assign the workpiece to a different zone. The larger Y threshold reflects the wider separation required along the front–rear direction in the implemented cell layout.

Zones 3 and 4 lie within the primary reachable workspace of the robot and therefore allow direct execution of pickup and bending operations. In contrast, Zones 1 and 2 correspond to regions with limited reachability. In these cases, the system automatically initiates a repositioning step, moving the workpiece to an intermediate configuration within the reachable workspace before continuing execution. This ensures that all subsequent operations remain within kinematic limits without requiring operator intervention.

If the sheet centroid lies within the dead-band region defined by $|x| < 10$ mm or $|y| < 50$ mm, the classification is considered unresolved. This occurs near the coordinate axes, where small variations in tracking data may lead to inconsistent zone assignment. To prevent incorrect execution, the system terminates the current session and prompts the operator to reposition the workpiece before restarting.

Figure 5.8 illustrates the resulting zone assignment and the neutral dead-band region around the coordinate axes.

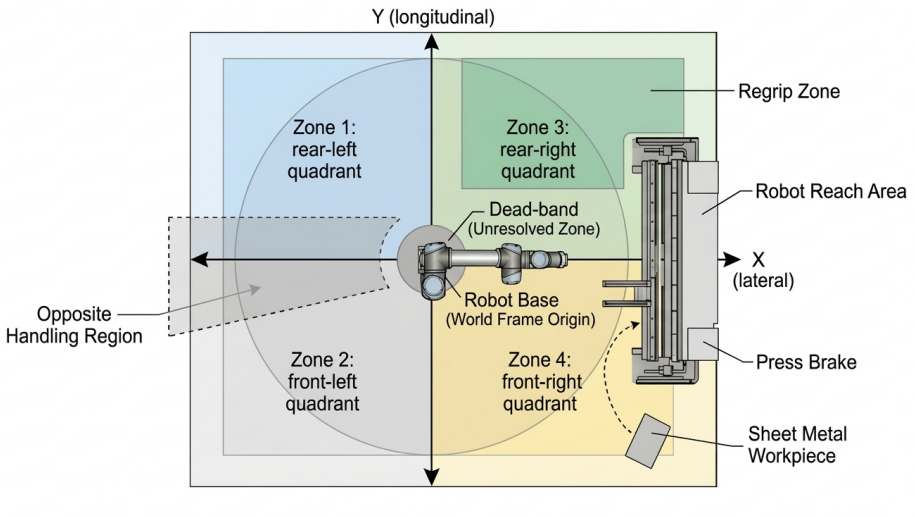


Figure 5.8: Spatial zone classification of the robot workspace based on the sheet centroid position.

The implementation first applies the dead-band check and only assigns a zone when the centroid lies outside the unresolved boundary region. The signs of the remaining X and Y coordinates then determine the corresponding quadrant. This order ensures that boundary cases are rejected before a zone-dependent handling strategy is selected.

Having defined the simulation environment, robot model, TCP configuration, object pose retrieval, and workspace segmentation strategy, the following sections describe the functional subsystems in the order of execution within a bending cycle. The workflow begins with safety validation, continues with workpiece orientation and grip planning, and then proceeds to gripper actuation, regrip or flip handling, and trajectory generation.

5.4 Safety Validation and Runtime Monitoring

Once the virtual cell layout and coordinate references are defined, the first operational step is to check whether execution can begin safely. The input to this stage is the current object-manager state of the fixed cell references and the operator's start decision. The output is either permission to continue with planning or a controlled stop before any robot motion is issued.

Safety validation is performed prior to motion generation to ensure that unsafe conditions and infeasible configurations are identified before robot execution, in accordance with NFR-03, NFR-04, and NFR-05 [12]. The system incorporates three complementary safety mechanisms. First, a fixture pose deviation check verifies that critical objects and fixtures remain within their expected positions before execution. Second, a blocking operator confirmation dialog prevents the workflow from

continuing until the operator has confirmed the setup. Third, a runtime motion error monitoring module supervises motion execution and detects command failures during operation. In addition, all trajectory waypoints are generated and verified using robot kinematics to ensure reachability and feasibility before execution, thereby preventing invalid motion commands from being issued [12].

Fixture pose deviation validation.

The fixture pose deviation check retrieves the current poses of fixed cell references from the drag&bot object manager and compares them with reference poses obtained during initial cell calibration. The monitored elements include the press brake reference, the finished-part placement positions on the regrip table, and the placement and attachment references at the flip station. These elements are treated as fixed references within the planning system, as grip positions, trajectory waypoints, and placement operations are computed under the assumption that their positions remain unchanged. If one of these references is moved, the corresponding position parameters in the configuration or operator interface must be updated before execution. Otherwise, the generated motion targets may remain geometrically valid in software while no longer matching the physical or simulated cell layout.

The active workpiece is treated differently from these fixed references. Since the sheet may be placed in different positions and orientations before a bending cycle begins, its pose is not compared against a single fixed reference pose. Instead, the system verifies that a valid sheet pose is available from the object manager and passes this pose to the zone classification and workpiece orientation strategy described in Sections 5.3 and 5.5. This separation is necessary because arbitrary workpiece placement is part of the intended workflow, while movement of fixed cell references indicates an inconsistent setup.

Each fixed reference is evaluated using both positional and orientation criteria. The positional deviation is computed using the standard three-dimensional Euclidean distance according to Equation 5.1 [32]:

$$\Delta_{pos} = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2} \tag{5.1}$$

In Equation 5.1, Δx, Δy, and Δz denote the differences between the current and reference positions along the three Cartesian axes. The resulting value Δ_{pos} is therefore the three-dimensional distance between the expected reference position and the currently reported position.

The orientation deviation is defined in this implementation by Equation 5.2:

$$\Delta_{ang} = \max\left(|\Delta RZ|, |\Delta RY|, |\Delta RX|\right). \tag{5.2}$$

In Equation 5.2, ΔRZ, ΔRY, and ΔRX denote the differences between the current and reference orientations about the corresponding axes. The maximum absolute value is used because a large deviation around any one axis is sufficient to make the reference pose unreliable for planning.

A fixture is considered out of tolerance when Δ_{pos} exceeds 5 mm or Δ_{ang} exceeds 0,10 rad (5,7°). The positional threshold is selected to remain below the minimum clearance used in grip planning. This ensures that fixture misalignment does not produce misleadingly valid grip configurations. The angular threshold corresponds to the minimum deviation that would affect the accuracy of vacuum cup contact on the sheet surface.

If one or more fixtures exceed the defined thresholds, the system generates a structured warning listing all affected fixtures together with their measured positional and angular deviations. This warning is presented through a blocking Human Interaction (HI) dialog titled *Fixture position warning*, allowing the operator to either proceed with explicit acknowledgement or cancel execution for further inspection. The operator's decision is recorded, ensuring that responsibility for accepting out-of-tolerance conditions remains with the human operator.

If a fixture is not detected in the object manager response, it is treated as missing and automatically flagged as a validation failure.

Operator authorisation and runtime error monitoring.

The operator confirmation dialog is positioned as the first step in the planning orchestrator startup sequence, preceding all subsequent processes including fixture validation, visualisation, and motion execution. The dialog enforces a mandatory decision: either initiate the automated workflow or cancel execution. The orchestrator does not proceed until a response is received from the HI service, ensuring that execution cannot begin without explicit operator authorisation. If the operator selects cancel, the process terminates without issuing any command to the robot runtime.

Although runtime monitoring becomes active during robot motion, it is described in this section because it belongs to the same safety layer as the pre-execution checks.

In addition to pre-execution validation, runtime supervision is implemented through a motion error monitoring module operating as a background subprocess. This component subscribes to the `/command_result` ROS topic via an independent WebSocket connection, enabling continuous monitoring of execution outcomes without interrupting the main command flow. When a non-zero result code is detected, indicating a motion execution failure, the system immediately triggers a modal HI notification to inform the operator.

The monitoring module does not directly interrupt the execution pipeline. Instead, it provides immediate feedback through the HI interface and requires operator acknowledgement before further interaction is permitted. This ensures that execution anomalies are not overlooked while maintaining operator authority over subsequent decisions.

Together, these mechanisms establish a layered safety structure. Pre-execution validation ensures safe initial conditions, while runtime monitoring provides immediate feedback on execution failures. Both mechanisms utilise the drag&bot HI service, ensuring a consistent and controlled interface for safety-related interactions. With these safeguards in place, the subsequent section addresses the workpiece orientation strategy.

5.5 Orientation and Grip Planning

Following safety validation, the first geometric planning step in each bending cycle is the determination of a valid pickup orientation for the sheet. The input to this stage is the tracked sheet pose and the active workspace zone. The output is a validated pickup orientation that is passed to grip planning. This step is required because the sheet may be placed in arbitrary orientations within the workspace, while only a subset of these orientations is kinematically feasible for the target bending operation.

5.5.1 Zone-Based Orientation Admissibility and Pickup Validation

Before grip planning is performed, the tracked sheet pose obtained from the object manager is checked from two complementary viewpoints. First, the in-plane rotation of the sheet must be reachable from the current workspace zone. Second, the vacuum gripper must approach the correct sheet surface so that the suction cups can establish contact.

Both checks are required because the sheet may be placed in different orientations, and a detected pose may be geometrically valid but still unsuitable for pickup if the yaw angle is outside the reachable range or if the tool frame is oriented towards the wrong side of the sheet. In the latter case, the suction cups may not contact the sheet correctly, which can lead to failed gripping, incorrect attachment in the simulation model, or invalid motion targets in the following planning stages.

The admissible range of sheet rotation about the Z-axis is defined by a zone-dependent window $[RZ_{\text{min}}, RZ_{\text{max}}]$. This constraint reflects the limited rotational capability of the robot end-effector within different regions of the workspace [12]. Only the rotation about the Z-axis is used for this admissibility window because it

represents the in-plane orientation of the sheet within the horizontal workspace. This yaw angle determines whether the robot can approach the selected sheet side from the current zone without exceeding its reachable orientation range. Since the reachable orientation range varies with the spatial position of the sheet, zone classification is performed prior to orientation evaluation. The values RZ_{min} and RZ_{max} are predefined admissibility limits for each workspace zone. They are established during system setup by evaluating the reachable sheet yaw range in the simulated cell with the configured robot, TCP, gripper, and surrounding stations. These limits are then stored in the configuration and are not recalculated during execution. If a candidate orientation falls outside the configured range, it is treated as not admissible and the next level of the orientation strategy is evaluated. This prevents the selection of geometrically valid but unreachable poses.

The rotations about the X- and Y-axes are evaluated in the same validation stage to verify the pickup side. These angles describe the tilt of the tool frame relative to the sheet surface, while the corresponding direction check indicates whether the tool Z-axis points towards or away from the required contact side. If the evaluated direction does not correspond to the required approach side, the corrective transformation in Equation 5.3 is applied:

$$RZ' = \text{wrap}(RZ + 180^\circ), \quad RY' = -RY, \quad RX' = \text{wrap}(RX + 180^\circ) \tag{5.3}$$

In this transformation, RZ is shifted by 180° to rotate the in-plane approach direction, while RY is mirrored and RX is shifted by 180° to obtain an equivalent pose from the opposite side of the sheet. The function $\text{wrap}(\cdot)$ normalises the resulting angle to the configured angular range. The pickup position itself is not changed; only the orientation is corrected so that the gripper approaches the sheet from the valid side.

5.5.2 Three-Level Orientation Selection Strategy

After zone-based admissibility and pickup-side validation, the system must still determine which equivalent tool orientation should be used for the detected workpiece pose. This decision is required because the TCP is fixed relative to the two-cup vacuum gripper shown in Figure 5.6. When the sheet is placed with a different in-plane orientation, the required TCP orientation changes accordingly. Some of these orientations may be outside the reachable range of the robot for the current workspace zone, while others may orient the suction cups towards an unsuitable side of the sheet. Therefore, the detected orientation cannot always be forwarded directly to grip planning.

To automate this decision with respect to the tracked workpiece position and orien-

tation, a three-level selection strategy is applied. The levels are evaluated sequentially, and execution terminates as soon as a valid orientation satisfying admissibility constraints is identified. This improves robustness while prioritising geometrically consistent and kinematically feasible configurations.

The workpiece is defined using four reference faces established during system commissioning, each associated with a fixed angular offset about the sheet's local vertical axis ($0°$, $180°$, $+90°$, $-90°$). These offsets allow the system to normalise arbitrary orientations into a consistent reference frame.

In this context, a reference face denotes a predefined side of the rectangular sheet that is used as an orientation reference for pickup planning. It is not a separate physical object, but a selected sheet-side orientation defined during system setup. The reference face is chosen because its associated pickup orientation provides favourable reachability for the configured UR5e TCP and the two-cup vacuum gripper. It therefore represents the preferred direct pickup orientation whenever the tracked workpiece pose can be transformed into this frame and still satisfies the admissibility checks described above.

Level 1: Reference-face equivalent orientation.

At the highest priority level, the tracked orientation is converted into a reference-face equivalent by removing the face-specific offset according to Equation 5.4:

$$R_{\text{new}} = R_{\text{raw}} \cdot R_z(-d_{\text{face}}) \tag{5.4}$$

In this equation, R_{raw} denotes the tracked orientation of the sheet, and d_{face} denotes the angular offset of the selected reference face. The rotation $R_z(-d_{\text{face}})$ is applied about the local Z-axis to remove this face offset from the detected pose. The result, R_{new}, is the reference-face equivalent orientation that is then checked against the zone-dependent admissibility window.

The corrected rotation matrix is then converted into ZYX Euler angles using standard rotation-matrix decomposition [12], [33]. The resulting angles are denoted as RZ, RY, and RX and are used for the admissibility and pickup-side checks.

After this transformation, Level 1 is accepted only if the resulting orientation satisfies the validation conditions introduced above. The computed RZ must lie within the zone-specific admissibility window, expressed as $RZ_{\text{min}} \leq RZ \leq RZ_{\text{max}}$, and the corresponding tool orientation must remain valid for pickup from the required sheet side. If either condition is not satisfied, the reference-face equivalent orientation is considered unsuitable for execution, and the strategy proceeds to Level 2. When both conditions are satisfied, this orientation is selected. This level is preferred because it ensures consistent reference alignment and reduces trajectory complexity.

Level 2: Detected-pose evaluation.

If Level 1 fails, the raw tracked pose obtained from the object manager is evaluated directly. This level uses the detected workpiece position and orientation as the pickup candidate, but only if the same pickup-side validation and zone-based reachability checks are satisfied. This accounts for cases where the initial sheet placement is already feasible and avoids unnecessary transformations. If the raw RZ lies within the admissibility window and the pickup orientation remains valid, the detected pose is selected. Otherwise, the strategy proceeds to the fallback level.

Level 3: Inverted-pose fallback.

If neither Level 1 nor Level 2 satisfies the admissibility condition, the strategy generates an inverted-pose fallback. This fallback is required because some workpiece placements cannot be reached using either the preferred reference-face orientation or the raw detected orientation, even though an opposite approach direction may still provide a possible pickup candidate. In this context, the inverted pose does not change the sheet position. It changes only the tool orientation so that the gripper is evaluated from an opposite orientation candidate, as described by Equation 5.5:

$$RZ' = \mathsf{wrap}(RZ \pm 180^\circ), \quad RY' = -RY, \quad RX' = -RX \tag{5.5}$$

In this fallback transformation, the in-plane angle RZ is rotated by 180° to test the opposite approach direction, while RY and RX are mirrored to form the corresponding tool orientation. The function $\mathrm{wrap}(\cdot)$ keeps the resulting in-plane angle within the configured angular range. This expands the set of possible pickup orientations when the primary checks fail, but it is not treated as equivalent to a fully validated Level 1 or Level 2 result. Since this fallback is selected only after the confirmed alternatives have failed, it is marked as *admissibility not confirmed*. This status is propagated to downstream planning modules so that the pose can be handled conservatively rather than assumed to be a fully confirmed orientation.

The implemented decision sequence follows the three levels described above. The system first corrects an invalid pickup side when necessary, then evaluates the reference-face equivalent orientation. If this fails, the detected pose is checked directly. Only when both confirmed alternatives fail is the inverted fallback selected and marked as *admissibility not confirmed*.

Listing 5.2 provides the corresponding implementation-level control flow. It shows how the detected sheet pose is converted into a selected orientation result before the grip planner is called.

```
1 Read tracked sheet pose from drag&bot object manager
2 Classify sheet centroid into the configured workspace zone
3 Correct pickup side if the gripper would approach an invalid face
4
```

```
Generate Level 1 reference-face equivalent orientation
if Level 1 satisfies zone and pickup admissibility then
    select Level 1 orientation as confirmed result
else
    Test Level 2 using the detected sheet orientation directly
    if Level 2 satisfies zone and pickup admissibility then
        select Level 2 orientation as confirmed result
    else
        Generate Level 3 inverted fallback orientation
        select fallback orientation with conservative status flag
    end if
end if

Forward selected orientation and status to grip planning
```

Listing 5.2: Pseudocode for three-level orientation selection.

Strategy outcome.

The proposed strategy is designed to determine a valid pickup orientation where the defined constraints allow it while prioritising kinematic feasibility and geometric consistency. Level 1 represents the preferred direct pickup orientation for the subsequent bending flow. If this orientation is not available, the detected pose is evaluated in Level 2 to determine whether the workpiece can still be picked directly from its current placement.

If neither the reference-face equivalent orientation nor the detected pose satisfies the validation conditions, the fallback orientation from Level 3 is treated as a handling candidate rather than a fully confirmed bending pose. In this case, the workpiece can be transferred to the reorientation table, where it is placed in a more favourable pose before the bending sequence continues.

By integrating admissibility constraints with this structured fallback mechanism, the method directly supports pre-execution validation and reduces the likelihood of infeasible motion generation. The selected orientation is then passed to the grip planning stage, where feasible suction-cup positions are calculated for the validated tool approach.

5.5.3 Grip Position Planning and Feasibility Validation

After a valid pickup orientation has been selected, the system must determine where the vacuum gripper can safely contact the sheet. The input to this stage is the validated pickup orientation, the workpiece geometry, and the configured two-cup gripper parameters. The output is a feasible grip configuration together with the approach, attachment, and retreat poses required for subsequent handling. This step is required because a geometrically reachable pickup orientation alone does

not guarantee that the suction cups can be placed on stable, continuous, and load-bearing regions of the sheet.

Workpiece geometry and feasible contact regions.

The workpiece is represented as a planar sheet with length, width, thickness, and mass obtained from the configuration. If the mass is not specified directly, it is estimated from the sheet volume and the configured material density. These parameters define the physical basis for grip planning, since both the available contact area and the required holding force depend on the sheet dimensions and mass.

The outer sheet boundary defines the maximum possible gripping area. Internal features, such as holes extracted from the STL model, are excluded from this area because a suction cup cannot form a reliable seal over a discontinuity. Each detected hole is represented by its centre position and radius in the sheet coordinate frame. A buffer is added around these regions so that selected cup positions maintain a safe distance from feature edges. The result is a set of feasible contact regions that can be used for candidate generation.

Grip candidate generation and geometric filtering.

Candidate suction-cup positions are generated using a grid-based search over the feasible sheet surface. The search begins with a coarse discretisation to identify usable regions and is then refined around promising areas to improve placement accuracy. Additional candidate positions are also evaluated near hole boundaries so that the available surface area can be used effectively without violating clearance requirements.

Each candidate is checked against geometric constraints before physical force validation is performed. The cup centre must remain inside the sheet boundary, satisfy the minimum edge clearance, avoid the buffered hole regions, and comply with the spacing requirements of the two-cup gripper. For the two-cup configuration, candidates are evaluated as pairs so that the distance between the suction cups provides a stable grip span across the workpiece.

After geometric filtering, only physically possible cup placements remain. These candidates are then passed to the vacuum holding force validation stage. This ordering ensures that the force model is applied only to candidate configurations that are already valid from a geometric point of view.

Vacuum holding force validation and final selection.

For each remaining grip candidate, the system evaluates whether the available vacuum force is sufficient to hold the sheet during handling. The contact area of one suction cup is calculated from the cup diameter using the standard circle-area relation according to Equation 5.6:

$$A_{\text{cup}} = \pi \left(\frac{d_{\text{cup}}}{2} \right)^2 \tag{5.6}$$

Here, d_{cup} is the suction cup diameter, and A_{cup} is the effective contact area of one cup. In this thesis, vacuum feasibility is evaluated using an adapted conservative model based on standard vacuum-gripper force principles [34]. The model estimates the available holding force from the pressure difference and suction cup area, and compares it against a safety-factor-weighted resultant load that combines gravitational and inertial effects [17]. The total available holding force generated by n suction cups is expressed in Equation 5.7, following the standard vacuum-force relation reported for vacuum grippers [17]:

$$F_{\text{vacuum}} = n \cdot \Delta P \cdot A_{\text{cup}} \cdot \eta \tag{5.7}$$

In this expression, n is the number of active suction cups, ΔP is the pressure difference, and η represents efficiency losses due to leakage, surface condition, and imperfect sealing. The required holding force is calculated using the adapted vacuum-handling load formulation in Equation 5.8 [34]:

$$F_{\text{required}} = SF \cdot m \cdot \sqrt{a_{xy}^2 + (g + a_z)^2} \tag{5.8}$$

Here, SF is the safety factor, m is the sheet mass, g is gravitational acceleration, and a_{xy} and a_z represent the expected horizontal and vertical acceleration components during handling. A grip candidate is accepted only if the available holding force is greater than or equal to the required holding force:

$$F_{\text{vacuum}} \geq F_{\text{required}} \tag{5.9}$$

Candidates satisfying both the geometric constraints and the holding force criterion are ranked using a scoring function. The score favours positions with sufficient edge clearance, distance from internal features, and stable spacing between the two suction cups. The highest-scoring configuration is selected as the final grip position.

Based on this selected configuration, the system derives three key pickup poses: the approach pose, the attachment pose, and the retreat pose. These poses define how the gripper moves towards the sheet, establishes contact, and leaves the pickup area after attachment. The complete grip planning workflow is summarised in Listing 5.3 and illustrated in Figure 5.9.

```
1 Load sheet geometry and grip-planning parameters
2 Generate candidate grip positions using coarse-to-refined grid-based search
```

```
Add supplemental candidates near hole boundaries while respecting
    clearance buffers
Exclude candidates violating edge-clearance constraints
Exclude candidates overlapping with hole regions and buffer zones

if no feasible candidates remain then
    terminate grip planning
end if

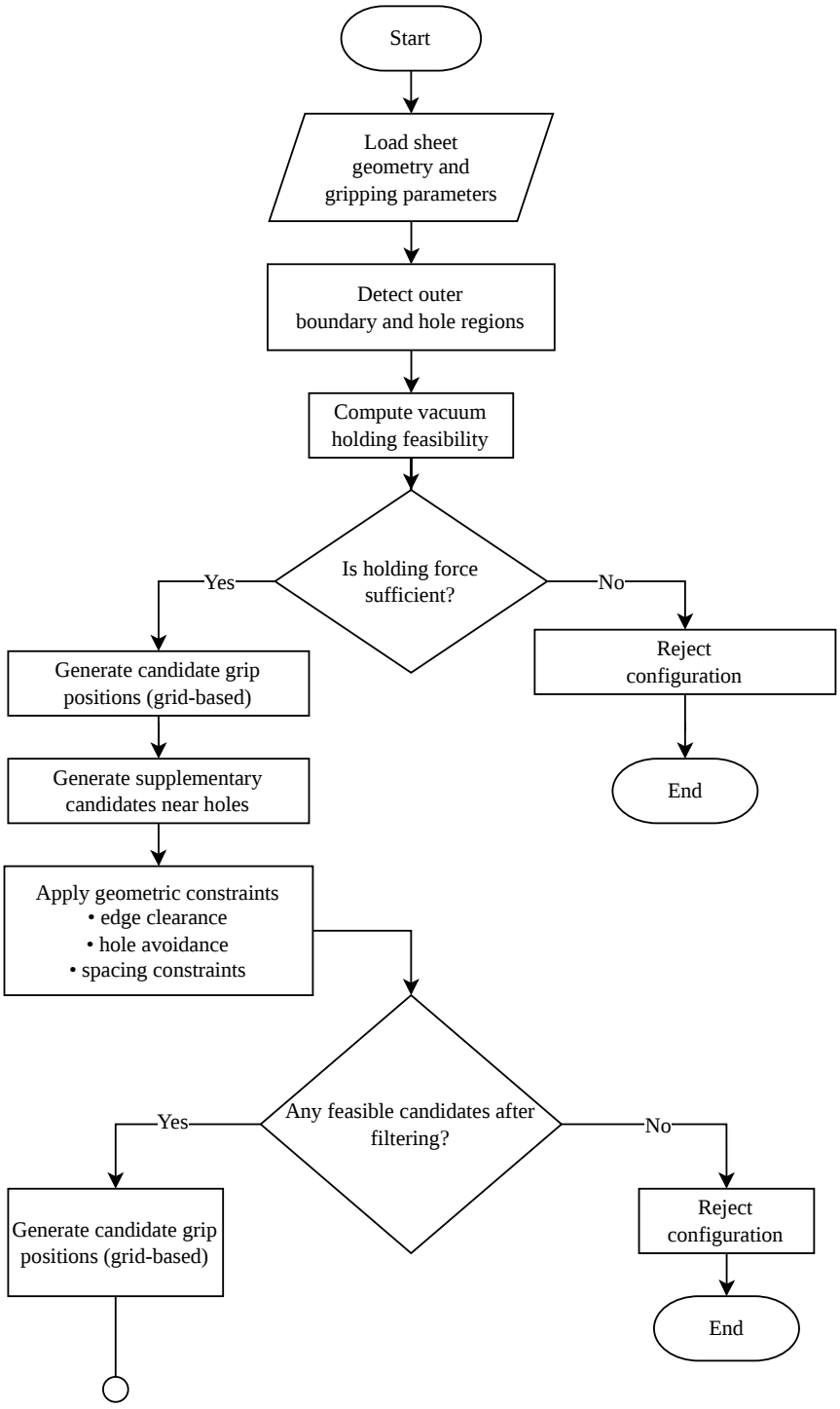
if two-cup configuration is required then
    form candidate pairs
    exclude pairs violating minimum spacing constraints
end if

Validate feasible configuration(s) using the vacuum holding force criterion
Score remaining configuration(s) based on geometric suitability
Select the highest-scoring grip configuration
Compute approach, attachment, and retreat waypoints (A1, B1, R1)
Forward selected grip result to trajectory generation
```

Listing 5.3: Pseudocode for grip candidate search and selection.

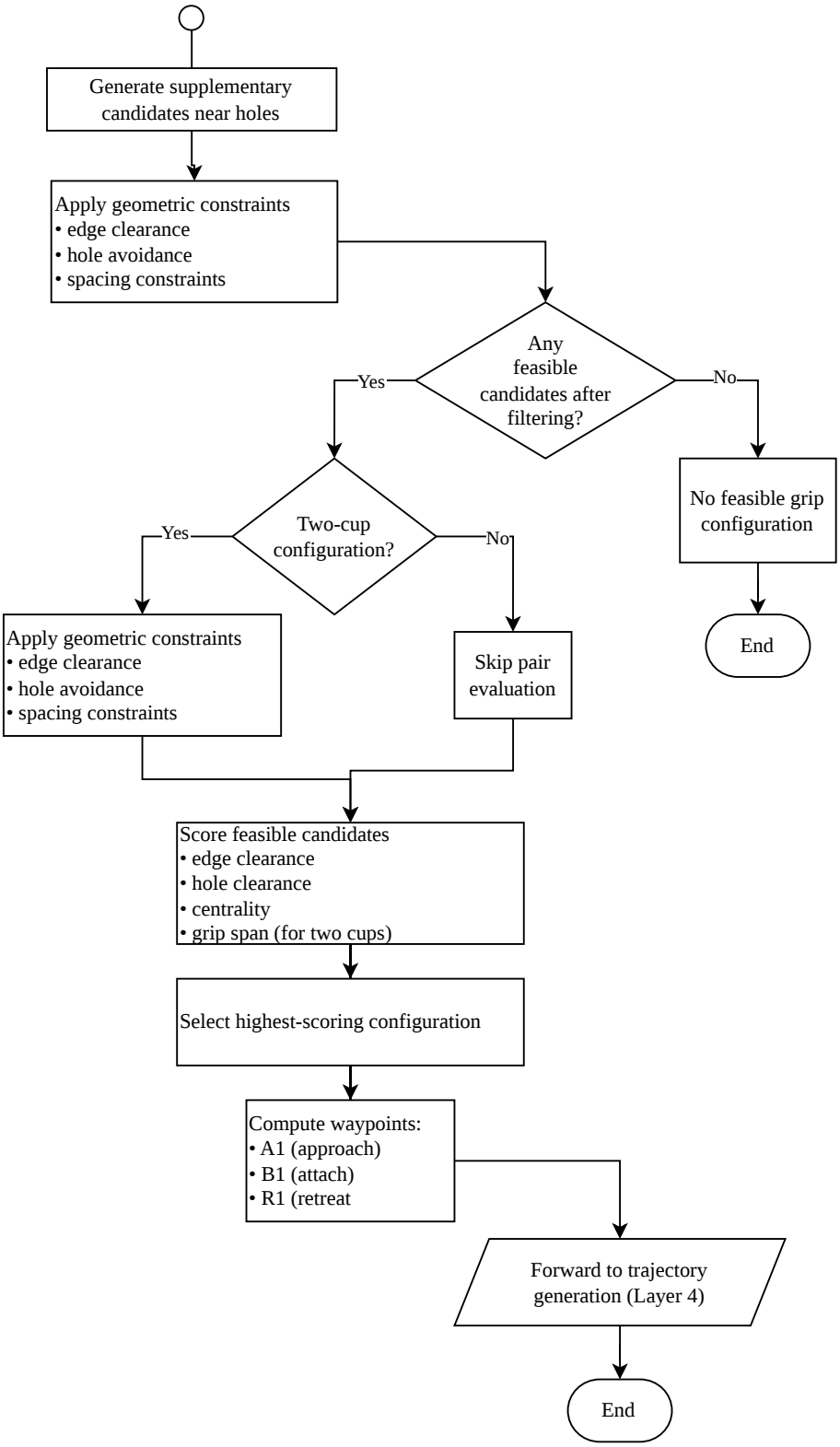
Figure 5.9 illustrates the complete grip planning pipeline, including force validation, candidate generation, constraint filtering, scoring, and waypoint derivation. The workflow is split into Part a and Part b for readability.

The selected grip configuration and derived pickup poses define where the gripper must contact the sheet. The following section describes how these planned grasp and release actions are represented during execution.



(a) Input validation, force validation, and candidate generation.

Figure 5.9: Grip position planning and feasibility validation workflow.



(b) Constraint filtering, scoring, final grip selection, and waypoint derivation.

Figure 5.9: Grip position planning and feasibility validation workflow.

5.6 Gripper State, Regrip, and Opposite-Side Handling

After grip planning has selected a feasible suction-cup configuration, the system must convert the planned grasp into a consistent execution state. This requires two actions to remain synchronised: the physical vacuum state of the gripper and the virtual attachment state of the workpiece in the simulation. The input to this stage is the selected grip configuration and the required grasp or release command. The output is a synchronised gripper and workpiece state that can be used by the regrip, flip, and trajectory execution stages.

This separation is necessary because vacuum actuation alone only represents the physical gripping command, while object attachment defines how the sheet follows the robot tool frame in the simulation. If these two states are inconsistent, the simulated workpiece pose may no longer match the intended robot motion, which can lead to incorrect pose propagation and invalid trajectory generation.

Vacuum gripper digital input and output sequence.

The vacuum gripper is controlled through digital output signals in the drag&bot IO system. One output activates the vacuum, while a second output releases it. These signals are treated as mutually exclusive so that the system does not command vacuum activation and release at the same time.

Control commands are issued via the `/robot/set_digital_ios` interface, which transmits structured IO instructions specifying the target output and its Boolean state. During a grasp operation, the vacuum is activated before the sheet is attached in the simulation. During release, the vacuum is switched off before the sheet is detached from the robot tool frame. Short guard delays are applied between these actions in the virtual cell to allow state propagation through the runtime interface; for physical deployment, these delays should be tuned to the measured gripper response or replaced by explicit vacuum-state feedback.

Kinematic object attachment and detachment.

In the simulation environment, the physical coupling between the gripper and the sheet is represented by kinematic object attachment. When the grasp is confirmed, the sheet object is attached to the robot tool frame so that subsequent robot motion updates the sheet pose consistently. When the gripper releases the sheet, the object is detached and remains at its last known pose.

Attachment is performed using the `/dnb_object_manager/attach` interface without forcing a position override during normal execution. This avoids artificial jumps between the tracked object pose and the simulated state, since the detected sheet pose may not exactly coincide with the instantaneous tool frame pose. Position override is reserved for controlled setup or initialisation cases where explicit pose alignment is required. Detachment is executed using the `/dnb_object_manager/detach`

interface within the same reference frame.

The grasp and release procedures therefore follow fixed execution sequences:

1. Grasp sequence: vacuum ON $\rightarrow$ dwell $\rightarrow$ attach object
2. Release sequence: vacuum OFF $\rightarrow$ dwell $\rightarrow$ detach object

These sequences are centralised so that all modules use the same grasp and release behaviour. This prevents inconsistent conditions where the vacuum state and attachment state do not correspond. With this synchronised handling mechanism defined, the system can safely proceed to regrip planning, opposite-side bending, or trajectory execution.

5.6.1 Regrip Planning and Execution

Even when a valid pickup orientation and a feasible grip configuration are established, the current grasp may not permit the robot to reach the required bending pose. The input to this stage is the current workpiece pose, selected grip configuration, and target bending pose. The output is either confirmation that direct execution is feasible or a regrip sequence that moves the workpiece into a reachable configuration. This limitation arises when the target configuration lies outside the reachable workspace or when kinematic constraints, such as joint limits or proximity to singularities, prevent a valid motion solution [33]. To address this implementation limitation within the evaluated virtual cell, the system implements an automatic regrip planning strategy that determines the required reorientation dynamically instead of relying only on predefined handling sequences.

Regrip decision strategy.

In contrast to conventional approaches based on fixed regrip sequences, the proposed system evaluates a discrete set of candidate workpiece orientations and selects a configuration that satisfies kinematic feasibility. This enables the system to determine both the required orientation and the number of regrip steps automatically.

The candidate set consists of discrete orientation increments corresponding to physically achievable rotations on the orientation table. These candidate orientations, together with the relevant station poses for deposit, re-grasp, retract, and approach motions, are defined through configuration parameters so that the procedure can be reproduced for the same virtual-cell layout. For each candidate orientation, the system performs the following sequence:

1. Update the workpiece pose according to the candidate orientation

2. Determine a valid pickup orientation using the strategy described in Section 5.5
3. Compute feasible grip positions using the planner described in Section 5.5.3
4. Evaluate kinematic feasibility using inverse kinematics and trajectory validation

A candidate orientation is accepted only when all feasibility conditions are satisfied. The evaluation follows a sequential selection strategy in which candidates are tested in a predefined order, and the first feasible solution is selected. This ensures computational efficiency while maintaining robustness for real-time execution.

The number of required regrip steps is implicitly determined from the difference between the current orientation and the selected feasible configuration. This approach avoids unnecessary intermediate operations while ensuring that the final pose remains reachable and executable within the robot workspace.

The implemented decision process therefore reuses the existing planning pipeline for each candidate orientation. This ensures that regrip selection remains consistent with orientation selection, grip planning, and trajectory generation instead of relying on a separate rule set.

Listing 5.4 summarises the regrip decision and insertion logic. The direct path is evaluated first; only if it fails does the system evaluate candidate regrip orientations and insert the corresponding handling sequence.

```
Evaluate direct trajectory from current grasp to next bending pose
if direct trajectory is feasible then
    continue execution without regrip
else
    for each configured regrip orientation candidate do
        update simulated workpiece pose for candidate orientation
        determine admissible pickup orientation
        compute feasible grip configuration
        validate trajectory from regrip pose to bending pose

        if all checks are feasible then
            select candidate orientation
            insert deposit, release, re-approach, grasp, and return actions
            continue bending sequence
        end if
    end for
end if

if no regrip candidate is feasible then
    trigger controlled stop and operator warning
end if
```

Listing 5.4: Pseudocode for regrip decision and insertion logic.

Regrip execution sequence.

Once a feasible orientation has been identified, the system executes a structured regrip sequence to realise the required reorientation. The sequence consists of the following steps:

1. Transport the workpiece from the bending station approach position to the orientation table deposit position while maintaining the current grasp
2. Release the workpiece through controlled vacuum deactivation and kinematic detachment
3. Move the robot to a retract standoff position to ensure collision-free clearance
4. Re-approach the workpiece at the updated orientation and perform the grasp sequence
5. Repeat the regrip cycle, if further orientation adjustment is required, until a grasp configuration is found from which the gripper can attach to the workpiece and the robot can move directly to the target bending pose
6. Perform the final grasp and return the workpiece to the bending station

Each motion segment is executed using smooth trajectories to minimise dynamic loads on the gripper and ensure stable handling. Intermediate standoff positions provide safe transitions between grasping and releasing operations.

System integration and configuration.

The regrip execution sequence operates within the existing motion framework and uses configuration-defined target positions, including deposit, re-grasp, retract, and approach poses. These parameters define the physical layout of the cell but do not determine the number of regrip steps.

The decision to trigger a regrip is handled by the planning orchestrator, which evaluates feasibility based on orientation admissibility and kinematic constraints. This separation between decision-making and execution ensures modularity and allows the regrip strategy to be extended without modifying the motion execution layer.

Compared to purely configuration-driven approaches, the proposed method improves adaptability by determining the required number of regrip steps dynamically. At the same time, it maintains computational efficiency by evaluating a structured set of candidate orientations rather than performing global optimisation. This balance supports predictable execution within the implemented workflow while keeping the decision process transparent and reproducible.

5.6.2 Opposite-Side Bending Workflow

After completing all primary-side bending operations, the system evaluates whether additional bends are required on the opposite side of the sheet. The input to this stage is the completed primary-side state and the remaining bend-side requirements. The output is a verified flip sequence that makes the opposite face accessible for the same downstream planning pipeline. These operations cannot be executed directly from the initial pickup configuration, as the required side is not accessible to the press brake tool. Consequently, the workpiece must be reoriented through a controlled flip operation before continuing the bending process.

Pre-flip verification and transition correction.

Before initiating the flip sequence, the system verifies that the current sheet pose corresponds to the expected reference configuration. This verification is performed using a wrap-safe six-degree-of-freedom pose comparison, with a positional tolerance of 2 mm and an angular tolerance of 1°. These values are configured verification thresholds used in the evaluated virtual cell; they define the accepted deviation before the system triggers a corrective transition rather than manufacturer-specified hardware limits.

This validation step is required because the sheet may have undergone regrip operations during the primary bending sequence. Any deviation from the expected pose would propagate errors into subsequent motion planning stages, leading to incorrect trajectory generation.

If the verification fails, a transition correction step is executed. The system first restores the reference orientation using the strategy described in Section 5.5. A controlled pickup and repositioning sequence is then performed to re-establish a consistent and verified alignment between the robot and the workpiece.

If the verification is successful, the correction step is omitted and the system proceeds directly to the flip sequence.

Sheet flip sequence and opposite-side bending execution.

The flip operation is performed using a dedicated station within the robotic cell. The workpiece is transferred through two predefined reference positions to achieve a physically feasible reorientation.

First, the robot transports the sheet to the flip station placement position while maintaining the current grasp. The workpiece is then released in a controlled manner. The robot subsequently moves to an intermediate standoff pose and re-approaches the sheet from the opposite side, establishing a new grasp that results in the flipped orientation.

This two-stage transfer is preferred over a direct in-gripper rotation. A direct rota-

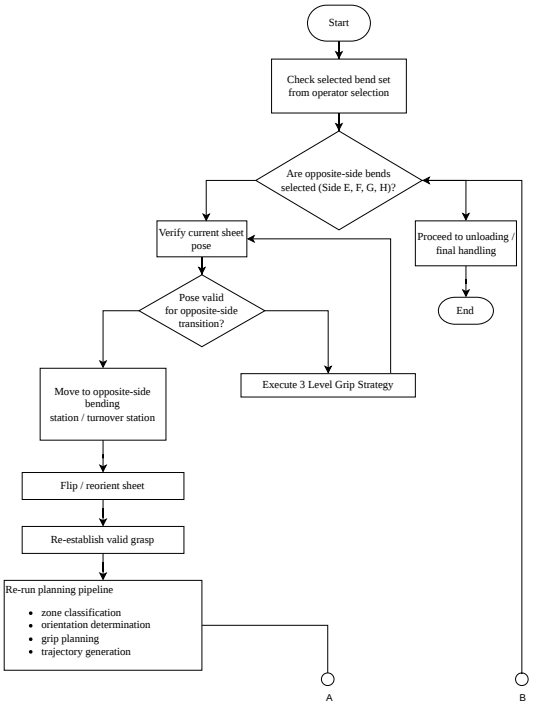
tion would require a continuous motion that exceeds the kinematic capabilities of the robot and may violate workspace or collision constraints [33].

Following the flip operation, the sheet is aligned such that the previously inaccessible side is oriented towards the press brake tool. The system then applies the same validation pipeline used for primary-side bending, including zone classification, orientation determination, grip position planning, and trajectory generation.

Reusing the same processing pipeline ensures consistent system behaviour across both primary and opposite-side operations. It also avoids the need for separate handling logic, thereby maintaining modularity and reducing system complexity.

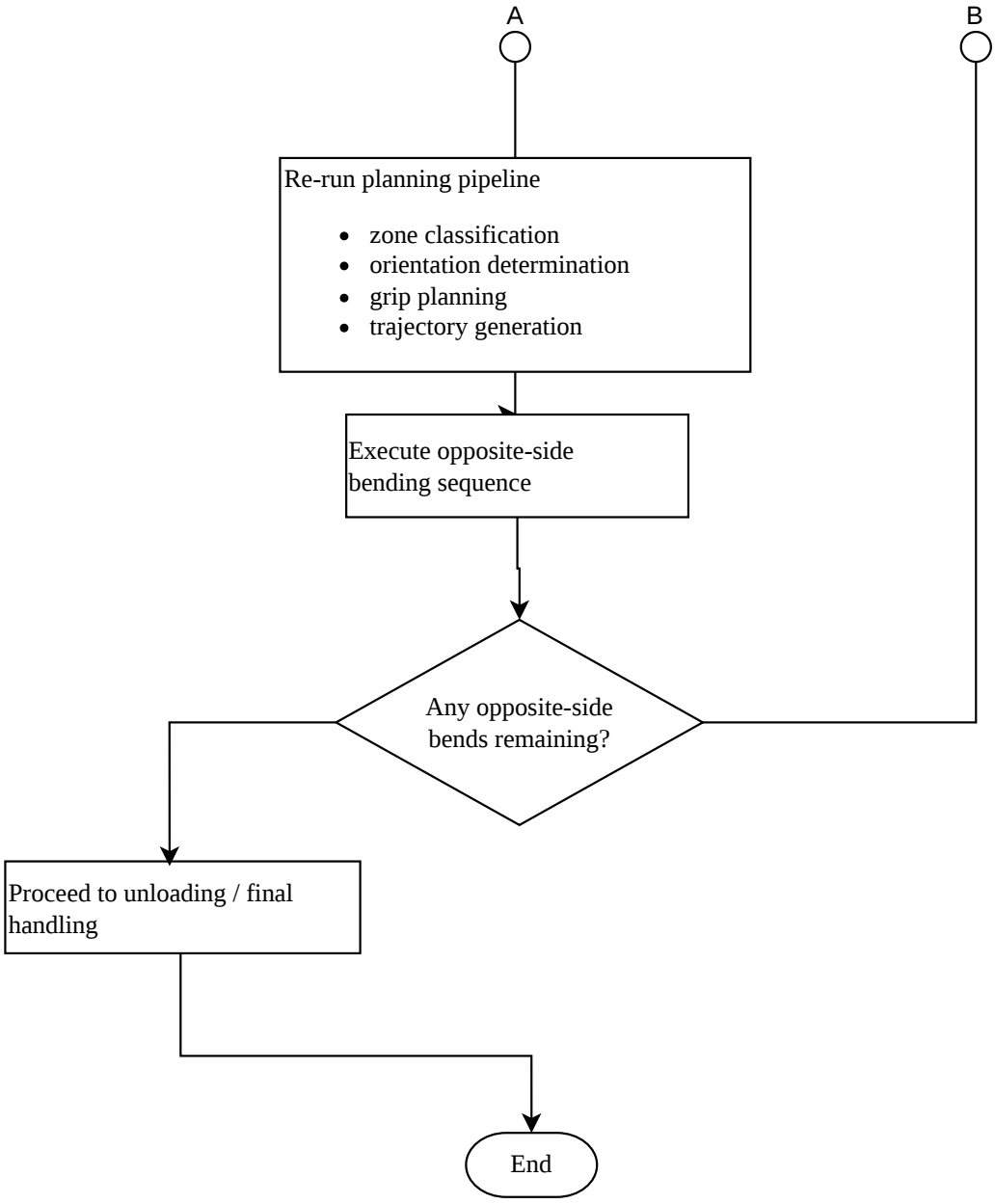
Figure 5.10 illustrates the complete opposite-side bending workflow, including pre-flip verification, optional transition correction, flip execution, and subsequent bending operations. The workflow is split into Part a and Part b for readability.

After completing all opposite-side bends, any remaining staged operations are executed using the existing handling framework. The system then proceeds to the final unloading sequence.



(a) Pre-flip verification, transition correction, and flip station approach.

Figure 5.10: Opposite-side flip workflow enabling access to previously unreachable bending surfaces.



(b) Opposite-side reorientation, validation, bending continuation, and unloading transition.

Figure 5.10: Opposite-side flip workflow enabling access to previously unreachable bending surfaces (continued).

5.7 Trajectory Generation and Execution

Trajectory generation constitutes the final stage of the planning pipeline, where validated task-level poses are transformed into executable robot motion. The input to this stage is an ordered sequence of TCP poses obtained from upstream modules, including grip planning, gripper control actions, regrip handling, and opposite-side operations. The output is a reduced sequence of executable robot motion commands transmitted to the drag&bot runtime. Each pose defines the desired end-effector position and orientation within the robot world frame.

Rather than executing direct point-to-point transitions, the system constructs a continuous and smooth trajectory composed of intermediate waypoints. This approach ensures kinematic feasibility, reduces dynamic loading, and improves execution stability during sheet handling.

5.7.1 Task Sequence Assembly for Execution

Before individual trajectories are generated, the system assembles an ordered task sequence from the validated data produced by the previous modules. The required inputs include the configuration parameters, the current robot pose, the tracked sheet pose, the selected bend side, the validated pickup orientation, the selected grip configuration, the regrip or flip decisions, and the target bending pose. These data define both where the robot must move and which handling actions must be executed between motion segments.

The generated execution program therefore contains motion actions and non-motion actions. Motion actions include approach, attachment, transport, reorientation, bending approach, retreat, and unloading poses. Non-motion actions include vacuum activation and release, as well as kinematic attachment and detachment of the workpiece in the simulation. If direct execution is not feasible, the corresponding regrip cycle is inserted into the sequence. If opposite-side bending is required, the flip sequence is inserted before the remaining bending operations continue through the same planning pipeline.

Each motion action is represented as a start-pose to target-pose segment. The start pose may be the current robot pose, the previous waypoint, a sheet attachment pose, or a pose reached after a regrip or flip operation. The target pose may be a gripping pose, an orientation-table pose, a flip-station pose, a bending pose, or an unloading pose. Between each start and target pose, the trajectory generator automatically inserts intermediate Cartesian waypoints according to the selected path structure and configured motion resolution. These intermediate poses are not manually programmed; they are derived from the validated task poses so that transitions such as robot start to sheet grip, sheet grip to reorientation table, and

reorientation table to bending pose can be executed smoothly.

The assembly procedure combines the validated pose data into one ordered action list. Regrip or flip actions are inserted only when required by the feasibility checks. For each motion action, the trajectory generator creates the lift, transfer, and descent structure, interpolates intermediate waypoints, validates feasibility, reduces redundant waypoints, and then passes the resulting motion and handling commands to the drag&bot runtime.

The waypoint-generation and path-reduction procedure is summarised in Listing 5.5. The listing shows how each motion segment is converted into a dense interpolated trajectory and then reduced before transmission to the robot runtime.

```
for each motion segment in the assembled task sequence do
    define lift pose above start pose using configured safe height
    define transfer pose above target pose
    define descent pose at target task pose

    for each phase between consecutive poses do
        compute path length and required sample count
        interpolate Cartesian position linearly
        interpolate orientation using SLERP
        apply minimum-jerk scaling to interpolation parameter
        append generated waypoint to dense trajectory
    end for

    validate dense trajectory against workspace limits
    remove waypoints below position and rotation change thresholds
    transmit reduced waypoint sequence as linear motion commands
end for
```

Listing 5.5: Pseudocode for waypoint generation and path reduction.

5.7.2 Trajectory Structure and Interpolation

Each start-pose to target-pose segment from the assembled task sequence is generated using a structured three-phase trajectory: lift, lateral transfer, and descent.

In the lift phase, the TCP is translated vertically to a predefined safe height to avoid collisions with the workpiece, fixtures, or surrounding environment. In the lateral transfer phase, the robot moves towards the target position while maintaining this clearance height. During this phase, the end-effector orientation is constrained to limit excessive joint rotations and reduce proximity to kinematic singularities. Finally, in the descent phase, the TCP is lowered to the target pose, where the required orientation for task execution is fully applied.

To further improve motion feasibility, an intermediate rotation pose is introduced

above the target location. This ensures that major orientation adjustments are completed before the final approach, reducing the risk of instability near contact regions.

Waypoint generation and interpolation.

The trajectory between successive poses is generated through interpolation in Cartesian space. For a start pose $\mathbf{T}_s$ and a target pose $\mathbf{T}_t$, the translational component is computed using standard linear interpolation according to Equation 5.10 [35]:

$$\mathbf{p}(\lambda) = (1 - \lambda)\mathbf{p}_s + \lambda\mathbf{p}_t, \tag{5.10}$$

where $\lambda \in [0, 1]$ is the interpolation parameter.

Rotational interpolation is performed using spherical linear interpolation (SLERP) in quaternion space, which provides smooth and constant-speed rotation between orientations [36].

To ensure smooth velocity and acceleration profiles, the interpolation parameter follows the standard fifth-order minimum-jerk trajectory in Equation 5.11 [37], [38]:

$$s(u) = 10u^3 - 15u^4 + 6u^5, \tag{5.11}$$

which guarantees zero velocity and acceleration at the start and end of each motion segment [37]. Such profiles are widely adopted in robotic trajectory planning to achieve smooth and physically consistent motion behaviour [18].

By sampling this progression at discrete intervals, the implementation generates intermediate TCP waypoints that approximate the continuous trajectory. The same scalar progression is applied to the Cartesian interpolation and to the quaternion-based orientation interpolation so that position and orientation evolve consistently along the segment.

The number of waypoints is determined from the Cartesian path length and a predefined spatial resolution, with a lower bound of two samples. This keeps the trajectory smooth without introducing an unnecessarily dense command sequence.

5.7.3 Execution Validation and Path Reduction

Each interpolated TCP waypoint is mapped to a corresponding robot joint configuration using inverse kinematics. For each waypoint, a feasible solution is selected based on continuity with the previous configuration, avoiding abrupt joint changes and ensuring smooth motion across the trajectory.

As the robot executes the sequence, all joints evolve continuously, resulting in a smooth end-effector motion that closely follows the planned Cartesian path.

Workspace constraints and singularity handling.

To ensure execution feasibility, each waypoint is validated against workspace constraints. The TCP pose is transformed into the corresponding flange pose, and its position is checked against the allowable workspace limits. If a waypoint exceeds these limits, it is projected onto the nearest feasible boundary.

Singularity avoidance is addressed by restricting orientation changes during the lateral motion phase. Large orientation adjustments are deferred to the approach phase, where the robot operates in a more stable configuration. This reduces the risk of excessive joint velocities and improves overall motion robustness.

Motion execution and path reduction.

The interpolation process generates a dense set of waypoints, which may exceed the processing capacity of the robot controller. To address this, a path reduction strategy is applied, retaining only those waypoints that exceed predefined positional or rotational thresholds. This preserves the essential characteristics of the trajectory while reducing communication overhead.

The resulting waypoint sequence is transmitted as a series of linear motion commands, which are executed by the robot controller using blending techniques. This allows smooth transitions between consecutive waypoints without stopping, producing continuous and stable motion.

Overall, the trajectory generation module forms a deterministic transformation pipeline in which validated task-level poses are converted into executable robot motion through interpolation, kinematic mapping, constraint validation, and controlled execution.

The implemented functions are evaluated later through the scenario structure in Chapter 6. Table 5.4 summarises this connection so that the implementation chapter closes with a clear link between the realised modules, their requirements, and the evidence used in the evaluation.

Implemented Function	Related Requirement	Chapter 6 Evaluation Scenario	Evidence Type
Automatic program generation from geometry and configuration	FR-00, NFR-01	Manual-versus-automated workflow comparison	Preparation time and execution-ready workflow state
Bend-side selection and workflow branching	FR-02	Side A direct bending and Side E opposite-side bending	Scenario-specific generated workflow behaviour
Orientation validation and correction	FR-04	Arbitrary initial orientation and position-change scenarios	Completion classification and planning time
Constraint-based grip planning	FR-06	Nominal, geometry-variation, and regrip-required scenarios	Successful execution or rejected infeasible grips
Automatic regrip planning and execution	FR-03	Regrip-required and Side E scenarios	Regrip count, execution time, and success rate
Opposite-side flip and reorientation workflow	FR-05	Side E opposite-side workflow	Success/failure rate and controlled-stop interpretation
Safety validation and controlled-stop handling	FR-07, NFR-03, NFR-05	Failure-prone and constrained-placement scenarios	Controlled-stop behaviour and warning response
Configuration-driven adaptation and deterministic execution	NFR-02, NFR-04, SR-01–SR-03	Geometry variation, position variation, and repeated automated runs	Scenario averages, repeated-run classification, and virtual runtime execution

Table 5.4: Mapping between implemented functions and the Chapter 6 evaluation evidence.

5.8 Implementation Summary

This chapter presented the implementation of the selected Concept B as a modular planning and execution pipeline for automated press brake sheet metal bending. The system integrates configuration handling, runtime communication, operator input, safety validation, orientation determination, grip planning, regrip and opposite-side handling, and trajectory generation within the drag&bot-based virtual cell.

A layered architecture was used to keep the implementation testable and maintainable. Each functional component has a defined responsibility, while validation is performed before motion dispatch so that invalid poses, weak grip configurations, and unsafe workspace conditions are detected before execution. The resulting workflow can generate robot programs automatically from configuration data and geometric inputs, reducing the need for manual waypoint definition.

Reliability and failure mode analysis. After the functional pipeline has been

implemented, the remaining question is how the system behaves when individual assumptions or process steps fail. A simplified FMEA-style scoring convention is therefore used to compare potential failure modes within the robotic bending workflow, evaluate their impact on system performance, and assess the effectiveness of implemented mitigation strategies [39].

The assessment considers three qualitative criteria: severity (S), occurrence (O), and detectability (D). These factors are combined into a Risk Priority Number (RPN) according to the thesis scoring convention defined in Equation 5.12 [39]:

$$\text{RPN} = S \times O \times D \tag{5.12}$$

Each parameter is assessed using an internal qualitative scale from 1 (low) to 10 (high). Higher RPN values indicate failure modes that require higher-priority mitigation within the scope of this thesis.

Identified failure modes and mitigations.

The analysis focuses on critical stages of the execution pipeline, including pickup orientation, grip planning, trajectory execution, regrip handling, and opposite-side operations. The identified failure modes are summarised in Table 5.5.

Failure Mode	Effect	S	O	D	RPN	Cause	Mitigation
Unreachable pickup pose	Pickup aborted	7	3	3	63	Pose outside valid zone	Zone and orientation checks block motion; operator is notified
Grip over hole	Vacuum loss risk	8	2	2	32	Hole or feature error	Hole-clearance and suction-area checks reject invalid grips
Insufficient holding force	Sheet slip or drop	8	2	1	16	Load or acceleration too high	Force validation with safety factor rejects weak grips
Invalid motion command	Execution interrupted	6	3	2	36	Command format error	Message formatting and command validation reduce interface errors
Workspace limit exceeded	Motion rejected	6	4	3	72	Pose beyond reach limits	Workspace limits constrain radial and vertical motion
Side E regrip or flip infeasible	Opposite-side workflow stops	7	2	3	42	Post-flip pose infeasible	Pre-flip checks and controlled stops prevent unsafe continuation
Inverse-kinematics failure	No valid robot pose	5	2	3	30	Attachment state conflict	Attach-detach discipline limits solver overload
Collision not detected in simulation	Unsafe path may remain	6	3	5	90	Limited collision model	Three-phase path reduces risk; collision-aware validation remains future work
Invalid configuration	Startup abort	3	3	1	9	Missing or invalid parameters	Startup validation blocks execution before motion begins
Physical effects not modelled	Production readiness not proven	6	3	5	90	Springback, slip, force, and calibration effects	Simulation scope is stated; physical validation remains future work

Table 5.5: Failure Mode and Effects Analysis (FMEA) of the robotic bending system.

The selected failure modes reflect the most critical dependencies in the system, particularly geometric accuracy, kinematic feasibility, grip reliability, and physical interaction between the robot and the workpiece. Mitigation is handled through fixture pose checks, orientation admissibility limits, grip filtering, vacuum force validation, workspace validation, waypoint reduction, and structured regrip and flip sequences.

The FMEA results indicate that the most critical risks are associated with pose accuracy, trajectory feasibility, incomplete collision modelling, and the transfer from simulation to physical bending. Pose and feasibility risks are addressed within the implemented system through layered validation, constraint-based planning, and structured motion generation. In contrast, collision-aware deployment and physical bending effects remain documented limitations rather than fully resolved implementation features.

Overall, the system demonstrates a controlled reliability profile within the evaluated simulation conditions. High-severity failure modes are either detected before continuation, converted into controlled stops, or identified as limitations for future physical validation. This indicates that the proposed architecture supports predictable simulated execution while still requiring additional validation before industrial commissioning.

A key contribution of this work lies in the integration of adaptive regrip planning and opposite-side handling within a deterministic execution pipeline. Unlike conventional approaches that rely on predefined sequences, the system dynamically evaluates feasible configurations and determines the required manipulation strategy at runtime. This improves flexibility while maintaining computational efficiency suitable for industrial applications.

The trajectory generation framework further enhances system robustness by combining smooth interpolation techniques with kinematic validation and path reduction strategies. This supports the generation of motions that are executable by the robot controller and stable under the evaluated simulation conditions.

The detailed requirement compliance matrix is provided in Appendix A.4, Table 4. It links the functional, non-functional, and software platform requirements to the corresponding implementation components and validation mechanisms. Overall, the implemented system achieves a balance between flexibility, safety, and computational efficiency, demonstrating the feasibility of automated robot program generation within the defined simulation-based scope.

6 Evaluation and Validation

6.1 Introduction to Evaluation

This chapter evaluates whether the implemented system satisfies the central hypothesis of this thesis: automated robot-program generation from workpiece geometry should reduce preparation effort while preserving feasible execution in the simulated press-brake cell. The evaluation therefore does not focus only on whether a motion sequence can be generated. It also examines how much effort is saved, how the system behaves when the workpiece geometry or position changes, how regripping and opposite-side bending affect process time, and how the system reacts when a scenario cannot be completed safely.

The evaluation is restricted to the simulation-based scope defined in Chapter 1. Consequently, the chapter validates robot-program generation, simulated execution feasibility, planning behaviour, regrip handling, and safe stopping behaviour. It does not claim validation of physical bending force, material springback, final bend-angle accuracy, gripper slip on real sheet metal, or collision-free operation in a commissioned industrial cell. This distinction is important because the implemented system can be judged rigorously within its intended scope without overstating its readiness for physical production.

The analysis is organised around two complementary evaluation layers. First, manual and automated workflows are compared using repeated workflow trials in order to quantify preparation time, execution time, and total process time. Second, automated feasibility trials are evaluated scenario by scenario to assess success rate, failure rate, planning time, and controlled-stop behaviour. Separating these two layers prevents the time-comparison results from being mixed with automated-only feasibility results and keeps the interpretation of each metric consistent.

6.2 Evaluation Methodology

6.2.1 Measurement Principles

All reported values are evaluated as trial-based measurements. Scenario-wise averages are interpreted first because the difficulty of the task changes with the selected bend side, workpiece geometry, workpiece position, and regrip requirement. Overall averages are then used only as compact summary indicators. This avoids

treating the complete trial set as if all cases had the same technical difficulty.

The arithmetic mean is used to obtain the representative value for each scenario. For a measured quantity x_i in trial i, the average value $\bar{x}$ is calculated as:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i. \tag{6.1}$$

In this equation, N denotes the number of trials in the evaluated scenario and x_i denotes the measured value in one trial. This equation is used for preparation time, execution time, total process time, planning time, and regrip count. It is suitable here because the objective is not to model a probability distribution, but to compare repeated workflow observations under defined scenario conditions.

Preparation time. Preparation time measures the effort required before a workflow is ready for simulated execution. In the manual workflow, measurement starts when the operator begins adapting the robot task through waypoint, function-block, or sequence changes. It ends when the manual workflow is execution-ready. In the automated workflow, measurement starts when the operator begins entering or updating the workpiece geometry, pose, and bend-side input. It ends when the system has generated an execution-ready workflow. The measurement includes input selection, configuration updates, manual waypoint editing where applicable, function-block adaptation where applicable, and preparation of regrip or flip logic where applicable. It excludes simulated robot motion, physical bending, post-processing, documentation, and figure preparation. The comparative evaluation uses 48 manual workflow trials and 48 automated workflow trials across direct bending, opposite-side bending, geometry variation, and position-change conditions.

Execution time. Execution time measures the simulated robot runtime after the workflow is ready. Measurement starts when the prepared or generated workflow begins execution and ends when the simulated sequence reaches its final state, completes unloading, or stops in a controlled way. It includes robot approach, gripping, transfer motion, bending-position motion, regrip motion, flip motion, retreat, and unloading where applicable. It excludes preparation, manual programming, operator input before execution, physical material deformation, and post-processing. Execution time is therefore interpreted as the simulated runtime of the prepared process, not as the full engineering effort.

Total process time. Total process time combines preparation and execution because a workflow that executes quickly but requires long manual preparation may still be inefficient in practice. For each trial, total process time is calculated as:

$$t_{\text{total}} = t_{\text{prep}} + t_{\text{exec}}. \tag{6.2}$$

Here, t_{total} is the total process time, t_{prep} is the preparation time, and t_{exec} is the simulated execution time. This equation is used to compare the complete effort from the start of task preparation to the end of simulated execution. It is the most important time metric in this chapter because the thesis objective is to reduce programming and preparation effort, not only robot movement time.

Planning time. Planning time is measured only for automated feasibility trials. It starts when the automated planning procedure is triggered after valid inputs are available and ends when the system has either produced a validated sequence or rejected the case. It includes orientation evaluation, grip selection, reachability decision-making, regrip decision-making, flip decision-making where required, and task-sequence generation. It excludes operator input, manual editing, robot motion, communication after dispatch, and physical bending. Ten repeated runs are evaluated for each automated feasibility scenario. Planning time is included as a secondary metric because it remains below one second in the evaluated scenarios; the practical workflow duration is dominated by preparation and execution time.

Success rate, failure rate, and controlled stop. Automated feasibility is evaluated using ten repeated runs per scenario. A trial is classified as successful when the intended scenario objective is completed without manual waypoint correction, unresolved feasibility violation, or uncontrolled interruption. In an opposite-side workflow, success also requires that the system inserts the required flip and regrip behaviour correctly.

A trial is classified as failed when the intended execution objective is not completed because of unresolved workspace, fixture, reachability, or feasibility constraints. The success rate R_{success} is calculated as:

$$R_{\text{success}} = \frac{N_{\text{success}}}{N_{\text{total}}} \cdot 100. \tag{6.3}$$

In this equation, N_{success} is the number of successful trials and N_{total} is the number of evaluated trials in the scenario. The failure rate is calculated in the same way using the number of failed trials.

A controlled stop is treated separately in the interpretation. It occurs when the system detects that safe continuation is not possible and prevents the workflow from proceeding. This is a system safety success because unsafe continuation is avoided, but it is also an execution failure because the intended process is not completed. This distinction is used in the requirement verification because safe stopping is evidence for validation logic, even when the process result itself is incomplete.

Regrip count. Regrip count measures how many intermediate regrip operations are required after the initial pickup. The first normal grip used to pick the sheet is not counted as a regrip. Measurement starts at workflow execution start and

ends at completion or controlled stop. Regrip count is averaged scenario-wise and is interpreted as an indicator of workflow complexity. A higher regrip count usually increases execution time because each regrip adds approach, release, repositioning, re-approach, gripping, and validation steps.

6.3 Experimental Setup and Scenarios

All trials were conducted in the implemented virtual press-brake cell described in Chapter 5. The evaluation uses the UR5e model, the configured TCP, the virtual press brake, the two-cup vacuum gripping concept, and the same general rectangular sheet-metal assumptions used during implementation. The process is evaluated as a simulated right-angle bending workflow; physical forming effects such as springback and bending force are outside the measurement scope.

Five scenario classes are evaluated. The first class is the nominal Side A direct-bending case. Side A represents a primary-side bend that is accessible from the initial workpiece orientation. It is the simplest valid case because the robot can generate a direct handling and bending sequence without flipping the sheet. This scenario verifies baseline program generation, bend-side interpretation, grip feasibility, and simulated execution.

The second class is the Side E opposite-side workflow. This is technically more demanding because the required bending face is not accessible to the press-brake tool in the initial orientation. Direct bending is therefore not possible: the tool would approach the wrong side of the sheet, and the resulting pose would not correspond to the required bend direction. To make the opposite face accessible, the sheet must be transferred to a flip configuration, released, re-approached, and gripped again from a suitable side. The regrip is required because the original grasp cannot simultaneously preserve reachability, maintain a valid sheet orientation, and allow the subsequent opposite-side bend. As a result, Side E workflows contain additional transfer, release, re-approach, flip, regrip, and validation steps. These steps are expected to increase execution time compared with Side A, but they are necessary for feasibility.

The third class covers workpiece geometry variation. In these trials, the sheet dimensions and thickness conditions vary within the rectangular workpiece scope. This scenario tests whether the system adapts by updating input parameters rather than requiring manual reteaching of the robot path. Geometry variation is important because changes in size and thickness affect grip-point feasibility, motion distance, and reachability.

The fourth class covers workpiece position changes. Two initial placement conditions are evaluated and referred to as Plane A and Plane B in the figures. Plane B repre-

sents a more constrained placement condition and is therefore expected to increase process difficulty. This scenario tests whether the automated system can absorb a placement change through parameterised planning, while the manual workflow requires greater reprogramming effort.

The fifth class covers failure-prone conditions. These trials intentionally test boundary behaviour where the system may not complete the workflow. Such cases are necessary because a strong evaluation must show not only successful operation, but also how the system behaves when the available workspace or fixture condition prevents feasible continuation.

6.4 Data Analysis and Graph Interpretation

6.4.1 Overall Time Performance

The main performance question is whether automation reduces the complete process effort. For this reason, the first comparison uses total process time rather than execution time alone. Across the 48 manual workflow trials, the average preparation time is 301,0 s, the average execution time is 40,5 s, and the average total process time is 341,5 s. Across the 48 automated workflow trials, the average preparation time is 11,1 s, the average execution time is 69,7 s, and the average total process time is 80,7 s.

These values show an important engineering trade-off. The automated workflow does not minimise every simulated robot movement. In fact, its average execution time is higher because the generated process includes structured approach motions, validation-driven transitions, and additional handling steps in complex scenarios. However, the preparation time decreases from 301,0 s to 11,1 s on average. As shown in Figure 6.1, this preparation-time reduction dominates the complete process result and reduces the average total process time from 341,5 s to 80,7 s.

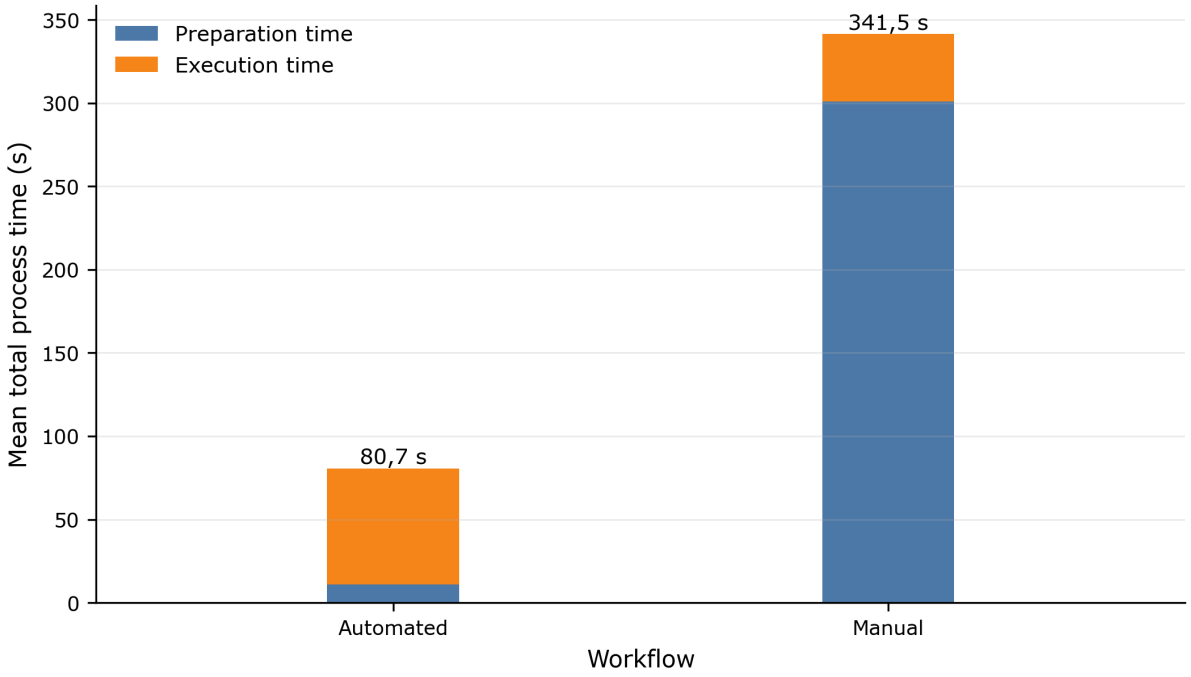


Figure 6.1: Comparison of mean total process time including preparation for manual and automated workflows. The figure separates preparation time and simulated execution time, showing that the automated workflow reduces the total process time mainly by reducing preparation effort.

Figure 6.1 supports the main hypothesis of this thesis. The relevant improvement is not that the robot always moves faster in the automated workflow; rather, the system avoids the large manual preparation effort required for waypoint and sequence adaptation. The average total-time reduction is approximately 76,4 %, calculated from the manual and automated total process times. The average preparation-time reduction is even larger, approximately 96,3 %. This indicates that the primary benefit of the proposed system lies in reducing engineering effort before execution.

6.4.2 Effect of Workpiece Position

The position-change scenario evaluates whether the workflow remains efficient when the initial placement of the workpiece changes. The two placement conditions differ in reachability difficulty. Plane A represents the easier placement condition, while Plane B introduces stronger workspace and fixture constraints. If a workflow is highly sensitive to initial placement, the process time should increase significantly when moving from Plane A to Plane B.

The measured averages show that the automated workflow is less sensitive to this position change. In the automated case, the average total process time increases from 74,5 s for Plane A to 87,0 s for Plane B. This is an increase of approximately 16,8 %. In the manual workflow, the average total process time increases from

180,5 s to 502,4 s, corresponding to an increase of approximately 178,3 %. Figure 6.2 shows this difference directly.

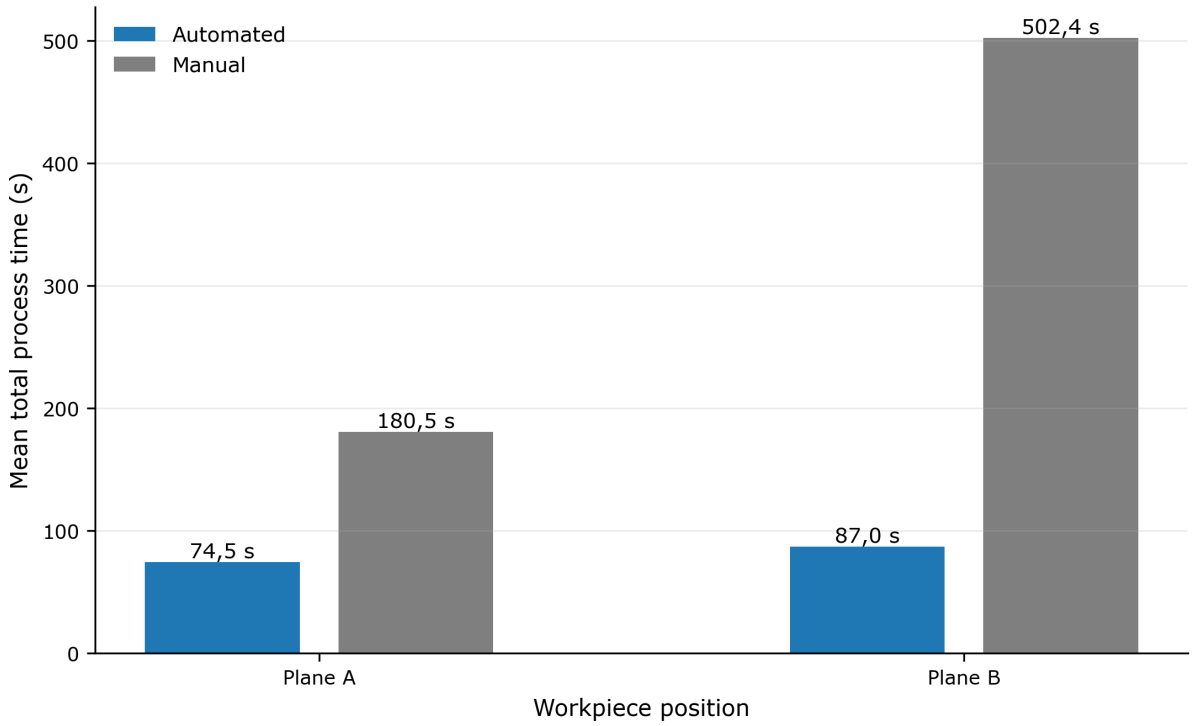


Figure 6.2: Effect of initial workpiece position on mean total process time for manual and automated workflows. Plane A and Plane B denote two placement conditions with different reachability constraints.

The trend in Figure 6.2 demonstrates that the manual workflow is strongly affected by position changes because a new placement requires additional human reasoning, waypoint adaptation, and sequence verification. The automated workflow is also affected, but the increase is much smaller because the changed placement is handled through parameterised planning. This result supports the adaptability objective defined in Chapter 3. It also explains why total process time is the more meaningful metric: position changes influence both preparation and execution, not only robot runtime.

6.4.3 Regrip and Opposite-Side Workflow Complexity

Regripping is not interpreted as an error in this evaluation. It is a planned response to reachability and orientation constraints. In Side A direct bending, the selected face is accessible from the initial orientation, so the workflow normally avoids intermediate regripping. In Side E opposite-side bending, the sheet must be flipped and regripped because the required face is initially inaccessible. Regrip actions therefore increase workflow complexity, but they also enable tasks that would otherwise be infeasible.

The relationship between regrip count and process time is visible in the constrained placement cases. A regrip adds at least one additional sequence of approach, release, retreat, re-approach, grip confirmation, and continuation motion. For this reason, scenarios with higher regrip counts are expected to have longer execution times. Figure 6.3 shows the mean regrip count grouped by workflow and placement condition.

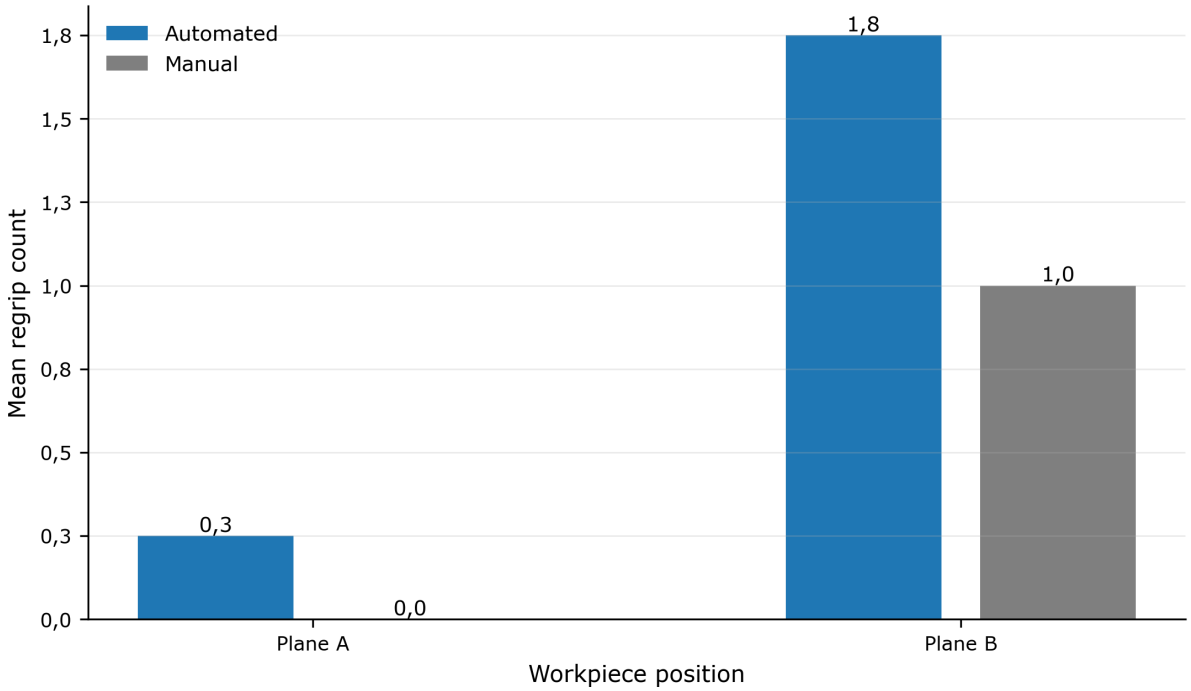


Figure 6.3: Mean regrip count for manual and automated workflows under different workpiece placement conditions. The count excludes the initial pickup and represents intermediate regrips required to maintain reachability and orientation feasibility.

Figure 6.3 shows that constrained placement increases the need for regripping. The automated workflow has a low mean regrip count in Plane A and a clearly higher value in Plane B. This is consistent with the time results in Figure 6.2: Plane B is more difficult, and additional regrip actions increase execution time. The conclusion is therefore not that regripping is undesirable in itself. Instead, the result shows that regripping is the mechanism by which the system preserves feasibility when direct handling is not sufficient.

The same reasoning applies to Side E. Direct bending cannot be used because the required face is not available to the press-brake tool. The flip sequence changes the face orientation, and the regrip establishes a new grasp that is compatible with the subsequent bend. The measured increase in execution time for complex workflows is therefore technically justified by the added manipulation steps. This is an important distinction: longer execution time in Side E does not indicate inefficient planning alone; it reflects a physically necessary handling sequence within the defined cell.

6.4.4 Planning-Time Interpretation

Planning time is evaluated as a secondary metric because it represents computational overhead rather than operator effort or robot runtime. The automated feasibility trials show that planning remains below one second on average in all evaluated scenario groups. The nominal case requires approximately 0,42 s on average, arbitrary initial orientation requires 0,48 s, the workspace-constrained case requires 0,54 s, the regrip-required case requires 0,58 s, and the opposite-side workflow requires approximately 0,61 s. The failure-prone case averages approximately 0,46 s because infeasible continuation is detected without completing a longer motion sequence.

These values show that computational planning time is not the limiting factor in the evaluated system. The practical benefit comes from reducing manual preparation time, while the practical cost appears mainly in additional simulated execution time for regrip and flip operations. For this reason, planning time is discussed as evidence of computational feasibility, but it is not used as the primary productivity metric.

6.5 Requirement Verification

The requirement verification links the measured outcomes to the functional, non-functional, and software requirements defined in Chapter 3. Tables 6.1 and 6.2 summarise the verification logic. Requirements that are mainly implementation-based are still evaluated here by linking them to the scenario in which the implemented behaviour becomes observable.

The verification shows that the strongest evidence is available for automated program generation, preparation-time reduction, direct bending, orientation handling, regrip insertion, and simulated execution in nominal conditions. The weaker evidence appears in opposite-side and failure-prone scenarios, where the system can identify and stop unsafe continuation but cannot always recover to a completed process. This distinction is important for an honest evaluation: safe failure handling supports the validation requirements, but it does not remove the need for improved recovery logic.

Req.	Verification Scenario	Observed Outcome	Evaluation Result
FR-00	Side A, geometry variation, position change	For valid evaluated inputs, the automated workflow reaches execution-ready state without manual waypoint construction; average automated preparation is 11,1 s.	Verified within simulation scope.
FR-01	Workspace-constrained and failure-prone cases	Infeasible or constrained cases are detected before unsafe continuation; failure-prone cases complete in 8 of 10 runs and stop safely in unresolved cases.	Verified for warning feedback, detection, and controlled response, not for universal recovery.
FR-02	Side A and Side E	Bend-side selection changes the generated workflow: Side A uses direct bending, while Side E requires flip and regrip logic.	Verified.
FR-03	Regrip-required and Side E cases	Regrip-required automated trials complete in 10 of 10 runs; Side E uses regrip as part of the flip workflow and stops safely if the remaining constraints cannot be resolved.	Verified for evaluated cases.
FR-04	Arbitrary initial orientation	Automated feasibility trials complete in 10 of 10 runs with approximately 0,48 s planning time on average.	Verified for tested orientation variations.
FR-05	Side E opposite-side workflow	Opposite-side workflow completes in 9 of 10 runs; one run stops in a controlled way because the remaining fixture/workspace condition is not resolvable.	Partly verified; capability shown, robustness gap remains.
FR-06	Side A, Side E, geometry variation	Successful trials indicate that grip feasibility checks provide executable grasp selections; unresolved cases are rejected rather than continued blindly.	Verified within simplified gripper and geometry assumptions.
FR-07	Failure-prone conditions	Controlled stops prevent unsafe continuation when constraints cannot be resolved.	Verified as safe behaviour; execution completion is not guaranteed in boundary cases.

Table 6.1: Functional requirement verification based on evaluated scenarios and observed trial outcomes.

Req.	Verification Scenario	Observed Outcome	Evaluation Result
NFR-01	Manual versus automated preparation	Average preparation time decreases from 301,0 s to 11,1 s; manual waypoint construction is not required in the automated workflow.	Verified.
NFR-02	Geometry and position variation	The automated workflow adapts to changed geometry and placement through configuration and operator input updates without source-code modification.	Verified within the tested rectangular sheet scope.
NFR-03	Failure-prone cases	Unresolved unsafe continuation results in controlled stop behaviour.	Verified for evaluated failure cases.
NFR-04	Repeated automated scenario runs	Nominal, orientation, workspace-constrained, and regrip-required scenarios each produce consistent completion classifications across 10 of 10 runs.	Verified as deterministic behaviour for tested conditions; broad statistical repeatability is not claimed.
NFR-05	Position-change and constrained placement	Plane B increases process difficulty and exposes fixture/-workspace sensitivity; the validation logic detects constrained continuation and supports controlled operator decisions.	Partly verified; constraint detection is shown, recovery remains limited.
SR-01–SR-03	All automated execution scenarios	Generated workflows execute in the virtual runtime using the configured communication and TCP assumptions.	Verified within the implemented virtual cell.

Table 6.2: Non-functional and software requirement verification based on evaluated scenarios and observed trial outcomes.

6.6 Success and Failure Analysis

The success and failure analysis focuses on the automated feasibility trials because these trials directly test whether the generated workflow completes under different scenario conditions. Each scenario consists of ten repeated runs. Figure 6.4 is introduced here because it summarises the boundary between validated operation and unresolved constraints.

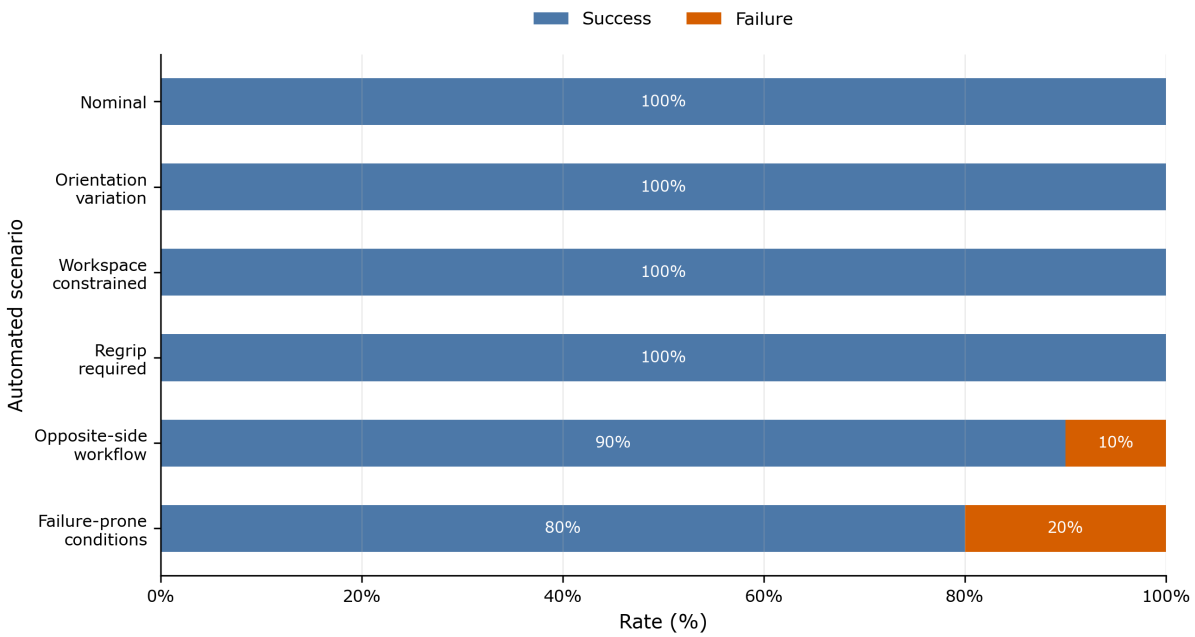


Figure 6.4: Automated success and failure rate across six feasibility scenarios. Each scenario contains ten repeated runs and is classified according to completion, failure, or controlled-stop behaviour.

As shown in Figure 6.4, the nominal feasible bending case, arbitrary initial orientation case, workspace-constrained configuration, and regrip-required bending case each achieve 100% success in the ten-run evaluation. These results indicate that the implemented pipeline is stable when the scenario remains within the intended planning envelope. The regrip-required case is particularly important because it shows that regrip insertion is not only implemented, but also executable in the tested conditions.

The opposite-side workflow achieves 90% success, with 9 of 10 runs completing successfully. The failed run is not interpreted as uncontrolled system behaviour. It represents a case where the workflow encounters a fixture or workspace constraint after partial planning and cannot continue to a valid completion. This result is significant because Side E is more complex than Side A: it requires flipping, regripping, revalidation, and continuation from a changed sheet orientation. The 90% success rate therefore validates the implemented concept, while also showing that opposite-side recovery remains less robust than direct bending.

The failure-prone scenario achieves 80% success, with 8 of 10 runs completed and 2 unresolved cases stopped. These failures define the boundary of the current implementation. In both cases, the system prevents continuation rather than forcing an infeasible motion. This is why controlled stops are interpreted as system safety success but execution failure. From an evaluation perspective, this behaviour is preferable to uncontrolled continuation, but it still shows that the current system requires stronger recovery strategies for difficult boundary cases.

6.7 Gap Analysis

The evaluation confirms several strengths. First, the preparation-time reduction is substantial: the average automated preparation time is 11,1 s, compared with 301,0 s for the manual workflow. This directly supports the objective of reducing manual programming effort. Second, the automated workflow remains much less sensitive to workpiece position changes. The total process time increases by approximately 16,8 % from Plane A to Plane B in the automated workflow, while the manual workflow increases by approximately 178,3 %. Third, the automated feasibility trials show full success in the nominal, orientation, workspace-constrained, and regrip-required scenario groups.

However, the evaluation also reveals clear gaps. The first gap is that physical bending quality is not validated. The chapter evaluates simulated robot-program feasibility, but it does not measure bend angle, springback, deformation, real gripper slip, or forming force. Therefore, the results support simulation-based program generation but not production-ready bending accuracy.

The second gap is the reduced robustness of the opposite-side workflow. Side E achieves 9 successful runs out of 10, while the nominal and regrip-required cases achieve 10 out of 10. This shows that flipping and regripping are implemented successfully in most cases, but the workflow is more sensitive to fixture and workspace constraints than direct bending.

The third gap is limited recovery in failure-prone conditions. The system correctly stops unresolved cases, but an 80% success rate means that boundary conditions still exceed the recovery capability of the current planner. The system detects the problem, which is positive for safety, but it does not always find an alternative feasible strategy.

The fourth gap concerns the manual-versus-automated comparison. The measured time reduction is strong, but the evaluation does not isolate operator learning effects, multiple operator skill levels, or long-term usability over a large workpiece family. The result therefore supports reduced workflow effort in the evaluated scenarios, but it should not be interpreted as a complete human-factors validation.

The fifth gap is collision-free industrial deployment. The virtual execution verifies generated motion under the implemented simulation assumptions, but it does not replace physical commissioning, calibrated cell modelling, or certified safety validation. This limitation is consistent with the scope of the thesis, but it remains essential for future transfer to a real cell.

6.8 Improvement Plan

The improvement plan follows directly from the observed gaps.

From a technical perspective, the first priority is stronger recovery for Side E and constrained placement cases. The current workflow can detect infeasible continuation, but it should be extended with a broader regrip search and alternative flip approaches when the first strategy fails. This would address the observed 90% success rate in opposite-side bending and improve robustness under fixture constraints.

The second technical improvement is collision-aware path validation. The implemented system validates reachability and workflow feasibility within the virtual cell, but full collision checking would improve confidence before physical deployment. This improvement should be integrated before the command-dispatch stage so that a motion is rejected or replanned before execution.

The third technical improvement is sensor-based pose correction. Position-change trials show that placement affects process time and difficulty. A perception or feedback mechanism could reduce dependence on assumed sheet pose and allow the system to correct deviations before planning. This would strengthen both NFR-05 and the practical transferability of the system.

From a validation perspective, the next step is to increase the number and diversity of trials. Each major scenario should be repeated across more workpiece dimensions, bend-side combinations, placement conditions, and regrip configurations. The current evaluation is sufficient for simulation-based thesis validation, but larger trial matrices would allow stronger claims about robustness.

Physical validation should also be added in future work. Such validation must measure bend angle accuracy, springback, gripper slip, sheet deformation, cycle repeatability, and safety behaviour on real hardware. Without these measurements, the system should continue to be described as simulation-validated rather than production-validated.

Finally, the evaluation should be made more granular. Execution time should be separated into approach, pickup, transfer, regrip, flip, bending-positioning, retreat, and unloading components. This would make it easier to identify which part of the workflow causes the largest time increase in complex scenarios. Similarly, manual preparation trials should include multiple operators to evaluate whether the time reduction remains stable across different skill levels.

6.9 Summary of Findings

The evaluation supports the thesis hypothesis within the defined simulation-based scope. The automated workflow reduces average preparation time from 301,0 s to 11,1 s and reduces average total process time from 341,5 s to 80,7 s. The main benefit is therefore not faster robot motion in every case, but the removal of manual waypoint and sequence preparation effort.

Scenario analysis shows that the automated system handles nominal Side A bending, geometry variation, position changes, orientation variation, and regrip-required cases within the evaluated conditions. Position changes affect the automated workflow much less than the manual workflow, which demonstrates the value of parameterised planning. Regrip analysis further shows that additional handling steps increase execution time, but they are necessary to maintain feasibility in constrained and opposite-side workflows.

The success and failure analysis provides a balanced result. Nominal, orientation, workspace-constrained, and regrip-required automated scenarios complete successfully in all ten evaluated runs. The Side E opposite-side workflow completes in 9 of 10 runs, while failure-prone conditions complete in 8 of 10 runs. Controlled stops are interpreted as safe system behaviour, but they remain execution failures because the process objective is not completed.

Overall, the system demonstrates a clear reduction in preparation effort and feasible simulated execution for the tested workflow classes. At the same time, the evaluation identifies remaining gaps in physical validation, collision-aware deployment, opposite-side robustness, and boundary-case recovery. These limitations do not invalidate the implemented concept; rather, they define the next development steps required before transferring the approach from simulation-based validation to industrial commissioning.

7 Conclusion and Future Work

7.1 Conclusion

This thesis presented the development and evaluation of an automated robotic system for press brake sheet metal bending. The objective was to reduce manual programming effort while improving adaptability and execution reliability in industrial bending operations.

The analysis of the state of the art in Chapter 2 identified key limitations in existing approaches, including the continued need for manual programming input, limited adaptability to changing workpiece conditions, and a lack of integrated planning frameworks. Based on these observations, a set of functional and non-functional requirements was defined in Chapter 3, focusing on automation, flexibility, and system robustness.

In Chapter 4, multiple solution concepts were evaluated, leading to the selection of Concept B as the most suitable approach. This concept introduced a structured planning pipeline that integrates orientation determination, grip planning, regrip handling, and trajectory generation into a unified system.

Chapter 5 described the complete implementation of the selected concept. The system was realised as a modular software architecture with clearly defined layers, enabling separation of concerns and maintainability. Key functionalities such as constraint-based grip planning, automatic regrip determination, and structured trajectory generation were implemented to support automated operation within the defined thesis scope.

The evaluation presented in Chapter 6 used repeated workflow and feasibility trials within the virtual study. The trial set covered nominal direct bending on Side A, opposite-side bending on Side E, geometry variation, workpiece position variation, regrip-required workflows, and failure-prone conditions. This allowed the evaluation to compare manual and automated process effort while also verifying simulated execution behaviour across the main scenario classes.

A comparison with the manual drag&bot workflow highlighted the reduction in programming and preparation effort. The manual workflow required an average preparation time of 301,0 s, whereas the automated workflow required 11,1 s. The average total process time decreased from 341,5 s for the manual workflow to 80,7 s for the automated workflow. This corresponds to an approximate preparation-time reduction of 96,3 % and an approximate total-time reduction of 76,4 %.

The research hypothesis stated in Section 1.4 was that automated robot-program generation from workpiece geometry reduces preparation time while maintaining feasible simulated execution. The evaluation in Chapter 6 supports this hypothesis within the defined simulation-based scope. Nominal, orientation-variation, workspace-constrained, and regrip-required automated scenarios completed successfully in all evaluated runs, while opposite-side and failure-prone cases identified the remaining robustness limits. Physical robot validation and broader industrial repeatability testing remain future work.

The work was developed from the requirement set into a configuration-driven planning pipeline and assessed in the drag&bot-based virtual simulation environment, keeping implementation, testing, and evaluation within the same controlled setup.

Overall, the developed system provides a structured and adaptable solution for robotic press brake bending. By combining automated decision-making with structured validation, the approach addresses key limitations of existing methods and contributes to the advancement of flexible robotic manufacturing systems within the scope of simulation-based development and evaluation.

7.2 Contributions

The main contributions of this thesis can be summarised as follows:

1. Development of an automated planning pipeline for press brake bending, integrating orientation determination, grip planning, regrip handling, and trajectory generation
2. Introduction of a constraint-based grip planning method that ensures both geometric feasibility and sufficient holding force
3. Design of an adaptive regrip strategy that determines required reorientation dynamically without predefined sequences
4. Implementation of a structured trajectory generation approach that ensures smooth and feasible robot motion
5. Integration of validation mechanisms for safety and feasibility, including pre-execution checks and runtime monitoring
6. Demonstration of reduced programming effort through repeated manual-versus-automated workflow trials

These contributions collectively address the challenges identified in the state of the art and provide a foundation for further development of automated robotic bending systems.

7.3 Limitations

Despite the successful implementation and evaluation of the system, several limitations remain.

The evaluation was conducted entirely within a virtual simulation environment. While this enables controlled testing, it does not fully capture real-world factors such as sensor noise, calibration inaccuracies, and mechanical tolerances.

In addition, the system does not include comprehensive collision detection or advanced dynamic modelling of contact interactions. These aspects are important for ensuring safe operation in more complex or constrained environments.

Furthermore, the current implementation focuses on a specific class of sheet metal geometries and bending operations. Although the evaluation includes repeated workflow and feasibility trials, it remains simulation-based and does not establish production readiness for a commissioned industrial cell. More complex geometries or multi-step industrial processes may require additional extensions to the planning pipeline.

7.4 Future Work

Based on the identified limitations, several directions for future work can be proposed.

Although the developed system achieves a functional and structured approach to robotic sheet handling and bending, it is inherently constrained by a series of simplifying assumptions introduced during the design phase. These constraints, while beneficial for ensuring stability and predictability, limit the system's applicability to more complex and realistic industrial scenarios. Consequently, several avenues for future work emerge, aimed at improving flexibility, intelligence, and robustness.

A key limitation of the current system lies in its strict assumption of rectangular sheet geometries aligned with the global coordinate frame. In practical manufacturing environments, components often exhibit irregular shapes and arbitrary orientations. Future work should therefore focus on generalizing the geometric representation to support non-rectangular profiles, including polygonal and free-form geometries. Additionally, enabling the system to operate independently of fixed axis alignment would significantly enhance its adaptability. This would require a more advanced geometric abstraction layer capable of handling arbitrary edge definitions and diverse feature types, including non-circular cutouts.

The bending capabilities of the system also present a significant opportunity for advancement. The restriction to single, 90-degree bends per edge simplifies the

planning process but does not reflect the complexity of real-world fabrication tasks. Future developments should introduce support for variable bend angles and multiple bends along a single edge. More importantly, this necessitates the formulation of an intelligent bend sequence planning framework capable of optimising operation order, minimising regrip requirements, and ensuring process feasibility. Such a framework could leverage constraint-based optimisation or heuristic search methods to generate efficient and manufacturable bending strategies.

From a robotics perspective, motion planning remains an area with considerable scope for improvement. The current system demonstrates reduced efficiency when handling certain orientations, particularly those involving significant rotational transformations. Future work should integrate advanced motion planning techniques that ensure collision-free, time-optimal trajectories. Furthermore, introducing adaptive calibration mechanisms would allow the system to remain functional even when the robot base or surrounding fixtures are repositioned, thereby reducing dependency on manual reconfiguration. The incorporation of learning-based methods could further enhance performance by enabling the system to refine its motion strategies through repeated execution.

The gripping subsystem, currently limited to vacuum-based handling, represents another constraint on system versatility. While effective for flat and uniform surfaces, vacuum gripping is not universally applicable. Expanding the system to support multiple gripping modalities, including mechanical and magnetic solutions, would broaden its operational scope. In addition, the integration of sensor feedback, such as force or slip detection, would enable real-time validation of grasp stability and improve overall reliability.

Regrip operations in the present implementation are restricted to discrete rotational increments, which limits the flexibility of part orientation. Future work should explore continuous orientation control and develop more sophisticated regrip planning strategies aimed at minimising intermediate handling steps. Streamlining opposite-side operations into a unified workflow, rather than relying on separate scripts, would further improve system efficiency and maintainability.

Safety considerations represent a critical gap in the current system. The absence of collision detection and trajectory validation poses a significant limitation for deployment in real-world environments. Future enhancements should prioritise the integration of real-time collision avoidance mechanisms and safe motion planning algorithms. Expanding the scope of the safety monitoring system to account for dynamic interactions between the robot, workpieces, and surrounding infrastructure would also be essential.

Finally, the system’s reliance on fixed object identifiers and predefined configurations reduces its robustness in dynamic environments. Future work should investigate the integration of perception-driven approaches, such as vision-based object

recognition and tracking, to enable more flexible and autonomous operation. Additionally, incorporating error detection and recovery strategies would allow the system to handle unexpected deviations more effectively, thereby improving overall resilience.

In conclusion, future developments should aim to transition the system from a rigid, rule-based framework toward a more adaptive and intelligent architecture. By addressing limitations in geometry handling, bending flexibility, motion planning, gripping strategies, regrip operations, safety, and system adaptability, the proposed solution can evolve into a more comprehensive and industrially viable robotic automation platform.

7.5 Final Remarks

This thesis demonstrates that automated planning and execution of robotic press brake bending is feasible within the evaluated simulation scenarios and beneficial for reducing preparation effort. By reducing manual programming effort across the repeated workflow comparison, the developed system provides a focused foundation for more efficient robotic bending workflows.

The presented approach provides a structured foundation for future research and development in robotic bending and related automation tasks.

Bibliography

[1] S. K. Gupta, D. A. Bourne, K. Kim, and S. Krishnan, "Automated process planning for sheet metal bending operations," *Journal of Manufacturing Systems*, vol. 17, no. 5, pp. 338–360, 1998.

[2] S. Aomura and A. Koguchi, "Optimized bending sequences of sheet metal bending by robot," *Robotics and Computer-Integrated Manufacturing*, vol. 18, no. 1, pp. 29–39, 2002.

[3] Z. Pan, J. Polden, N. Larkin, S. van Duin, and J. Norrish, "Recent progress on programming methods for industrial robots," *Robotics and Computer-Integrated Manufacturing*, vol. 28, no. 2, pp. 87–94, 2012.

[4] P. Neto, "Off-line programming and simulation from cad drawings: Robot-assisted sheet metal bending," in *Proceedings of the 39th Annual Conference of the IEEE Industrial Electronics Society (IECON)*, Department of Mechanical Engineering, CEMUC, Vienna, Austria, 2013, pp. 4235–4240.

[5] Universal Robots A/S, *UR5e Technical Specifications*, Odense, Denmark, 2026. Accessed: Apr. 20, 2026. [Online]. Available: https://www.universal-robots.com/manuals/EN/HTML/SW5_25_1/Content/prod-usr-man/complianceUR5e/H_g5_sections/appendix_g5/tech_spec_sheet.htm

[6] *Robots and robotic devices – vocabulary*, International Organization for Standardization, 2012.

[7] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2005, ISBN: 978-0-13-123629-5.

[8] M. R. Pedersen et al., "Robot skills for manufacturing: From concept to industrial deployment," *Robotics and Computer-Integrated Manufacturing*, vol. 37, pp. 282–291, 2016.

[9] S. Kalpakjian and S. R. Schmid, *Manufacturing Engineering and Technology*, 7th. Pearson Education, 2014.

[10] M. P. Groover, *Automation, Production Systems, and Computer-Integrated Manufacturing*, 4th. Pearson, 2016.

[11] Acieta, *Robotic press brake, welding & palletizing cell for cotterman*, 2018. Accessed: Mar. 27, 2026. [Online]. Available: https://www.acieta.com/automation-application/robotically-integrated-bending-solutions-ribs/

[12] B. Siciliano and O. Khatib, Eds., *Springer Handbook of Robotics*, 2nd ed. Cham: Springer, 2016, ISBN: 978-3-319-32550-7.

[13] F. Tao, H. Zhang, A. Liu, and A. Nee, "Digital twin in industry: State-of-the-art," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019.

[14] T. Lozano-Pérez, "Robot programming," *Proceedings of the IEEE*, vol. 71, no. 7, pp. 821–841, 1983.

[15] J. Bohg, A. Morales, T. Asfour, and D. Kragic, "Data-driven grasp synthesis – a survey," *IEEE Transactions on Robotics*, vol. 30, no. 2, pp. 289–309, 2014.

[16] W. Wan and K. Harada, "Regrasp planning using 10,000s of grasps," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 1929–1936.

[17] F. Gabriel, M. Fahning, J. Meiners, F. Dietrich, and K. Dröder, "Modeling of vacuum grippers for the design of energy efficient vacuum-based handling processes," *Production Engineering*, vol. 14, pp. 545–554, 2020.

[18] A. Gasparetto, P. Boscariol, A. Lanzutti, and R. Vidoni, "Trajectory planning in robotics," *Mathematics in Computer Science*, vol. 6, no. 3, pp. 269–279, 2012.

[19] RoboDK, *Robodk: Simulation and offline programming software for industrial robots*, 2024. Accessed: Mar. 27, 2026. [Online]. Available: `https://robodk.com`

[20] drag&bot GmbH. "About us," Accessed: Apr. 30, 2026. [Online]. Available: `https://www.dragandbot.com/en/company/about-us`

[21] drag&bot GmbH. "Anyrobot interface documentation." Ulm, Germany, Accessed: Mar. 2, 2026. [Online]. Available: `https://docs.dragandbot.com/anyrobot-interface`

[22] KEBA Group AG. "Keba acquires german software company drag and bot gmbh," Accessed: Apr. 30, 2026. [Online]. Available: `https://www.keba.com/en/news/corporate/drag-and-bot`

[23] G. Pahl, W. Beitz, J. Feldhusen, and K. H. Grote, *Engineering Design: A Systematic Approach*, 3rd ed. London: Springer, 2007.

[24] T. Lechler, E. Fischer, M. Metzner, A. Mayr, and J. Franke, "Virtual commissioning – scientific review and exploratory use cases in advanced production systems," *Procedia CIRP*, vol. 81, pp. 1125–1130, 2019.

[25] M. Quigley et al., "ROS: an Open-Source Robot Operating System," in *Proceedings of the ICRA Workshop on Open Source Software*, Kobe, Japan, 2009.

[26] D. Coleman, I. A. Sucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: A MoveIt! case study," *Journal of Software Engineering for Robotics*, vol. 5, no. 1, pp. 3–16, 2014.

[27] S. Lindgren, *Unity for ros2 documentation*, `https://unity-ros2.readthedocs.io/en/latest/`, 2019.

[28] C. Cadena et al., "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016.

[29] V. Pimentel and B. G. Nickerson, "Communicating and displaying real-time data with websocket," *IEEE Internet Computing*, vol. 16, no. 4, pp. 45–53, 2012.

[30] C. Crick, G. Jay, S. Osentoski, B. Pitzer, and O. C. Jenkins, "Rosbridge: Ros for non-ros users," in *Robotics Research*, Springer, 2017, pp. 493–504.

[31] Y. Zhang et al., "Invertible liquid neural network-based learning of inverse kinematics and dynamics for robotic manipulators," *Scientific Reports*, vol. 15, p. 42 311, 2025.

[32] OpenStax. "Calculus volume 3: Vectors in three dimensions." Open textbook, Rice University, Accessed: May 1, 2026. [Online]. Available: https://openstax.org/books/calculus-volume-3/pages/2-2-vectors-in-three-dimensions

[33] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017, ISBN: 9781107156302.

[34] Schmalz GmbH, *Vacuum handling systems: Design principles and force calculation*, Technical reference for vacuum gripping force models, 2020.

[35] LibreTexts. "Mathematical foundations: Linear interpolation," Accessed: May 1, 2026. [Online]. Available: https://eng.libretexts.org/Bookshelves/Electrical_Engineering/Electronics/Microwave_and_RF_Design_II_-_Transmission_Lines_(Steer)/01%3A_Introduction_to_Distributed_Microwave_Circuits/1.A%3A_Appendix-_Mathematical_Foundations

[36] K. Shoemake, "Animating rotation with quaternion curves," in *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '85)*, ACM, 1985, pp. 245–254.

[37] T. Flash and N. Hogan, "The coordination of arm movements: An experimentally confirmed mathematical model," *The Journal of Neuroscience*, vol. 5, no. 7, pp. 1688–1703, 1985.

[38] A.-N. Sharkawy, "Minimum jerk trajectory generation for straight and curved movements: Mathematical analysis," *Advances in Robotics: Reviews*, 2021.

[39] *Failure modes and effects analysis (fmea and fmeca)*, International Electrotechnical Commission, 2018. Accessed: May 1, 2026. [Online]. Available: https://webstore.iec.ch/en/publication/26359

Appendix

This appendix documents the AI-assisted material used during thesis preparation and provides supplementary evaluation and traceability information. In line with the applicable thesis guidelines, large software artefacts and machine-readable logs are retained in the electronic submission rather than reproduced in full in the printed appendix.

A.1 AI Usage Documentation

Generative AI assistance was used in a limited and supportive capacity during document preparation. The support was restricted to language refinement, grammar correction, structure checking, formatting support, LaTeX assistance, and consistency review. It was not used to generate experimental data, measurement values, evaluation outcomes, system requirements, implementation logic, or scientific conclusions.

All AI-assisted text and formatting suggestions were reviewed by the author before use. The author retained responsibility for the technical content, interpretation of results, and final thesis wording.

A.2 Prompt Log and Supporting Files

A machine-readable prompt log is retained in the electronic submission as `appendix/ai_prompt_log.txt`. The log is written in a YAML-compatible plain-text structure and records the retained AI usage prompts, the AI tool, provider, model family, purpose, and associated files where applicable. Supporting thesis source files referenced by those entries are included in the submission package.

A.3 Supplementary Evaluation Documentation

Chapter 6 presents the simulation-based evaluation of the implemented system. The evaluation is based on repeated manual-versus-automated workflow trials and automated scenario trials covering nominal Side A bending, opposite-side Side E bending, geometry variation, workpiece position variation, regrip-required workflows, and failure-prone conditions. Table 1 summarises the evaluation groups and

the corresponding measured quantities.

Evaluation Group	Scenario Coverage	Purpose	Measured Metrics
Manual workflow trials	Side A, Side E, geometry variation, and workpiece position variation	Quantify manual preparation effort, simulated execution time, and total process time for comparison with automated generation.	Preparation, execution, total time, regrip count
Automated workflow trials	Side A, Side E, geometry variation, and workpiece position variation	Quantify automated preparation effort, simulated execution time, and total process time under the same workflow classes.	Preparation, execution, total time, regrip count
Automated feasibility trials	Nominal, orientation variation, workspace-constrained, regrip-required, opposite-side, and failure-prone scenarios	Verify scenario-wise completion, controlled-stop behaviour, planning time, and robustness limits of the generated workflow.	Success rate, failure rate, planning time, controlled stop

Table 1: Evaluation trial structure used for the final simulation-based validation.

The table is included to make the measurement structure transparent without duplicating the full machine-readable trial records in the printed appendix. The complete numerical analysis and interpretation are provided in Chapter 6.

A.4 Requirement Traceability and Compliance Tables

The following tables provide the detailed requirement traceability and implementation compliance mappings referenced from Chapters 3 to 5. They are placed in the appendix to keep the main thesis flow focused while preserving the audit trail between requirements, implementation modules, and validation activities. The mappings are aligned with the scenario-based verification presented in Chapter 6. Tables 2 and 3 provide the traceability mapping, while Tables 4 and 5 summarise the corresponding implementation compliance mapping.

Req. ID	Requirement	Implementation	Verification
FR-00	Generate execution-ready drag&bot programs from geometry, configuration data, and bend-side input	Integrated planning pipeline for configuration processing, orientation selection, grip planning, trajectory generation, and execution logic	Manual-versus-automated workflow comparison and automated scenario validation in Chapter 6
FR-01	Detect unreachable or infeasible target poses before command dispatch	Reachability checks and admissibility validation	Workspace-constrained and failure-prone scenario validation with controlled-stop classification
FR-02	Bend-side selection before automated program generation or execution	Operator dialog and configuration input logic	Side A direct-bending and Side E opposite-side workflow validation
FR-03	Automatically insert a regrip operation when the next target pose is not directly reachable	Regrip cycle logic with IO and attach-detach control	Regrip-required and Side E opposite-side scenario validation
FR-04	Automatic orientation validation and correction for changed initial workpiece orientation	Orientation conversion, rotation normalisation, and strategy selection	Orientation-variation scenario validation
FR-05	Automatically generate flip and reorientation workflow for opposite-side bending	Opposite-side workflow with sheet flip, regrip, revalidation, and controlled-stop handling if recovery fails	Side E opposite-side workflow validation
FR-06	Determine grip positions that satisfy edge-clearance, hole-avoidance, and holding-force checks	Grip planning with edge-clearance, spacing, and vacuum-force checks	TC-06 grip-planning test and functional validation of valid and rejected grips
FR-07	Detect unsafe conditions and replan or stop execution before unsafe continuation	Safety validation, pre-execution checks, replanning, and controlled stop logic	Failure-prone scenario validation and controlled-stop classification

Table 2: Requirement Traceability Matrix – functional requirements (FR-00 to FR-07) mapped to implementation modules and verification scenarios.

Req. ID	Requirement	Implementation	Verification
NFR-01	Limit operator interaction to configuration or input selection and execution confirmation	Bend-side selection and execution confirmation only	Usability validation and workflow comparison without manual waypoint or block programming
NFR-02	Configure workpiece geometry, process settings, and safety parameters without source-code modification	Configuration reader and YAML-based process configuration	Geometry-variation and position-change scenario validation through configuration or operator input updates
NFR-03	Provide warning or error feedback before continuation for unsafe or infeasible conditions	Warning generation and operator notification logic	Warning message validation
NFR-04	Deterministic execution sequence for a given configuration	Deterministic execution loop with automatic waypoint generation and logging	Repeated automated scenario validation using completion, failure, or controlled-stop classifications
NFR-05	Check fixture pose deviations against defined positional and angular tolerance limits	Configurable pose-deviation threshold and perturbation handling	Fixture perturbation and constrained-placement validation using 5 mm and 0,10 rad thresholds
SR-01	Connect to and validate generated programs in the configured drag&bot virtual execution environment	Runtime client connection to the configured drag&bot environment	Platform integration check
SR-02	Route planning-to-runtime communication through the AnyRobot WebSocket interface	WebSocket-based communication layer for structured command exchange	Interface integration check
SR-03	Generate all motion targets using the configured TCP identifier `project_3_tcp`	TCP-based transformations in trajectory generation and execution	TCP configuration check

Table 3: Requirement Traceability Matrix – non-functional requirements (NFR-01 to NFR-05) and system requirements (SR-01 to SR-03) mapped to implementation modules and verification scenarios.

Requirement ID	Requirement	Implementation Mapping
FR-00	Generate execution-ready drag&bot programs from geometry, configuration data, and bend-side input	Integrated planning pipeline covering configuration handling, orientation (Section 5.5), grip planning (Section 5.5.3), regrip planning (Section 5.6.1), and trajectory generation (Section 5.7)
FR-01	Detect unreachable or infeasible target poses before command dispatch	Reachability validation, pre-execution checks, and controlled error handling before motion dispatch (Section 5.4)
FR-02	Allow bend-side selection before automated program generation or execution	Implemented through bend-side selection, execution confirmation, and drag&bot cockpit interaction (Section 5.2.3)
FR-03	Automatically insert a regrip operation when the next target pose is not directly reachable	Dynamic regrip planning and execution based on kinematic feasibility evaluation (Section 5.6.1)
FR-04	Handle changed initial workpiece orientation through automatic validation and correction	Three-level orientation strategy with admissibility validation and fallback logic (Section 5.5)
FR-05	Generate the flip and reorientation workflow for opposite-side bending	Flip workflow, regrip handling, revalidation, and reuse of the planning pipeline for secondary operations (Section 5.6.2)
FR-06	Determine grip positions that satisfy edge-clearance, hole-avoidance, and holding-force checks	Constraint-based grip planning with geometric filtering and force validation (Section 5.5.3)
FR-07	Detect unsafe conditions and replan or stop execution before unsafe continuation	Fixture pose validation, operator confirmation, replanning logic, and controlled-stop handling (Section 5.4)

Table 4: Requirement compliance mapping – functional requirements

Requirement ID	Requirement	Implementation Mapping
NFR-01	Limit operator interaction to configuration or input selection and execution confirmation	Bend-side selection, execution confirmation, and drag&bot cockpit interaction (Section 5.2.3)
NFR-02	Configure workpiece geometry, process settings, and safety parameters without source-code modification	Centralised YAML configuration architecture, operator input updates, and parameter-driven system design (Section 5.2.2)
NFR-03	Provide warning or error feedback before continuation for unsafe or infeasible conditions	Warning generation, modal operator notifications, and controlled stop handling (Section 5.4)
NFR-04	Deterministic execution sequence for a given configuration	Structured modular pipeline and fixed execution order across system layers (Sections 5.2–5.8); repeated automated scenario classifications are evaluated in Chapter 6
NFR-05	Check fixture pose deviations against defined positional and angular tolerance limits	Configurable pose-deviation thresholds, fixture deviation checks, and controlled-stop handling (Section 5.4)
SR-01	Connection to and validation in the configured drag&bot virtual execution environment	Implemented through the configured drag&bot virtual runtime environment for simulated execution and validation of generated robot programs (Sections 3.2.3 and 5.2)
SR-02	Communication through the AnyRobot WebSocket execution interface	Implemented through the AnyRobot interface using authenticated WebSocket-based communication for motion commands, service calls, input-output operations, and execution feedback (Sections 3.2.3 and 5.2.2)
SR-03	Use of the configured TCP identifier `project_3_tcp` for all motion targets	Implemented through the configured TCP definition `project_3_tcp`, which is used consistently for generated poses and trajectory execution (Sections 3.2.3 and 5.3)

Table 5: Requirement compliance mapping – non-functional and system requirements

A.5 Hardware Implementation Report

The implemented system was evaluated in a configured virtual press-brake cell rather than on a physical production installation. This section records the hardware and platform configuration used for the simulation-based implementation so that the scope of the results remains transparent and consistent with Chapter 6.

Configuration item	**Implementation report**
drag&bot platform	drag&bot runtime package 3.6.0, used as the virtual robot-programming and execution environment for the thesis implementation.
Runtime interface	AnyRobot WebSocket-based interface used for command exchange between the planning software and the drag&bot runtime.
Simulation setup	Virtual press-brake cell containing the UR5e robot model, press brake, orientation table, flip station, vacuum gripper, and tracked sheet object.
Robot configuration	UR5e manipulator model with six degrees of freedom, used for all simulated handling and bending-related motion sequences.
TCP definition	Configured TCP `project_3_tcp` with offsets $x = 0{,}000$ m, $y = 0{,}034$ m, $z = 0{,}194$ m, $RZ = -90{,}0°$, $RY = -75{,}0°$, and $RX = 0{,}0°$.
Gripper setup	Two-cup vacuum gripper represented in the simulation by digital input-output control, vacuum activation and release states, and kinematic object attachment-detachment logic.
System configuration	Workpiece dimensions, bend-side selection, safety limits, admissible workspace zones, gripper parameters, and runtime settings are loaded from configuration data before execution.

Table 6: Hardware and platform configuration used for the simulation-based implementation.

This configuration supports the reported simulation-based validation, but it does not replace physical commissioning. Physical robot validation, bend-angle measurement, gripper slip testing, forming-force assessment, and certified safety verification remain outside the scope of the present thesis.